Claude AI Mastery — Beginner End-to-End Notes

A plain, beginner-friendly transcription of the course notes. Every section below is drawn only from the 46 extracted transcripts in this folder.

Table of Contents

Demo 1: What is Claude? Introducing Anthropic's AI Assistant, Claude Code, and Claude Cowork 8
What you'll learn 8
Step-by-step notes 8
Key takeaways 10
Demo 2: Exploring the Claude Desktop App, Pricing Plans, and Claude's Three Modes 11
What you'll learn 11
Step-by-step notes 11
Key takeaways 13
Demo 3: Hands-On with Claude Cowork — Connectors, Gmail Automation, and Task Prompts 15
What you'll learn 15
Step-by-step notes 15
Key takeaways 18
Demo 4: Managing Credits, Global Instructions, and Comparing Claude vs ChatGPT for Analysis 19
What you'll learn 19
Step-by-step notes 19
Key takeaways 21
Demo 5: Real-World Shopping/Travel Comparisons and Building Interactive Artifacts 22
What you'll learn 22
Step-by-step notes 22
Key takeaways 25
Demo 6: Claude Artifacts — Building Reusable Mini-Apps, Calculators and Dashboards 27
What you'll learn 27
Step-by-step notes 27

Key takeaways 30
Demo 7: Introduction to Claude Skills — Teaching Claude Your Instructions Once 31
What you'll learn 31
Step-by-step notes 31
Key takeaways 33
Demo 8: Creating and Using Custom Claude Skills (Hands-On) 34
What you'll learn 34
Step-by-step notes 34
Key takeaways 36
Demo 9: Claude Memory Settings and Projects 38
What you'll learn 38
Step-by-step notes 38
Key takeaways 41
Demo 10: Claude Styles — Controlling How Claude Talks, Writes, and Responds 42
What you'll learn 42
Step-by-step notes 42
Key takeaways 44
Demo 11: Claude Connectors — Turning Claude into a Digital Assistant 45
What you'll learn 45
Step-by-step notes 45
Key takeaways 46
Demo 12: More Connector Examples — Google Drive, Chrome Extension, and Automated Email Replies 48
What you'll learn 48
Step-by-step notes 48
Key takeaways 49
Demo 13: Chrome Extension in Practice, and a Deep Dive into Data Analysis with Claude 51
What you'll learn 51
Step-by-step notes 51
Key takeaways 53
Demo 14: Introducing Claude Cowork — Your AI "Digital Employee" on the Desktop 55

What you'll learn 55
Step-by-step notes 55
Key takeaways 57
Demo 15: Cowork in Action — Real Business Documents, Web Scraping, and the CCOR Prompting Framework 58
What you'll learn 58
Step-by-step notes 58
Key takeaways 60
Demo 16: Balanced Life, Debugging n8n Workflows with Claude, and the CCOR Prompt Format 62
What you'll learn 62
Step-by-step notes 62
Key takeaways 64
Demo 17: Understanding the claude.md File (Project-Specific Instructions for Claude) 65
What you'll learn 65
Step-by-step notes 65
Key takeaways 67
Demo 18: Revision — Claude Chat, Cowork, Artifacts, Skills, Connectors, Memory, CCOR, and claude.md (Quiz Format) 68
What you'll learn 68
Step-by-step notes 68
Key takeaways 71
Demo 19: Web Scraping with Apify — Connecting Claude to Real-Time Web Data for Lead Generation 72
What you'll learn 72
Step-by-step notes 72
Key takeaways 74
Demo 20: Using Claude for Stock Investment Decision Support and "Simpler Way" Thinking 75
What you'll learn 75
Step-by-step notes 75
Key takeaways 77
Demo 21: Scheduling Automated Tasks with Claude Cowork (the "Scheduler") 79
What you'll learn 79

Step-by-step notes 79
Key takeaways 80
Demo 22: Claude Dispatch — Controlling Your Desktop Claude App from Your Phone 82
What you'll learn 82
Step-by-step notes 82
Key takeaways 83
Demo 23: Foundations Review — Semantic vs Keyword Search, and 10 Generative AI MCQs 84
What you'll learn 84
Step-by-step notes 84
Key takeaways 86
Demo 24: Foundations Review Part 2 — AI, ML, Deep Learning, and How GPT Actually Works 87
What you'll learn 87
Step-by-step notes 87
Key takeaways 89
Demo 25: Foundations Review Part 3 — n8n Automation Basics (Nodes, Triggers, and Workflow Activation) 90
What you'll learn 90
Step-by-step notes 90
Key takeaways 91
Demo 26: Prompt Engineering Revision (Quiz-Style Recap) and Prompt vs Context Engineering 92
What you'll learn 92
Step-by-step notes 92
Key takeaways 94
Demo 27: Using Claude's "Dispatch" Feature to Control Your Desktop from Your Mobile 95
What you'll learn 95
Step-by-step notes 95
Key takeaways 97
Demo 28: Combining Claude with Canva for Design, Posters, and Social Media Automation 98
What you'll learn 98
Step-by-step notes 98
Key takeaways 100

Demo 29: Motivational Interlude on Consistency (Kaizen/1% Rule) and an Introduction to Claude Design 101
What you'll learn 101
Step-by-step notes 101
Key takeaways 102
Demo 30: Hands-On Walkthrough of Claude Design — Posters, Logos, Carousels, Website Mockups, and Mobile App Prototypes 104
What you'll learn 104
Step-by-step notes 104
Key takeaways 107
Demo 31: Claude Design for Mockups, Prototypes, and Animations, Plus Tips to Save Claude Credits 109
What you'll learn 109
Step-by-step notes 109
Key takeaways 111
Demo 32: Automation Workflows with N8N, Semantic vs Keyword Search, Model Selection, and More Credit-Saving Tricks 112
What you'll learn 112
Step-by-step notes 112
Key takeaways 115
Demo 33: Introduction to Claude Code — What It Is and Why It Changes How You Build Software 116
What you'll learn 116
Step-by-step notes 116
Key takeaways 119
Demo 34: Quick Recap Quiz — Claude Chat vs Claude Code vs Claude Coworker vs Claude Design 120
What you'll learn 120
Step-by-step notes 120
Key takeaways 121
Demo 35: Hands-On with Claude Code — Building, Enhancing, and Debugging a Python Project from the Terminal 122
What you'll learn 122
Step-by-step notes 122
Key takeaways 125

Demo 36: Installing and Using Claude Code from the Terminal 126
What you'll learn 126
Step-by-step notes 126
Key takeaways 128
Demo 37: Building a Complete Student Fee Management System with Claude Code 129
What you'll learn 129
Step-by-step notes 129
Key takeaways 131
Demo 38: Building and Publishing Web Applications with Claude Code 132
What you'll learn 132
Step-by-step notes 132
Key takeaways 134
Demo 39: Understanding Compounding, and Building/Publishing Compound-Interest Calculators with Claude Code 135
What you'll learn 135
Step-by-step notes 135
Key takeaways 137
Demo 40: Using Claude Code Inside VS Code, and Understanding Plan Mode 138
What you'll learn 138
Step-by-step notes 138
Key takeaways 140
Demo 41: Building an Expense Tracker App with Claude Code and a Firebase Database 142
What you'll learn 142
Step-by-step notes 142
Key takeaways 143
Demo 42: Adding Google Sign-In (Per-User Data) and Deploying to GitHub Pages 144
What you'll learn 144
Step-by-step notes 144
Key takeaways 145
Demo 43: Buying a Custom Domain and Pointing It to a GitHub Pages App 147
What you'll learn 147

Step-by-step notes 147
Key takeaways 149
Demo 44: Firebase Test Mode vs Production Mode, Security Rules, and Prioritizing Your Time 150
What you'll learn 150
Step-by-step notes 150
Key takeaways 152
Demo 45: Designing an "AI-Powered Problem Solver" App (Using AI Inside an AI-Built App) 153
What you'll learn 153
Step-by-step notes 153
Key takeaways 155
Demo 46: Automating n8n Workflow Creation and Deployment Directly from Claude Code (MCP + Skills) 156
What you'll learn 156
Step-by-step notes 156
Key takeaways 158

Demo 1: What is Claude? Introducing Anthropic's AI Assistant, Claude Code, and Claude Cowork

This video is the instructor's introduction to a new phase of the course, moving from N8N automation into Claude (Anthropic's AI system). The instructor admits he is a complete beginner with Claude and will learn it together with the students. The session is mostly conceptual — explaining what Claude is, why it is being talked about so much, and showing real-world examples of what "agentic" AI (AI that takes actions, not just gives answers) can do.

What you'll learn

- What Claude is and which company makes it (Anthropic)
- The difference between Claude Chat, Claude Code, and Claude Cowork (the three main "flavors" of Claude)
- Why Claude/Anthropic has become extremely influential in the software industry
- What "agentic AI" means in practice, through real examples (browser + system access)
- Why data-analysis-style jobs are being affected by tools like Claude Cowork
- A word of caution about giving AI tools access to sensitive/financial information

Step-by-step notes

1. Why the course is shifting to Claude

The instructor explains that after covering generative AI foundations, prompt engineering, and N8N automation, the next major topic is Claude. He mentions that a student (Lakhbir) strongly recommended focusing only on Claude because of how powerful it has become, and that Anthropic's Claude-related tools have reportedly affected the stock value of major Indian software companies when new AI "co-worker" features were released.

2. Defining Claude in simple terms

The instructor asks Claude itself: "What does Claude provide, in a single line?" The answer given was:

"Anthropic's Claude is an advanced AI assistant that can understand, write, analyze, and help people with coding, learning, business, and daily tasks through natural conversation."

Key point: Claude is not just for coding — it is meant to help in every part of a person's life (learning, business, personal tasks).

3. Claude Code

Claude Code is described as an AI-powered coding assistant from Anthropic that helps developers write, understand, debug, and edit code directly from the terminal using natural language instructions. The instructor compares it to GitHub Copilot — both let you describe what you want and have the AI write/edit/debug the code for you. Because of tools like this, a large share of routine programming work can now be done automatically.

4. Claude Cowork (Claude's "co-worker" agent)

Claude Cowork is described as a digital co-worker from Anthropic that can access your files, plan tasks, and complete multi-step work automatically on your computer — "like a real assistant." Unlike Chat (which only answers questions), Cowork can actually perform actions such as:

- Opening and reading files/folders
- Analyzing data across many files (the instructor gives an example of a tax department reviewing years of filings across "one crore" folders)
- Browsing websites and extracting information
- Creating documents/Excel files and sending them via WhatsApp or email

5. Two abilities that make an AI tool powerful: browser access + system access

The instructor highlights that when an AI tool can both (a) access a web browser and (b) access your computer/system, it becomes extremely powerful because it can research information online and then act on your files/apps with it.

6. Real-world example demonstrated (using ChatGPT Codex, not Claude)

The instructor shares a personal example (done with OpenAI's ChatGPT Codex, a similar competing tool) where he gave one voice prompt: "Visit my competitor institutes' websites, find their new AI batches, list dates and faculty in Excel, then send that Excel to my admin's WhatsApp contact." The tool opened multiple websites, gathered the data, built an Excel sheet, and sent it via WhatsApp — all from one spoken instruction in about a minute.

7. Another example: auto-generating documentation

He describes another task: take a folder full of JSON workflow files (from N8N), generate documentation explaining each workflow, compile everything into a Word document with meaningful headings, and email/WhatsApp the finished document. Manually this would take 4–5 hours; the AI agent completed it in about 2 minutes.

8. Everyday/personal use cases mentioned

- Asking AI for the single best home remedy for a cold (e.g., steam inhalation) instead of multiple suggestions
- Planning and booking an entire trip (flights, hotel) after just describing the destination and dates
- The shift in how people should now interact with AI: instead of asking for "information," you should ask AI to actually "do the activity" (e.g., "book my tickets" instead of "what are the ticket prices")

9. Bigger picture / industry impact

- The instructor references a case in Germany where a coffee shop is reportedly run entirely by AI managing vendors and ingredients without human interaction.
- He notes that OpenAI's CEO has spoken about AI causing rising unemployment because AI agents can now perform actions, not just provide information.

- A student asks whether Claude Cowork can bypass CAPTCHA/verification checks on websites; the instructor says it generally can navigate around such obstacles by finding alternative ways to get the information, though he is not 100% certain.

10. Security caution

When asked about security, the instructor advises: never give an AI agent access to financial/payment credentials. Everything else (files, browsing, non-financial tasks) can reasonably be delegated, but financial access should be avoided or handled very carefully.

Key takeaways

- Claude is Anthropic's AI assistant family; it comes in three main forms: **Chat** (conversational, like ChatGPT), **Code** (for building/debugging software), and **Cowork** (an agentic assistant that can access files, browse the web, and complete multi-step tasks automatically).
- "Agentic AI" means the AI takes real actions (opening files, browsing sites, sending messages) rather than just answering questions.
- Tasks that used to take hours (data analysis, documentation, research and booking) can be completed in minutes using agentic tools.
- Never give AI tools access to financial/payment information — treat that as an important safety boundary.
- The instructor's plan: spend roughly the next 30–40 days learning Claude in depth before returning to coding automation topics.

Demo 2: Exploring the Claude Desktop App, Pricing Plans, and Claude's Three Modes

This session starts with an open discussion among experienced students (Lakhbir, Shiv, Naveen, and others) sharing real-world experience using Claude Code at work, then moves into the instructor's first hands-on walkthrough of the Claude desktop application — installing it, exploring pricing tiers, and trying Claude Chat for the first time.

What you'll learn

- Real-world lessons about Claude Code costs and "agentic loops" from students already using it professionally
- What Opus, Sonnet, and Haiku (Claude's three models) are, and how their cost differs
- How to sign up for and install the Claude desktop app
- The three Claude flavors again, now shown live in the interface: Chat, Code, and Cowork
- Claude's pricing plans: Free, Pro, and Max
- How to create and use a Claude "Project"
- A live demo of Claude generating a full infographic and PDF automatically

Step-by-step notes

1. Student experience-sharing: risks of "agentic loops"

Before the demo, students who already use Claude Code at work shared cautionary experience:

- Lakhbir explained that building an initial prototype application might cost around $40, but if Claude Code gets stuck trying to fix a single bug, it can enter an "agentic loop" (repeatedly trying different approaches) and burn through $2,000–$5,000 in tokens.
- This is why "specification-driven development" is recommended: clearly specify requirements, design a plan, break it into sub-plans, and review each step, rather than letting the AI freely retry indefinitely.
- Another student (Shiv) echoed that without understanding token usage and monthly usage limits, teams can become fully dependent on the tool and get stuck when limits run out.

2. Cost-saving tip shared by a student

One student suggested that instead of paying for a Claude subscription, you can install Claude Code locally and connect it to a cheaper backend model like **DeepSeek** (very low cost, around 87 cents per million output tokens versus roughly $15 for Opus-tier pricing) or a free/near-free model like "Gemma 4," while still getting most of the day-to-day functionality.

3. The three Claude models

- **Opus** – most powerful and most expensive; use carefully for complex tasks.
- **Sonnet** – balanced, general-purpose model.

- **Haiku** – fastest and cheapest; good for small, simple tasks (less "credit"-hungry).

4. Claude's three main products (recap and live view)

- **Claude Chat** – conversational assistant (like ChatGPT), accessible at claude.ai
- **Claude Code** – for generating, editing, and debugging code end-to-end
- **Claude Cowork** – the agentic "co-worker" that can complete multi-step workflows across your files and apps

Chat is available on web, desktop, and mobile. Code and Cowork are recommended to be used through the **desktop app** since they need deeper access to your local files/system (their full functionality isn't available in the browser version).

5. Installing Claude Desktop

1. Go to claude.ai
2. Choose "Download for Windows" (or your OS) to get the desktop app installer
3. Sign in (the instructor used "Continue with Google")
4. Accept Anthropic's consumer terms and privacy policy
5. Choose a plan (Free / Pro / Max — see below)

6. Claude's pricing plans (as observed)

- **Free** – Chat only; Claude Code and Claude Cowork are **not** included
- **Pro** – around $17/month if billed annually, or about $20/month (roughly $23–24 with tax) if billed monthly; includes Claude Code and Claude Cowork, more usage, and features like connecting Slack/Google Workspace and Model Context Protocol (MCP) integrations
- **Max** – a higher-usage tier above Pro (the instructor notes Pro is generally sufficient and recommends starting with a monthly plan rather than committing to a year)

7. First look at the Claude Chat interface

After onboarding (entering your name, role, etc.), the instructor opened a New Chat and asked: "Explain 5 important bullet points about N8N automation." He also pointed out:

- The model selector at the top shows available models (Sonnet, Haiku, and "more models")
- A "Projects" feature for grouping related chats/files under one context
- An "Artifacts" feature for building things like apps, documents, games, and templates (explored more in later sessions)

8. Creating a Project

6. Click "Projects" in the sidebar
7. Click to start a new project and give it a name (the instructor created one called "YouTube content generation")
8. Describe what you're trying to achieve

9. Once created, you can add files and instructions specific to that project, and all future chats inside it will remember that context automatically

9. Live demo: 30-day learning plan + auto-generated infographic

The instructor asked Claude to "provide an action plan — how many days is best to learn Claude" and got a structured 30-day plan (Day 1–2: explore Claude, Day 3: learn prompt basics, etc., including references to Anthropic's documentation and prompt engineering guide). He then asked Claude to "generate an infographic for the same [30-day plan]." Live in the chat, Claude:

- Installed the necessary tools/packages on the fly
- Wrote a Python script to generate a PDF
- Executed that script to actually produce the infographic/PDF

A student explained this is possible because Claude can install packages and write/execute code in real time to produce outputs like PDFs, Excel files, or images — something ChatGPT's plain chat interface does not do as smoothly.

10. Free plan usage limit

During the demo, the free plan hit its 5-hour message limit, showing a message that usage resets at a specific time or can be upgraded for higher limits.

11. Anthropic's market position (context/motivation)

The instructor spent time explaining why Claude/Anthropic is such a big deal: Anthropic (founded around 2021–2023 by former OpenAI staff) had a private valuation reported around $350 billion, and later reports mentioned figures approaching $900 billion — compared to the combined market value of India's top seven software companies (reported as roughly $250–370 billion with about 16 lakh employees combined), while Anthropic has only around 3,500 employees. He also mentioned IBM's share price reportedly falling around 23% after Anthropic released a mainframe/COBOL migration-related plugin, as an example of how disruptive these AI tool releases have been.

12. Dispatch feature (briefly introduced)

"Dispatch" is a feature (seen in Cowork) that lets you pair your phone with the desktop app (via a QR-style scan) so you can send instructions from your mobile phone that get executed on your desktop Claude app — as long as the desktop app stays running.

Key takeaways

- Claude has three flavors: **Chat** (conversation), **Code** (coding assistant), **Cowork** (agentic task automation) — Code and Cowork require the Pro plan and are best used via the desktop app.
- Claude's models — **Opus** (expensive/powerful), **Sonnet** (balanced), **Haiku** (cheap/fast) — should be chosen based on task complexity to manage cost.

- Watch out for "agentic loops": if Claude Code gets stuck on a bug, it can burn a large number of tokens/credits quickly; use clear, specific, well-scoped prompts (specification-driven development) to avoid this.
- Claude can install tools and write/execute code on the fly to produce real outputs like PDFs and infographics, directly inside a chat.
- Projects let you group related chats and files together so Claude retains consistent context.
- Pro plan pricing is roughly $20/month (or ~$17/month billed annually); Free plan does not include Code or Cowork.

Demo 3: Hands-On with Claude Cowork — Connectors, Gmail Automation, and Task Prompts

This session is a detailed, hands-on walkthrough of the Claude desktop interface, focused on the Cowork mode. The instructor demonstrates connecting Gmail to Claude, setting permission levels, and running real automated tasks like analyzing email data and generating reports — similar in spirit to N8N workflows but driven entirely by natural-language prompts.

What you'll learn

- The different areas of the Claude desktop UI: Chat, Cowork, Code, Projects, Schedule, Artifacts, Dispatch, and Customize
- How "Plugins," "Skills," and "Connectors" work in Claude
- How to connect Gmail to Claude and control what permissions it has (read-only vs. write actions needing approval)
- How to write an effective prompt for a Cowork task
- A real demo: counting course enrollments from Gmail confirmation emails and emailing a report
- Cost/credit considerations, including model choice and prompt length

Step-by-step notes

1. Recap: why the desktop app is essential

Claude Code and Claude Cowork functionality are only available in the desktop app, not the browser-only web version — so installing the desktop app is necessary to follow along.

2. Different "start" buttons per mode

- In **Chat**, starting a conversation shows "New Chat"
- In **Cowork**, it shows "New Task" (because Cowork is meant for completing specific tasks/actions, not just chatting)
- In **Code**, it shows "New Session" (related to a coding session)

3. Key sections inside Cowork

- **New Task** – start a new agentic task
- **Projects** – group related Cowork tasks/context together (same concept as in Chat)
- **Schedule** – like a cron job; you can schedule a task to run automatically at a set time (e.g., "every morning at 7:00, do X")
- **Live Artifacts** – view/create interactive artifacts
- **Dispatch** – pair your mobile phone with the desktop app so you can send tasks/instructions from your phone; the desktop app must remain running for this to work
- **Customize** – manage plugins, skills, and connectors

4. Plugins and Connectors

- **Plugins** extend Claude with capabilities for specific domains: design, productivity, marketing, engineering, data, finance, product management, sales, legal, and more. The instructor emphasizes that these plugin releases from Anthropic have been disruptive to entire industries (e.g., data analysis, cybersecurity) because a single plugin can automate work that used to require a dedicated software company or team.
- **Connectors** let Claude integrate directly with external services/apps, such as Gmail, Canva, Microsoft 365, Google Drive, Notion, Google Calendar, Slack, and HubSpot.

5. Connecting Gmail to Claude

10. Go to Customize → Browse Plugins → Connectors
11. Find Gmail and click Connect
12. Choose the account to connect and confirm you trust Claude to access it
13. Review the permission list, which is split into:

- **Always allow** (no confirmation needed) — e.g., retrieve a specific email thread, list labels, search email threads (all **read-only** actions)
- **Needs approval** (Claude will ask before doing this) — e.g., create a new draft email, create/delete a label, add a label to a message, delete emails (all **write/destructive** actions)

By default, only read operations are auto-allowed; anything that changes data requires your explicit confirmation — an important safety design.

6. Demo scenario: counting course enrollments from Gmail

The instructor set up a realistic business scenario: whenever a student registers for a course on "durgasoftonline.com," a confirmation email arrives in Gmail. He wanted to know, for the last 7 days, which courses had the most enrollments — normally a task that would take an admin staff member hours to do manually by opening and counting emails one by one.

7. Writing the prompt (with ChatGPT's help first)

Interestingly, the instructor first asked **ChatGPT** to help him write a clear, detailed prompt for Claude Cowork, since he wasn't yet confident writing one directly. The generated prompt specified:

- Connect to the Gmail account and perform a **read-only** analysis (explicitly: do not delete, archive, move, or label anything)
- Search emails from the last 7 days where the subject starts with "confirmation"
- Identify the related course name (from the subject or, if unclear, from the email body)
- Group and count enrollments by course
- Sort in descending order of enrollment count
- Output a clean summary table: total confirmation emails, unique courses, and a per-course serial number, course name, and enrollment count

A student later asked whether Claude itself can generate such prompts (yes — the instructor confirmed Claude Chat can also format/optimize prompts for you), so relying on ChatGPT for this step isn't required, just what the instructor was more comfortable with at the time.

8. Running the task in Cowork

14. Paste the prompt into a New Task in Cowork
15. Choose the model — **Haiku** was recommended here since this is a fairly simple task and Haiku uses fewer credits
16. Claude worked step-by-step: searching Gmail, retrieving matching threads, and compiling results
17. Result: 51 total confirmation emails in the last 7 days across 19 unique courses, with the most popular course clearly identified and a full ranked table produced

9. Sending the results

The instructor then asked Claude to email the report. Because Gmail's write permissions require approval, Claude asked whether to:

18. Send the email directly, or
19. Save it as a Gmail draft

He chose "create a Gmail draft," Claude asked for confirmation ("yellow"/approval), and the draft appeared correctly in the Gmail account's draft folder. He then manually sent that draft to demonstrate it worked end-to-end.

10. A note on politeness in prompts (cost reminder)

When a student typed a "please"-style, overly polite prompt, the instructor reminded the class of a point from the earlier prompt-engineering sessions: avoid unnecessary conversational words (like excessive "please," greetings) in prompts to an AI, since every extra word consumes tokens/credits without adding value — write prompts efficiently, like machine instructions, not like a message to a person.

11. More Cowork examples given

- "Find the top 5 AI-related news items and send them by email" — this required Cowork specifically (not just Chat) because sending an email is an *action*, not just information retrieval.
- Deleting spam emails automatically via a simple prompt, instead of building a full N8N workflow for it.
- Scheduling a task to check competitor institutes' websites every morning for new course batches and emailing a formatted list (institute name, batch, date) to an admin, automatically.

12. Security discussion: how much approval is required?

A student asked whether every action needs a manual human approval. The instructor clarified: it depends on which permissions you granted when connecting the service. Since he only allowed **read** access by default (for safety), any write action (like sending mail or creating drafts) requires explicit

confirmation each time. If you instead enable "allow all" permissions on a connector, Claude would perform write actions without waiting for confirmation — which is riskier.

13. Usage/credit tracking

In Settings → Usage, you can see:

- **Current session** usage percentage (which resets after a few hours, not daily)
- **Weekly limits** across all models (a separate longer-term cap)

There is no daily reset, but there is a session-based reset (roughly every couple of hours) and a weekly usage cap.

14. Prompt reuse and token optimization

A student asked if you could skip generating a "proper" long prompt and just give Claude your raw, casual instructions directly. The instructor explained that short, vague instructions don't give Claude enough clarity to complete complex tasks reliably — a more detailed, structured prompt (like the one ChatGPT generated) produces better results. He also noted you can ask Claude (or ChatGPT) to further **optimize a prompt to use fewer tokens while keeping the same functionality**.

Key takeaways

- Cowork's UI centers around **Tasks** (not chats), plus **Projects**, **Schedule** (cron-like automation), **Dispatch** (mobile-to-desktop control), and **Customize** (plugins/skills/connectors).
- Connectors (like Gmail) let Claude interact with real external accounts; by default only read-only actions are auto-approved, and any write/destructive action requires your explicit confirmation — this is an important safety feature.
- Writing a clear, detailed, and well-structured prompt (specifying scope, format, and constraints) produces far better Cowork results than a vague instruction — you can ask any chat AI to help format/optimize such prompts.
- Choose Haiku for simple, low-stakes tasks to save credits; save Sonnet/Opus for more complex or important work.
- Avoid unnecessary conversational filler in prompts ("please," greetings) — it wastes tokens/credits.
- Cowork can fully replace many simple N8N-style automations (e.g., deleting spam, monitoring competitor sites, sending scheduled reports) using just a well-written natural-language prompt instead of building a workflow.

Demo 4: Managing Credits, Global Instructions, and Comparing Claude vs ChatGPT for Analysis

This session is a shorter, practical continuation covering how to manage Claude's usage/credits wisely, how to set global behavior instructions (similar to a system prompt), keyboard shortcuts for switching modes, and a comparison of Claude versus ChatGPT for research and data-analysis-style tasks.

What you'll learn

- Important rules about switching models and managing credits/usage
- Why long conversations cost more, and why starting new chats for unrelated topics saves credits
- How to set global "Instructions for Claude" (a persistent behavior/persona setting)
- Keyboard shortcuts to quickly switch between Chat, Cowork, and Code modes
- How "usage credits" work for continuing past your plan's included limit
- A live comparison of Claude vs. ChatGPT answering the same research question
- Why Claude is considered stronger for producing dashboards/visual analysis from data like PDFs, Excel files, and images

Step-by-step notes

1. Recap of prior sessions

- Claude has multiple access modes: desktop app, web version
- Three modes: Cowork, Code, Chat
- Three models: Opus (costly/complex), Sonnet (balanced), Haiku (efficient/cheap)

2. Important rule: you cannot change models mid-conversation

Once a chat or Cowork task has started, you **cannot switch the model** partway through — you must start a new conversation to use a different model. Because of this, always check which model is selected **before** you start typing your first message.

3. What happens when your daily/session credits run out

- Normally, you must wait for credits to refresh (roughly a session-based reset, not necessarily daily)
- Alternatively, in Settings → Usage, there's an option to **turn on usage credits** — this lets you keep working past your included limit by paying extra for additional usage instead of being blocked, so you can "continue seamlessly without interruption." This is optional and needs to be explicitly enabled.

4. Avoid long conversations in a single chat

Because Claude has to keep remembering the full context of a long-running conversation, longer chats consume more credits over time. The instructor recommends starting a **new chat** whenever you move to an unrelated topic, rather than continuing everything in one giant conversation thread. (He notes ChatGPT doesn't seem to have this same issue as noticeably.)

5. Setting global "Instructions for Claude" (a system-prompt-like setting)

This is compared directly to the "system prompt" concept covered earlier in the N8N/AI agent sessions — a way to give the AI persistent, standing instructions so you don't have to repeat them every time.

20. Go to Settings → General
21. You can set your display name, describe "what best describes your work" (your role/profession)
22. Under "Instructions for Claude," you can write standing guidance, e.g., "ask clarifying questions before giving detailed answers," or specify a persona (e.g., "act as an experienced Java trainer") — this mirrors the "persona" concept from prompt engineering
23. These instructions apply across all chats and Cowork tasks

Setting your role (e.g., software engineer, data scientist, interior designer) helps Claude tailor its responses to be more relevant to you by default.

6. Adaptive thinking

A model-selection feature called "adaptive thinking" (previously called "extended thinking") can be enabled so Claude automatically decides how much extra reasoning/research effort to apply, instead of you manually choosing a more expensive model each time.

7. Keyboard shortcuts

- **Ctrl+1** → switch to Chat mode
- **Ctrl+2** → switch to Cowork mode
- **Ctrl+3** → switch to Code mode

8. Usage credits screen walkthrough

In Settings → Usage, you can see current session usage percentage, usage across all models, and an option ("include usage credits") that, once turned on, allows you to keep using Claude beyond your plan's included limit by paying for the extra usage — useful if you don't want to be blocked mid-task.

9. Voice input question

A student asked whether Cowork can be fully voice-controlled (not just dictating text via Windows+H, but actually issuing spoken commands that Claude interprets and acts on). The instructor confirmed this is possible in principle — voice input gets converted to text, which is then treated as a normal prompt — and said they would test it together in a future session.

10. Live comparison: Claude vs. ChatGPT for a research question

The instructor asked both tools the same question — "find new AI-related online batches available at DurgaSoft" — first in ChatGPT, then in Claude. Both could answer, but the instructor's broader point was about a *different* kind of comparison (data analysis), described next.

11. Why Claude stands out for data analysis

The instructor described (without live data on hand, promising to demonstrate later) how Claude can take something like a personal credit card statement, PDF, Excel file, or image and produce:

- Colorful pie charts and bar charts
- A clean analysis dashboard showing spending by category
- Concrete suggestions on where to cut costs or improve revenue (using a business example: analyzing DurgaSoft's inquiry and revenue data)

He contrasts this with ChatGPT, which is very strong at *content generation* but, in his experience, not as strong at producing this kind of visual dashboard/analysis output directly in chat.

12. The "iPhone effect" analogy

The instructor uses an analogy: once people get used to using an iPhone, they find it hard to go back to other phones. He suggests the same thing tends to happen once people get comfortable with Claude — they stop wanting to go back to other AI chat tools for certain tasks, especially visual analysis.

13. Discussion: AI's effect on jobs

The instructor and students discuss that data-analysis-related jobs (previously considered a guaranteed path to employment in India, requiring skills like Python, SQL, PySpark, and Excel) are being significantly impacted now that AI plugins can do this analysis automatically. There's also a mention of a news story about large-scale HR layoffs at a big American company, tied to increasing AI automation of routine analytical/management work. The advice given: focus on being more creative and building skills that go beyond repetitive tasks, since AI increasingly handles the repetitive work itself.

Key takeaways

- You cannot switch models mid-conversation — pick your model **before** starting a chat/task.
- Long, mixed-topic conversations cost more credits; start a fresh chat for each new, unrelated task.
- "Instructions for Claude" in Settings works like a persistent system prompt/persona — set it once instead of repeating instructions every time.
- Turning on "usage credits" lets you pay to continue past your included plan limit instead of being blocked.
- Keyboard shortcuts Ctrl+1/2/3 quickly switch between Chat, Cowork, and Code modes.
- Claude is considered noticeably stronger than ChatGPT at producing visual dashboards and structured analysis from data files (PDFs, Excel, images, credit card statements).
- Industry impact: traditional data-analysis skills are losing value as AI plugins automate that work directly; more creative, higher-level skills are becoming more important.

Demo 5: Real-World Shopping/Travel Comparisons and Building Interactive Artifacts

This session has two main parts: (1) practical demos of using Claude Chat to research and compare prices for real purchases (books, flights) across different models, and (2) a deep dive into Claude "Artifacts" — interactive tools/apps Claude can build for you, demonstrated by creating a cost calculator and an interest/EMI calculator.

What you'll learn

- How to use Claude to research and compare prices across e-commerce sites instead of checking each site manually
- How answer quality differs between the Haiku, Sonnet, and Opus models on the same prompt
- How to use Claude for travel price/flight research
- How to generate a structured business "briefing" report from a single prompt
- What Claude "Artifacts" are and how they differ from regular chat answers
- How to build, edit, download, and publish an interactive artifact (e.g., a calculator)

Step-by-step notes

1. Recap and framing

The instructor reiterates the "iPhone effect" analogy from the previous session and reports that after a few days of using Claude, he now feels ChatGPT is "not up to the mark" in comparison — mainly because of the kinds of interactive outputs (discussed below) that Claude can produce.

2. Example: everyday personal use — meal planning

A student shared a real example: someone uses Claude to build a weekly meal plan by telling it what food is in the fridge; Claude produces a day-by-day plan that a cook can follow all week, showing that Claude's usefulness isn't limited to work tasks.

3. Demo 1 — Comparing online prices for a product (book) across models

Scenario: the instructor wants to buy the paperback book "48 Laws of Power" by Robert Greene at the best price.

24. He wrote a prompt (first refining it using the "format the prompt" trick — asking Claude/ChatGPT to turn his rough wording into a clearer, well-structured prompt): "I want to buy the paperback version of 48 Laws of Power online. Compare prices across different online platforms and trusted bookstores, and suggest the best buying option based on price, delivery, seller reliability, and edition quality."
25. Running this with **Haiku**: Claude searched multiple platforms (Flipkart, Amazon, Bookchor, Snapdeal, etc.), compared listed prices, and recommended one option balancing price and reliability (around

₹585 on Flipkart in this case), explaining its reasoning (free shipping, cash on delivery, trusted seller, fast delivery).

26. Running the **same prompt with Sonnet**: Sonnet did noticeably deeper research — it identified that there are multiple *different paperback editions* in the Indian market (an original international edition vs. cheap low-quality reprints), flagged red flags (many sub-₹400 listings on Amazon/Flipkart are low-quality reprints, with poor buyer reviews on page quality), and gave separate recommendations for "best overall" and "best budget pick."
27. Running the **same prompt with Opus**: Opus produced yet another version of the analysis, with slightly different pricing details, since it's the most capable/expensive model and does more thorough checking.

Lesson: the same prompt run on Haiku vs. Sonnet vs. Opus can give meaningfully different depth of research — more expensive models generally do more thorough comparison but cost more tokens.

4. Demo 2 — Comparing flight prices (Hyderabad to Srinagar)

Using Sonnet, the instructor asked Claude to "check available flights and charges from Hyderabad to Srinagar next week and suggest the best flight and timing with respect to low cost." Claude researched multiple airlines (Air India, SpiceJet, Akasa, Indigo) and gave day-by-day guidance, noting that flights on Tuesdays/Wednesdays tend to be cheaper, with a specific recommended flight and estimated fare range. A student pointed out this could be combined with **Cowork's Schedule feature** to automatically re-check prices at specific times (e.g., midnight, when prices are often lower) and send an alert (email/WhatsApp) when the best price is found — turning this into a recurring automated price-watch task.

5. Demo 3 — Generating a structured business briefing

The instructor asked Claude to identify "three major changes happening in online software training and generative AI education," two skills becoming more valuable, and one major risk to watch — output formatted as a concise briefing. Claude's response (used as a realistic example for the training institute) included:

- Agentic AI replacing simple chatbot training; automation tools like N8N entering curricula
- Entry-level IT hiring in India shrinking as companies shift away from high-volume hiring
- Valuable emerging skills: prompt engineering + agent design, and AI-integrated full-stack + cloud development
- Major risk: outdated curriculum leading to student unemployment if institutes don't integrate generative AI/agentic AI/automation into their courses
- Recommended action: embed AI/agent/automation modules into existing courses rather than launching them as separate standalone courses, and train faculty first

This is used as a real illustration of how a single well-crafted prompt can produce a decision-ready business report.

6. What are Artifacts?

Definition (from Anthropic, read aloud in the session): "Claude artifacts are interactive files or outputs — like code, documents, websites, dashboards, or apps — that Claude creates and shows in a separate preview panel for editing and reuse."

The key word is **interactive** — unlike a plain chat answer, an artifact is something you can actually use, change inputs in, and get updated results instantly.

7. Building an artifact: a training-cost calculator

Prompt given: "Create a simple DurgaSoft teaching cost calculator for instructors and training management. Add input fields for: number of instructors, instructor payment per hour, course duration, extra expense percentage (internet, software tools, marketing, etc.). The calculator should automatically show total training cost, cost per week, and cost per instructor. Make it simple, interactive, and easy to understand — when values change, results should update instantly."

Result: Claude built a working interactive calculator directly in the chat. Changing any input (e.g., number of instructors from 3 to 2, hourly rate from ₹500 to ₹600, course duration from 8 to 7 weeks) instantly recalculated the total cost, cost per week, and cost per instructor — with a visual expense breakdown.

8. Downloading and publishing an artifact

28. Click **Download as HTML** to save the artifact as a standalone HTML file you can open in any browser (double-click to run — no backend/server needed; it's plain HTML/CSS/JavaScript)
29. Or click **Publish Artifact** to get a shareable public link others can open in their browser (this does not consume the viewer's own Claude credits, since it's just a static hosted page)
30. Published artifacts also appear under your own account's Artifacts list for later reuse

The instructor opened the downloaded HTML file in a code editor to show students that it's simple HTML with CSS styling and JavaScript functions handling the calculations — no complex framework involved.

9. Building a second artifact: an interest/EMI calculator

Prompt: "Create a simple and interactive interest calculator for a principal amount of ₹1 lakh. Include: principal amount (default ₹1 lakh), interest rate options ranging from 1 to 3 rupees [per hundred], compounding period options (1 month, 3 months, 6 months, 1 year). Output should show principal, selected rate, selected compounding period, total interest, and final amount. Results should update instantly when inputs change. Keep the UI simple and clear, use Indian currency formatting, and add a short summary explaining the profit for the selected option."

This was built first with Opus. The instructor then demonstrated **iteratively editing** the same artifact with follow-up prompts, e.g.:

- "Add a 9-month compounding option" — Claude updated the calculator to include the new option among the existing ones without needing to rebuild everything from scratch
- "Extend the interest rate range up to 5" — updated accordingly

Testing different values live showed how compounding more frequently (e.g., every 6 months vs. every year) results in a higher final amount due to "interest on interest."

10. Real-world use cases discussed by students

- Calculating the real annualized return on a property investment (e.g., bought for ₹60 lakh in 2022, now worth ~₹95 lakh–1 crore) by working backward to find the actual compound annual growth rate — often revealing returns closer to what a bank fixed deposit would give (around 7–8%), correcting inflated verbal claims about property appreciation.
- Calculating true home loan cost over time (e.g., paying 7.5% interest on a loan versus the appreciation of the purchased asset) to judge whether taking a loan makes financial sense.
- Building a mutual fund growth calculator with annually increasing contributions to see a realistic long-term projection.
- Sharing a finished HTML artifact (like the calculator) with someone (e.g., an accountant) who doesn't even need a Claude account — since it's just a standalone HTML file.

11. Important caution about accuracy

A student pointed out that before handing an artifact/calculator to someone else (e.g., a client or accountant), you must **manually verify the calculations are correct**, since it's not guaranteed to be perfectly accurate every time — always test and validate the logic before relying on or distributing it.

12. Clarifying question: do you need to say "create an artifact" explicitly?

No — Claude automatically recognizes when your prompt is asking for an app/tool/interactive output (e.g., "create a calculator") and generates it as an artifact by default; you don't need to use the word "artifact" yourself.

Key takeaways

- Different Claude models (Haiku, Sonnet, Opus) can give meaningfully different depth of research/analysis for the same prompt — more expensive models tend to dig deeper (e.g., catching that cheap book listings were low-quality reprints).
- Claude Chat alone (without Cowork) can do rich comparison-shopping and travel research just from a well-formed prompt; combining this with Cowork's Schedule feature enables ongoing automated price-watching.
- **Artifacts** are Claude's way of producing interactive, reusable outputs (calculators, dashboards, mini-apps) instead of plain text answers — inputs can be changed and results update instantly.
- Artifacts can be downloaded as plain HTML/CSS/JavaScript files (no backend required) or published as a public shareable link.
- You can iteratively refine an existing artifact with follow-up prompts (e.g., "add a 9-month option") rather than starting over.
- Real, practical uses include cost calculators, EMI/interest calculators, and investment return calculators — genuinely useful for both business and personal finance decisions.

- Always manually verify an artifact's calculations before sharing it with others or relying on it for real decisions.

Demo 6: Claude Artifacts — Building Reusable Mini-Apps, Calculators and Dashboards

This session is a deep, hands-on continuation of the "artifacts" concept, where Durga Sir builds and demonstrates several real artifacts inside Claude — a loan interest calculator, an interest-difference calculator, a flowchart, a software ROI calculator, and a household electricity dashboard — while clarifying exactly when Claude does (and does not) treat an output as an artifact.

What you'll learn

- What an artifact is in Claude and why it exists
- The rules Claude uses to automatically decide something deserves an artifact
- How to explicitly force something to become an artifact
- How to enable/disable artifacts in Claude settings
- How to view, edit, download, and publish artifacts
- How to build several practical mini-apps using plain prompts

Step-by-step notes

1. Recap: what is an artifact?

An artifact is a reusable output that Claude creates and saves separately from the normal chat flow, so you can view, edit, download, or share it independently. Anthropic's own description (read from a Gamma-generated slide in the video): "Claude artifacts are interactive files or outputs — like code, documents, websites, dashboards, or apps — that Claude can show in a separate preview panel for editing and reuse." Examples given: a one-page website, an HTML calculator, a flowchart, an image, an SVG, or a mini dashboard.

2. Why Claude treats artifacts specially

Unlike ChatGPT (per the instructor), Claude gives special importance to reusable outputs by keeping a dedicated "Artifacts" section in the left sidebar where all your saved artifacts live and can be reopened, downloaded, or re-shared at any time.

3. The "three golden rules" for when Claude auto-creates an artifact (from the presentation shown in-session):

31. It's fairly large — for example, more than roughly 50 lines of content (like a full essay or a page of code).
32. You're likely to reuse it — documents, code, designs you'll want again later.
33. It stands on its own — it makes sense without needing the rest of the chat for context.

4. How to turn artifacts on/off

Go to your profile (bottom-left) → **Settings** → **Capabilities**. There you'll find a toggle for artifacts ("Generate code, documents, and designs in a dedicated window alongside your conversation"). It is

enabled by default. If disabled, there's an option about disabling code execution/file creation to change it.

5. Forcing an artifact explicitly

If Claude doesn't automatically recognize something as an artifact, you can just say "create an artifact for this" (or "put this in an artifact" after the fact) in your prompt. This is not mandatory but is recommended if you specifically want the publish/save option to appear.

6. Demo: Interest calculator (recap from previous day)

An HTML/CSS/JS single-page interest calculator was built earlier and shown again — downloadable and previously published as a public link.

7. Demo: Interest Difference Calculator

Prompt used (built with help from ChatGPT to draft the requirements, then given to Claude):

"Create a cloud artifact — interest difference calculator. I want a simple, clean, interactive calculator to compare old interest and new interest. Inputs: principal amount, old interest rate, old compounding period, new interest rate, new compounding period, duration (1–20 years, use a slider bar). Calculate the compound interest for both. Use HTML, CSS, Bootstrap, and JavaScript only. Make it a Claude artifact. Simple, clean UI, mobile-friendly design, Indian currency format."

Steps:

34. Open a new chat.
35. Paste the prompt.
36. Claude builds the calculator in about a minute using HTML, Bootstrap, and JavaScript (chosen because those specific technologies were named in the prompt).
37. Test it: e.g., ₹1,00,000 at 2% yearly for 1 year gives ₹24,000 interest; at 1.5% it gives ₹18,000 — a ₹6,000 difference, useful for a lender/finance-owner deciding whether to offer a lower rate.
38. Key clarification from class discussion: even if the app technically uses multiple "files" (HTML/CSS/JS or a React app with multiple components), Claude still treats the **whole application as one single artifact** — not one artifact per file.

8. Does the word "artifact" have to be in the prompt?

No — Claude usually recognizes reusable content automatically and creates an artifact without your asking. But if you explicitly type "artifact" (or ask afterward "make this into an artifact"), Claude will more reliably route it into the dedicated Artifacts space, and — importantly — using the word artifact is often what makes the **Publish** option appear. Without it, you may only get a "download" option for the reusable component. So: not compulsory, but recommended if you want the publish/share feature.

9. Publishing an artifact

Once something is (or is converted into) an artifact, click the three dots on it → **Publish and copy link**. This gives a public URL others can open (it's hosted on Claude's own domain, not your custom domain — if you want your own domain you'd need separate hosting).

10. Demo: Flowchart

Prompt: "Create a flowchart showing a simple Durgasoft student admission process — student inquiry, counseling call, demo class, course explanation, fee — add one branch: from counseling call to 'not eligible / wrong course selection' if the student's background doesn't match requirements." Claude generated a downloadable flowchart image with a brief explanation. Even a flowchart is treated as an artifact — you can ask Claude to explicitly "put this in an artifact" if it doesn't happen automatically.

11. Demo: Software ROI Calculator

Prompt: "Create a simple interactive software ROI calculator. Input fields: total software cost, number of employees using it, hours saved per employee per week, average employee cost per hour. The calculator should automatically show total hours saved per year, total money saved per year, net profit/ROI, and payback period in months. Make it beginner-friendly, interactive, and update instantly when values change."

- Example numbers tested: $12,000 annual cost, 20 employees, 5 hours saved/week, $25/hour → 52,000 hours saved/year worth $130,000, net profit/ROI $118,000, payback in 1.1 months (very profitable).
- Changing the cost to $120,000 flips it to a loss (about −89% ROI), showing the calculator updates live.

12. Demo: Household Electricity Usage Dashboard

Created via the dedicated **Artifacts → New Artifact → Start from scratch** flow (accessible from the left sidebar, not from a normal chat box — this tab is only for browsing/creating artifacts, not a place with its own chat input). The dashboard tracked AC, fan, TV, fridge, washing machine usage with hours, rate, and a "high bill alert" if the projected monthly bill exceeds a ₹3,000 threshold, plus bar/pie charts of usage by appliance.

13. Real-world use cases discussed in Q&A

- Front-end UI components (e.g., a student feedback form) can be created as artifacts and later connected to a backend/database.
- Artifacts are for the "long chat window" / rapid prototyping use case; they are not the right concept for building full production applications with many interlinked files — that's what **Claude Code** is for (mentioned as the next big topic).
- Common reusable snippets across a data pipeline (e.g., shared data-cleaning steps) are conceptually similar to "reusable components," but practically, developers don't call that code an "artifact" — that term is used specifically for the chat-window reusable output feature, not for general code reuse.

- To connect an artifact's data to a real database, you need external tools such as an n8n workflow (previously demonstrated with a Lovable-built app whose form data was captured into Google Sheets/a database via n8n).
- For sharing a static HTML/output artifact broadly, options mentioned: download and run locally, or host the static files on Amazon S3 (S3 static hosting gives a public URL without needing to spin up EC2/Kubernetes).

Key takeaways

- An artifact = any reusable output (code, calculator, image, PDF, flowchart, dashboard, mini web app) Claude saves separately so you can view, edit, download, reuse, or publish it.
- Claude auto-detects artifacts using three cues: size (~50+ lines), reusability, and standalone completeness — but you can always force it by saying "create an artifact."
- An artifact can be a whole multi-file app (HTML+CSS+JS or React) — it's still counted as one artifact, not one per file.
- Explicitly using the word "artifact" in your prompt is the reliable way to unlock the **Publish** (public link) option.
- Artifacts live inside a dedicated left-sidebar "Artifacts" section, separate from your regular chat history; enabled by default under Settings → Capabilities.
- Artifacts are best for small reusable apps/snippets/documents inside the chat — not a substitute for real software development, which is better handled with Claude Code.

Demo 7: Introduction to Claude Skills — Teaching Claude Your Instructions Once

A largely theoretical session (with the instructor openly learning alongside the class) introducing Claude Skills: what they are, why they matter, and how they differ from artifacts — illustrated through a Gamma-generated presentation and everyday analogies (a wife who knows exactly how you like your chicken curry, an admin who already knows the video-editing process).

What you'll learn

- What a Claude "skill" is and how it differs from an "artifact"
- Why skills save time, money (tokens), and improve consistency
- How Claude decides when to automatically use a skill
- The high-level anatomy of a skill (SKILL.md, scripts, resources)
- Real-life analogies to build intuition before the hands-on session

Step-by-step notes

1. Recap of prior sessions

Claude has three models — **Opus**, **Sonnet**, and **Haiku**. Opus consumes the most credits/tokens and is best for deep analysis or extended/adaptive thinking; the class had already covered usage limits, extending usage with extra payment, and the artifacts concept.

2. Defining "skill" using Claude itself

The instructor asked Claude directly to define its own skill concept. Claude's definition (read aloud in class): "A Claude skill is a reusable custom instruction set that teaches Claude how to perform a special task automatically and consistently, without repeating prompts every time."

3. Five important points about Claude Skills (from a follow-up prompt asking Claude to explain in "simple Indian English"):

39. **Reusable instructions** — save a set of instructions once and reuse them again and again without retyping the same prompt every time (useful for email writing, PPT generation, coding help, document formatting, content creation, workflow automation).
40. **Custom workflows** — skills can be tailored to your company's processes, teaching style, or personal work style.
41. **Automatic detection** — Claude automatically detects when a skill is needed and loads it during conversation without you manually selecting it every time.
42. **Time & consistency** — skills save time, reduce repeated work, maintain consistency, and make Claude behave like a dedicated personal assistant.
43. **Fixed instructions = consistent output** — because the underlying instructions don't change between uses, results stay consistent (with natural small AI variation), unlike having to retype a slightly different prompt each time.

4. Skill vs Artifact — the key distinction raised in Q&A

- An **artifact** is a reusable *output* (something Claude produced that you want to reuse).
- A **skill** is a reusable *instruction set* (knowledge/commands you teach Claude so it performs a task the same way every time).
- Skills are not tied to a single chat session — they're something like a "global instruction" associated with your account, so once defined they apply across new chats.
- Multiple skills can be added to your account, not just one.

5. Comparisons to similar concepts in other tools

- Similar in spirit to **Gemini's "Gems"** and **ChatGPT's Custom GPTs** — you pre-load instructions/context so you don't retype them — though the instructor notes Claude's automatic skill-detection (recognizing when a skill applies without you manually picking it) may go further than manually selecting a Gem or Custom GPT each time.

6. "The secret superpower" — auto-selection and efficiency (from the presentation):

- Claude auto-selects the right skill based on what you ask (e.g., "make me a slide deck from this brief" auto-matches a slides skill) — no manual setup needed, though you can also name a skill directly for precision.
- Skills are **token-efficient**: instead of re-explaining full context every time (which wastes tokens), Claude loads skills progressively, pulling in only what's needed for the task — like a chef grabbing the one recipe card needed instead of reading the whole cookbook.
- Result: faster responses, lower cost, and sharper/more consistent outputs.

7. Pre-built skill examples mentioned

PowerPoint deck builders, Word/branded docs, Excel, PDF exports, and a "Skill Creator" for building your own custom skills.

8. Everyday analogies used to build intuition

- Teaching N8N to students once means they can later answer N8N questions without being re-taught — same idea as defining a skill once and reusing the "knowledge" repeatedly.
- Telling your wife "prepare chicken curry" and she already knows your preferred style (spice level, consistency) — you don't need to re-explain every time.
- Instructing an admin once (download the gym-class video, cut clips under 90 seconds into shorts, longer clips into long-form videos, upload) means daily you only need to ask "have you uploaded?" — you don't need to restate the whole process each time.
- Telling an accountant once "every morning transfer 10% of yesterday's revenue to these 4 accounts" — later you just ask "did you transfer?" without re-specifying amounts/accounts.

9. Updating a skill

If a skill's format needs to change permanently, you update/modify the existing skill definition — the next time it runs, it uses the updated version (no need to redefine from scratch, only edit what changed). For one-off tweaks, you can just provide the full instructions again for that one-time need.

Key takeaways

- A **skill** = predefined, reusable instructions that teach Claude how to perform a specific task exactly your way, every time, without you re-explaining.
- Skills are different from artifacts: artifacts are reusable *outputs*; skills are reusable *instructions/behavior*.
- Claude auto-detects and loads the relevant skill based on your prompt — you rarely need to manually pick one.
- Benefits: time saved, reduced repetition, consistent output, and lower token/cost usage since Claude only loads what's needed.
- You can create your own custom skills (in addition to Anthropic's predefined ones) — the practical how-to was promised for the next session (Demo 8).

Demo 8: Creating and Using Custom Claude Skills (Hands-On)

The hands-on follow-up to Demo 7 — this session shows exactly where skills live in the Claude UI, how to browse and enable predefined skills (like Canvas Design), and how to build your own custom skills from scratch, including a real "Employee Feedback Report Generator" and a "Daily Task Planner."

What you'll learn

- Where to find and manage skills in the Claude interface
- How to enable a predefined skill and use it in a prompt
- How to create your own custom skill (two different methods)
- How to test, edit, enable/disable, and delete a skill
- How skills interact with the artifact feature
- What happens when you don't specify a skill at all

Step-by-step notes

1. Recap of skill advantages (elicited from the class):

- Reusability — no need to retype large instructions repeatedly.
- Improved performance/consistency.
- Cost/token savings — optimization happens because the instructions are already predefined, so less needs to be generated fresh each time.

2. Where skills live in the Claude UI

In the Claude web app there are four key icons/sections: **New chat**, **Projects**, **Artifacts**, and **Customize**. Under **Customize**, there are tabs for **Skills**, **Connectors**, and **Plugins** (Connectors and Plugins to be covered in later sessions).

3. Browsing predefined (Anthropic/partner) skills

Go to Customize → Skills → click the **+ (plus)** icon next to "Skills" → **Browse skills**. This shows Anthropic's built-in predefined skills, including:

- **Skill Creator** — for creating new skills.
- **Canvas Design** — creates visual art/infographics as PNG or PDF documents using a defined design philosophy.
- **Web Artifacts Builder**
- **MCP Builder**
- **Theme Factory** (brand guidelines)

4. Enabling / disabling a predefined skill

Click **+** next to a skill (e.g., Canvas Design) to add it — it then appears under "Personal skills" with a gear/settings icon instead of a plus, indicating it's active. Click the three dots on a skill to **Disable**, **Enable**, or **Uninstall** (permanently remove) it.

5. Using a skill in a chat

Type **/** (forward slash) in the chat box — this brings up a picker of currently available skills (e.g., Canvas Design, Skill Creator). Select one, then type your prompt; Claude will note that it's "consulting" that skill before generating output. This is conceptually similar to selecting an MCP tool via a "+" menu in other tools, or plugins in ChatGPT.

6. Demo: Using the Canvas Design skill

Prompt used (sourced from ChatGPT): "Create a visually powerful one-page infographic for Durgasoft showing the top three benefits of AI for students, employees, trainers, and business professionals. The design should look modern, premium..." Claude explicitly says it is "consulting the design skill" and generates a downloadable PDF/PNG infographic. A second test ("create an image with the top five benefits of Python") also used Canvas Design and produced a PNG, though the instructor notes the visual quality wasn't perfect.

7. Important clarification: no guarantee of auto-selecting the "right" skill

If you don't specify a skill (no /), Claude will try to pick an appropriate one automatically, but there's no guarantee it picks the one you intended — so for precision, explicitly select or name the skill.

8. Creating your own custom skill — Method 1: Write skill instructions

Steps:

44. Go to **Customize → Skills → + (plus) → Create new skill**.
45. Choose **Write skill instructions**.
46. Fill in: **Skill name**, **Description**, and **Instructions**.
47. Example built live — "Employee Feedback Report Generator":

- Trigger: when this skill is used, help the user create a professional employee feedback report.
- First ask for: employee name, job role, review duration, major accomplishments, challenges handled, skills demonstrated, areas needing improvement, manager comments/notes.
- Output structure (fixed sections every time): **Employee overview, Major contributions, Core strengths, Growth opportunities, Final evaluation.**

48. Click **Create**. The skill appears in your skills list, enabled by default. From there you can **Edit**, **Replace**, **Download** (to share with someone else, who can then upload it into their own Claude), or **Delete**.
49. Test it: start a new chat, use / to select the skill, and supply the raw facts about an employee (e.g., "Rahul Sharma, Python trainer, Jan–June, ran 4 online batches, improved feedback rating from 4.1 to 4.7..."). Claude generates the report using the fixed 5-section format automatically, including things like manager review date and signature line.
50. Consistency confirmed: run it for 100 different employees and the section structure stays fixed every time — only the content changes. This is the "consistent output" benefit of skills.

9. Skills + Artifacts together

If you want the skill's output to be reusable/shareable (e.g., a Word doc to send to many people), just add the word "artifact" in your prompt (e.g., "create artifact" along with your skill-based request) and a **Publish artifact** option appears. You can also bake "always output as an artifact" directly into the skill's instructions so every run of that skill produces an artifact automatically, without needing to add the word each time.

10. Creating your own custom skill — Method 2: "Create with Claude" (Skill Creator)

51. Go to Customize → Skills → + → Create new skill → **Create with Claude**.
52. Select the **Skill Creator** skill and describe what you want (e.g., a "Daily Task Planner" skill: ask the user for today's tasks, deadlines, high-priority work, and available working hours; produce a schedule with high-priority tasks first, estimated time per task, productivity tips, and an end-of-day checklist, formatted with headings/bullet points, understandable within 1 minute).
53. Unlike writing instructions manually, this method has Claude **run test cases** automatically while building the skill.
54. When ready, Claude offers to "package" the skill — click **Save skill** at the top to finalize it.
55. Test in a new chat: select the "Daily task planner" skill (or just type the prompt — Claude auto-detects and reads the skill even without explicit / selection) and confirm the output follows the defined format.

11. What happens with no skill at all

The instructor disabled the "Daily task planner" skill and re-ran the same prompt — Claude answered using its **general knowledge** with a noticeably different (less structured, less "your format") output, proving the skill really was driving the earlier consistent format.

12. Managing skills

- **Enable/Disable**: three-dot menu on the skill, or a toggle at the top when viewing a skill.
- **Edit**: predefined skills provided by Anthropic (like Canvas Design) **cannot be edited directly** — only enabled/disabled/uninstalled. But you can view/copy the skill's underlying logic and turn it into your own customized/extended skill.
- **Sharing skills**: skills can be **downloaded** and given to a colleague, who can **upload** it into their own Claude account to use it.
- Skills can, in principle, be chained — using multiple skills together for one larger task — though this wasn't demonstrated live.

Key takeaways

- Skills live under **Customize → Skills** in Claude; you can Browse predefined skills or create your own.
- Two ways to create a custom skill: (1) **Write skill instructions** manually, or (2) **Create with Claude** using the Skill Creator (which also runs test cases).
- Use / in the chat box to explicitly select a skill; without it, Claude tries to auto-select but isn't guaranteed to pick the intended one.

- A well-defined skill produces a **fixed, consistent output structure** every time, regardless of how much the specific input content varies.
- Combine skills with the word "artifact" (in the prompt or baked into the skill instructions) to make skill outputs publishable/shareable.
- Predefined (Anthropic) skills can't be edited directly, but you can copy their logic into a new custom skill.
- Skills can be downloaded and shared with others, who upload them into their own Claude account.

Demo 9: Claude Memory Settings and Projects

This session covers two major Claude customization features: **Memory** (how Claude can remember things across chats) and **Projects** (dedicated, isolated workspaces for organizing chats, files, and instructions around one topic) — plus a partial demo of building a custom "text-to-speech" skill that calls an external Google Gemini API.

What you'll learn

- The difference between Claude's two memory settings
- How to enable/disable/reset memory and see what Claude remembers about you
- What a Claude Project is and why it's useful
- How to create a project, add files, and set project-specific instructions
- How a custom skill can give Claude abilities it doesn't have natively (e.g., calling an external API)

Step-by-step notes

1. Recap of Skills

Skills only need to be created once but can be used any number of times; predefined skills (like Canvas Design) exist, and you can create your own custom ones.

2. Advanced skill example: giving Claude a capability it doesn't have (Text-to-Speech)

Claude has no built-in text-to-speech (TTS) feature, but Google Gemini does. The instructor built a custom skill ("Gemini TTS") containing step-by-step instructions telling Claude how to call the Google Gemini REST API (using an API key) to convert given text into an audio (.wav) file. The skill's instructions specify: input text, use Gemini 2.5 TTS via the Gemini REST API, ask for the API key as a secret, run a defined Python script, and deliver the result.

- Important clarification: the actual text-to-speech conversion happens on Google's Gemini servers, not inside Claude — Claude just orchestrates the request/response using the skill's instructions and your API key.
- The live demo hit a network issue and didn't fully complete, but the concept was demonstrated: you can also chain a second skill ("Upload to Drive") so Claude can save the generated audio into your Google Drive, even though Claude has no native Google Drive integration either.
- This illustrates that skills can extend Claude beyond its built-in capabilities by teaching it, step-by-step, how to call any external API (the class also discussed extending this idea to video generation via Google Flow, or calling any custom REST/web service with headers and auth parameters).
- Multiple skills can work together (e.g., "generate audio" skill + "upload to Drive" skill) either by naming them in your prompt or by having Claude auto-detect and chain them.

3. Memory Settings — Setting 1: "Search and reference chats"

Location: **Profile → Settings → Capabilities → "Search and reference chats"** ("Allow Claude to search for relevant details in past chats").

- **Disabled by default.**
- When enabled, Claude will search your **old/previous chats** while answering a new question in a different chat — e.g., if you mentioned in one chat "I watched the movie Bahubali, which is excellent," a later new chat asking "which movie did I watch?" can be answered correctly only if this setting is on.
- Demonstrated live: with the setting off, a new chat had zero memory of the earlier Bahubali mention ("I don't have any information about you..."). After enabling it (and reopening/restarting), the same question was answered correctly by referencing the earlier chat.
- Trade-off raised in class: enabling this increases token usage/cost because Claude has to search past conversation data — this is likely why it's off by default. Benefit: more personalized, context-aware answers, especially valuable for long-term projects.

4. Memory Settings — Setting 2: "Generate memory from chat history"

Location: same Capabilities page, second toggle: **"Generate memory from chat history"** ("Allow Claude to remember relevant context from your chats" — controls memory for both chats and projects).

- This is **permanent memory** — Claude periodically (about every 24 hours) summarizes your conversations into a stored profile/context summary, rather than just searching old chats on demand.
- Demonstrated: after telling Claude "Remember me — I'm a software trainer at Durgasoft, we do online/classroom training, my team handles teaching, content creation, student support, tools used are Zoom/Google Sheets/YouTube..." and enabling this setting, a brand-new chat asking "What do you know about me?" correctly summarized this profile back.
- With memory generation disabled, the same question returned "Memory is turned off for your account, so I don't retain anything from past conversations."
- You can view exactly what Claude has stored about you (there's a viewable memory file/summary in settings, refreshed roughly every 24 hours) and edit or delete individual entries (e.g., remove sensitive info like credit card numbers if it was accidentally captured).
- You can also **Pause memory** (temporarily stop adding new memories but keep existing ones) or **Reset memory** (permanently delete all memory, including project memory — cannot be undone).
- You do not need to literally say "remember me" every time — if you enable the setting, Claude captures relevant details automatically from normal conversation. Saying "please remember this" just makes the intent explicit.
- Analogy used in class: Setting 1 (search old chats) is like RAM/temporary lookup; Setting 2 (generate permanent memory) is like a hard disk/permanent storage.
- Memory storage has a limited size (mentioned as roughly ~200 "tokens"/entries worth); when full, older/less relevant data is likely dropped in favor of newer information (exact behavior wasn't 100% confirmed by the instructor). Paid/higher tiers may offer larger memory.

- You can also **import memory from other AI providers** (e.g., a ChatGPT memory export) into Claude.

5. Privacy note on memory/data

A student asked about data security/enterprise restrictions. The instructor pointed to **Settings → Privacy → Data controls**, which explains Anthropic's data handling: with your permission, Anthropic may use your chats/coding sessions to help train/improve their models; Anthropic does not sell your data to third parties; data is deleted promptly on request except where needed for safety violations. Corporate restrictions on using Claude at all (e.g., companies blocking Claude via licensing agreements) are separate from Claude's own settings and depend on the organization.

6. What is a Claude Project?

Defined as: "A dedicated workspace where you can organize chats, files, instructions, and AI tasks related to one specific topic or project." Analogy: like a Java/Python "project" that groups related classes/files together — an isolated workspace unrelated to your other chats or projects.

7. Creating a project — step by step

56. Click **New project** in the left sidebar.
57. Give it a name and description (example built live: "The Richest Man in Babylon" — a personal reading/content project) and describe your goal.
58. Once created, a project page shows three sections automatically:

- **Memory** — project-specific memory that appears after a few chats within that project (separate from your global memory).
- **Instructions** — custom instructions Claude should follow for every chat inside this project (e.g., "response should start with 'Life Skills with Durga'", "answer only from the uploaded reference files — if the answer isn't in the files, say 'Sorry, information is not available'", "respond in simple, concise Indian English").
- **Files** — upload PDFs, documents, or reference material (from device, pasted text, or GitHub) that Claude should use as its knowledge source for that project — conceptually similar to the RAG (Retrieval-Augmented Generation) approach covered earlier in the course.

59. After uploading reference chapters and setting instructions, questions asked inside that project ("What is the main theme of this book?", "Who is the main character?") are answered strictly from the uploaded files, following the defined tone/format — demonstrating project-level isolation (the project won't casually pull unrelated general knowledge once you add a "only answer from provided files" instruction).

8. Advantages of Projects (elicited from class + instructor):

- **Everything in one place** — related chats, files, and instructions live together on one page.
- **No need to repeat instructions** — Claude remembers project context and preferences automatically for every chat inside that project.
- **Consistent responses** — output format/style stays the same across all chats in the project.

- **Supports file uploads** — PDFs/notes/documents can ground Claude's answers (RAG-style).
- **Best for long-term work** — ideal for courses, books, business plans, research projects, or (as a student suggested) storing an API guide/spec so Claude can generate user stories or consistent documentation from it.

Key takeaways

- Two separate memory toggles exist under Settings → Capabilities: **"Search and reference chats"** (search old chats on demand, off by default, costs more tokens) and **"Generate memory from chat history"** (build a permanent, periodically-updated summary/profile, off by default).
- You can view, edit, delete, pause, or fully reset what Claude remembers about you at any time.
- Skills can extend Claude beyond its built-in features by teaching it, step by step, how to call external APIs (e.g., Google Gemini for text-to-speech, or Google Drive for file uploads) — Claude orchestrates the call using the skill's instructions and your provided credentials.
- A **Project** is an isolated workspace combining chats + memory + custom instructions + reference files around one topic, useful for long-running work where you don't want to repeat context every time.
- Project instructions can enforce answering strictly from uploaded files, effectively creating a simple RAG-style, scoped assistant.

Demo 10: Claude Styles — Controlling How Claude Talks, Writes, and Responds

This session covers Claude's "Styles" feature — the built-in response styles (Normal, Concise, Explanatory, Formal, Learning) and, most notably, how to create your own fully custom style (including a live demo of a "Durga Sir style" and a Hinglish teaching style). The instructor also notes that Styles were being migrated into the Skills feature around May 20th.

What you'll learn

- What a "style" is in Claude and how it differs from a skill
- The built-in predefined styles and what each one does
- How to switch styles for a single question
- How to create and save your own fully custom style
- How to build a multilingual/regional-language teaching style
- The relationship between styles and skills going forward

Step-by-step notes

1. Recap and definition

A "style" in Claude means **the way Claude talks, writes, and responds** — its tone, structure, and communication approach. This connects back to earlier prompt-engineering lessons about asking for a "friendly style," "professional style," "concise vs detailed," or "beginner vs advanced" responses — Styles turns that into a persistent, selectable setting instead of something you type every time.

2. Where to find styles

In a chat, click the **+ (plus)** icon in the chat box → options include Add files, Add to project, Skills, Connectors, Web research, and at the end, **"Use a style."** (Note: at the time of recording, Claude showed a banner saying "styles are migrating to skills starting May 20" — so this feature was being merged into the Skills system.)

3. Built-in predefined styles demonstrated

Using the same test prompt ("Explain about N8N") with different styles selected:

- **Normal** — Claude's default balanced response style.
- **Concise** — a much shorter, to-the-point answer compared to Normal.
- **Explanatory** — noticeably more detailed, described by Claude itself as "a deep dive," with more elaboration than Normal or Concise.
- **Learning** — turns the response into an interactive, Socratic teaching conversation: Claude first asks about your background ("Are you completely new? Do you have experience with Zapier or Make?"), then teaches step by step using relatable everyday examples (e.g., comparing N8N automation to manually copying an email invoice into a spreadsheet and messaging an accountant), checks

understanding with questions, introduces vocabulary gradually (e.g., defining "workflow" and "node" only after building intuition), and gives you small challenges/exercises along the way.

- Only **one style can be active at a time** — you cannot combine multiple predefined styles simultaneously.

4. Creating your own custom style — Method 1: Describe your style directly

60. Click **+ → Use a style → Create and edit styles → Create custom style**.
61. You can either paste a writing sample (Claude infers the style from example text — content isn't stored beyond matching the style) or **describe the style** in instructions.
62. Example instructions used to build a "Durga Sir style": "I am an Indian classroom trainer. Speak in very simple Indian English. Teach in a friendly, energetic manner. Use conversational classroom style. Frequently use words like 'friends', 'have you got my point?', 'very very important', 'simple logic friends', 'left and right', 'observe carefully'. Explain concepts step-by-step with real-life examples — first explain why the concept exists, then..."
63. Note: overly long/complex instructions (with bullet points/emojis) initially caused an error — simplifying/shortening the instructions and removing special characters fixed it.
64. Click **Create**, then rename the style (e.g., "Durga sir style") for easy selection later.
65. Select it via **+ → Use a style → Durga sir style**, then ask a question (e.g., "Explain about N8N") — Claude responds peppered with the specified catchphrases and teaching rhythm, matching the described persona.

5. Creating your own custom style — Method 2: "Generate the style"

An alternative creation flow lets you choose to define a style by objective/audience or by voice/tone, or use "Generate the style," where Claude helps draft it for you rather than you writing full instructions yourself.

6. Multilingual/regional styles

Demonstrated creating a **Hinglish** (Hindi + English mixed) version of the "Durga Sir" style, and mentioned this works for any language/regional dialect (Telugu, Marathi, Telangana-flavored English, etc.) — you simply specify in the style's instructions to mix languages naturally and include locally familiar phrases. When selected, Claude's response comes out as a genuine Hindi-English (or other language) mix in that persona's voice, which the class noted is especially valuable for learners more comfortable in their native language than in formal English.

7. Styles apply to voice mode too

A student asked whether a selected style also applies when using Claude's voice/microphone chat mode — the instructor confirmed that once a style is selected, it should apply consistently whether you type or speak (a live voice-mode test was blocked only by a browser microphone permission issue during the Zoom recording, not a Claude limitation).

8. Styles vs. Skills — the key distinction clarified in Q&A

A student asked: "Skills define what data to use and how to think/respond; is Style purely about formatting, not content?" The instructor confirmed: **yes, Style is only about tone/format**, not about instructing Claude on data or reasoning — that's the domain of Skills. This is exactly why, starting around May 20th, Anthropic was migrating the Styles feature to be part of the broader Skills system (a banner about this migration was visible in the UI during the session).

Key takeaways

- A Style controls **how** Claude communicates (tone, verbosity, teaching approach) — not what data or logic it uses (that's a Skill's job).
- Built-in styles include Normal, Concise, Explanatory, and Learning (interactive, Socratic-style teaching) — only one can be active at a time.
- You can build a fully custom style by describing your persona, tone, catchphrases, and explanation approach — useful for consistent branding/teaching voice.
- Custom styles can be multilingual/regional (e.g., Hinglish, Telugu-mixed), which is especially helpful for learners more comfortable outside pure English.
- Selected styles apply across both text and voice chat.
- Styles were in the process of merging into the Skills feature (per an in-app migration notice around May 20th), reflecting that Styles is really a formatting-focused subset of the broader Skills capability.

Demo 11: Claude Connectors — Turning Claude into a Digital Assistant

This video introduces the concept of Claude Connectors, showing how connecting Claude to Gmail, Chrome, and web search transforms it from a simple chatbot into an assistant that can take real actions on your behalf.

What you'll learn

- What a Claude connector is and why Claude needs connectors to be useful beyond simple Q&A
- The difference between "Claude alone" (a brain with no hands), "Claude + one connector" (an AI assistant), "Claude + many connectors" (a digital employee), and "Claude + many connectors + automation" (a digital workforce)
- How to open the Connectors panel and add/remove connectors
- How to connect the Gmail connector and use it to ask Claude to check your inbox and draft/send emails
- The difference between read-only tool permissions (no approval needed) and write/delete permissions (which require your approval)
- Real examples of using Claude with web search plus a connector to research information and email it to you
- The business advantages of using connectors

Step-by-step notes

66. **Why connectors matter.** The instructor compares Claude to a very talented employee stuck in a room with no internet or phone — no matter how smart, that person can't be useful without a way to interact with the outside world. Similarly, Claude by itself only answers questions; it cannot open a website, check your email, or send messages unless it is connected to those tools.
67. **What a connector is.** A connector is defined as "a bridge that allows Claude AI to connect with external applications and services." Examples given: the Gmail connector lets Claude read/send email, the Chrome connector lets Claude control your browser, a Slack connector connects to Slack, a Calendar connector connects to Google Calendar, and so on.
68. **The escalating power levels:**

- Claude alone = a simple chat/LLM ("just a brain")
- Claude + one connector = an AI assistant (it can now access data and act, e.g., send a mail)
- Claude + many connectors = a "digital employee" (can do many kinds of tasks across apps)
- Claude + many connectors + automation = a "digital workforce" (can replace parts of what a whole software team/company does, running without a human triggering each step)

69. **How to open Connectors in Claude.** Go to the "+" (plus) icon in the chat box, or Customize → Connectors. From there you can "Browse connectors" or "Add custom connector." The instructor shows a long list of available predefined connectors including Gmail, Google Drive, Google Calendar,

Slack, Notion, Canva, Microsoft 365, Figma, Adobe, Postman, Metabase, ClickUp, Shopify, HubSpot, Turkish Airlines, and more — you enable only the ones relevant to your use case.

70. **Connecting Gmail.** Click the "+" next to the Gmail connector, sign in with your Google account, and grant permission. Once connected, Claude shows a list of "tool permissions" it has: read-type actions (e.g., "retrieve a specific email thread," "list labels," "search email threads") work automatically, while write/delete actions (create a draft, create/delete a label) require your explicit approval each time before Claude will do them.
71. **Claude in Chrome.** The instructor also mentions the "Claude in Chrome" connector, which lets Claude control browser tabs. (Note: this needs a Mac in this session's context — the instructor says a full walkthrough would come in a later class since the extension needed macOS at that point.)
72. **Live demo — reading Gmail.** With Gmail connected, the instructor asks in a new chat: "Check my Gmail and provide the top most important mail I have to address today." Claude reads the inbox, evaluates urgency, and reports back the most pressing item (e.g., a student-reported technical issue) — showing it can genuinely access and reason over real inbox data.
73. **Live demo — web search + course info.** The instructor asks Claude to find "Agentic AI related batches in Durgasoft" and present them in table form. Claude uses its built-in web search capability (not a special connector) to look up the website and returns a clean table with batch dates and contact info.
74. **Live demo — search the web and email results.** The instructor asks about IPL 2026 cricket match info and other real-time topics (flight prices, iPhone prices across e-commerce sites, interview prep for Google) and then asks Claude to email the results. Key point: for the *research* part, Claude just uses web search (no connector needed); for actually *sending the email*, the Gmail connector is what makes that possible. Claude always prepares a **draft** first rather than sending automatically — you must review and manually hit send in the Gmail chat app (this "objection/review before sending" behavior was noted as expected/safe behavior, distinct from full automation which can send without a human step).
75. **Practical takeaway examples.** The instructor demonstrates several use cases: finding the cheapest online price for a product across multiple e-commerce sites, researching a company before a job interview, finding the cheapest flight dates for a trip, and asking Claude to predict things (like a sports outcome) — noting Claude's answers aren't guaranteed to be 100% accurate but are a useful starting point.

Key takeaways

- Without connectors, Claude only "knows things" — with connectors, it can "do things" (send mail, browse, act in external apps).
- Advantages of connectors: saves time, reduces manual work, provides real-time information, automates repetitive tasks, increases productivity.
- Predefined connectors cover most common apps (Gmail, Drive, Calendar, Slack, etc.); custom connectors can also be built for internal/company-specific tools.
- Read-only actions from a connector generally don't need approval; write/delete actions do.

- Claude currently drafts emails rather than sending them outright in chat mode — true "send without asking" automation is reserved for workflow/automation features (covered in later videos).
- If a connector is for a paid third-party service (e.g., Canva), you still need your own paid subscription to that service — the Claude subscription only covers using Claude itself.
- The instructor's advice: don't waste Claude on trivial one-line questions — use it for bigger, business-level goals since that's where connectors show their real value.

Demo 12: More Connector Examples — Google Drive, Chrome Extension, and Automated Email Replies

This video builds on Demo 11 by adding the Google Drive connector to a real end-to-end workflow (auto-replying to a course inquiry email with a syllabus attached), and then introduces the Claude Chrome extension for controlling the browser directly, plus a first look at Claude's data analysis ability.

What you'll learn

- How to connect and use the Google Drive connector alongside Gmail
- How to build a multi-step workflow: read an email inquiry → look up course info on a website → fetch a file from Google Drive → compose a professional HTML reply → save it as a draft
- The distinction between chat-based Claude (which asks for approval before sending) and n8n/automation workflows (which can send without asking) — previewed here, covered fully in the co-work/automation topics
- How to install and use the "Claude for Chrome" browser extension to let Claude act inside your browser (open tabs, search, send WhatsApp messages, etc.)
- Early limitations noticed: PDF handling consumes more credits, permission approval is needed for many actions in chat mode, and usage credits/limits can run out
- A first example of Claude analyzing uploaded data (foreshadowing the deeper data-analysis demo in Demo 13)

Step-by-step notes

76. **Setting up the Google Drive connector.** Same process as Gmail: Customize → Connectors → Browse Connectors → click "+" next to Google Drive → sign in and grant access. Google Drive's tool permissions also separate read operations (download file, get file metadata — usually pre-approved) from write/delete operations (which need explicit approval).
77. **Building the syllabus.** Before the live demo, the instructor prepared a "Durgasoft Cloud AI course syllabus" using ChatGPT, saved it as a PDF locally, then manually uploaded that PDF into a folder in Google Drive so Claude would have something real to fetch during the demo.
78. **End-to-end workflow example.** With Gmail and Google Drive both connected, the instructor sends this kind of prompt in a new chat: *"Check my Gmail, and if there's an inquiry about the Cloud course, check Durgasoft for new batch information on the cloud, get the syllabus from Google Drive, compose a colorful and professional response, and send a reply to that mail."* Claude then, step by step and visibly: searches Gmail threads, finds the inquiry email, fetches batch details from the Durgasoft website, searches Google Drive for the syllabus PDF, and composes a complete HTML-formatted reply — pulling from three different sources into a single polished response.
79. **Sending is still manual in chat mode.** Claude prepares a **draft** in Gmail rather than sending automatically. A student asks why — the instructor explains this is because Claude, when used through chat, intentionally asks for review before sending; true "send without a human clicking

Send" only happens in automation tools like n8n workflows (discussed in earlier related sessions, not this video in detail).

80. **Reverse workflow example.** The instructor also shows the opposite direction: ask Claude to check Gmail for "how many students requested Java interview questions in the last month," summarize the results, generate a PDF report, and upload that PDF back into Google Drive. This demonstrates connectors working together in both read and write directions.
81. **Practical warning on PDFs and credits.** Several students note that PDF creation/handling consumes noticeably more usage credits than working with Word or Excel files, and that Claude has usage credit limits that can be hit mid-session (the instructor's own session hits a "you are out of usage credits" message during the demo).
82. **Handling spam email with Claude.** A student asks how to make Claude clean up spam. The instructor's answer: describe your general spam-detection rule to Claude in plain language (e.g., "go to my spam folder, and if a mail looks like genuine spam, delete it; otherwise label it not spam") rather than describing each email individually — Claude can then apply that rule across the whole folder.
83. **Downloading email attachments.** A student demonstrates that Claude (via the Gmail connector) can locate an email by subject line, download its attachment, and generate a downloadable file link directly in the chat.
84. **Introducing the Claude for Chrome extension.** Search "Claude for Chrome" in Google, install it from the Chrome Web Store, pin it to the toolbar, and authorize it. Once installed, you can open the extension in any tab and give it instructions — it will act inside your actual browser (opening new tabs, searching Google, navigating pages) rather than just replying in a chat window.
85. **Chrome extension demo — live web actions.** The instructor asks the extension "What IPL matches are remaining?" and watches Claude open a new tab, search Google, read the results, and answer — with an "Approve Plan" confirmation step shown before it acts. He also asks it to find the highest-viewed video in a YouTube playlist, summarize it into five bullet points, and then send those bullet points to a WhatsApp contact via the already-open WhatsApp Web tab — all done by Claude operating the browser directly, similar to browser automation tools like Selenium/RPA but driven by natural-language instructions instead of code.
86. **Limitations observed live.** The Chrome extension ran into usage-credit limits mid-demo; also, some actions required a paid/Pro plan. YouTube transcript summarization sometimes failed for certain videos (older videos worked better than newer ones in this session). The instructor notes he'll continue exploring this feature in a later session.

Key takeaways

- Multiple connectors (Gmail + Drive) can be chained together in a single prompt to build a realistic, multi-source workflow (read inquiry → look up info on the web → fetch a file from Drive → send a professional reply).
- Claude in chat mode drafts rather than auto-sends emails; only automation tools truly "send" without a human click.

- The Claude for Chrome extension lets Claude directly control your browser tabs (open pages, search, interact with WhatsApp Web, etc.) — conceptually similar to Selenium/RPA browser automation, but driven by plain-English prompts instead of code.
- PDF-heavy tasks consume more credits than Word/Excel-based tasks — prefer Word/Excel where possible to save credits.
- Reverse workflows (data → summarized file → uploaded back to Drive) work just as well as forward workflows (email → info lookup → reply).
- For paid third-party connectors (e.g., Canva), you must separately hold a subscription to that third-party service.

Demo 13: Chrome Extension in Practice, and a Deep Dive into Data Analysis with Claude

After a short motivational segment on positive thinking, this session shows more live examples of the Claude Chrome extension (including sending messages to a WhatsApp group) and then spends the bulk of the video walking through a detailed, prompt-by-prompt data-analysis demo using a sample Excel sales dataset.

What you'll learn

- More live examples of the Claude Chrome extension acting inside the browser (reading a YouTube video, checking IPL live scores, posting a message to a WhatsApp group)
- Why some actions may need repeated "Approve Plan" confirmations, and how this differs from RPA tools like Selenium/UiPath (no scripting required, but a chrome/system access grant is required)
- A structured walkthrough of using Claude to perform business data analysis on an uploaded Excel file, using more than a dozen different prompt styles
- How to frame prompts as different "personas" (CEO, investor, marketing manager, business auditor) to get different types of analysis from the same data
- The difference between "data analysis" (analyzing existing data) and "data science" as separate but related fields
- How Claude uses its built-in analysis/code-execution skill (running behind-the-scenes computations) to generate dashboards, charts, and HTML reports from raw data

Step-by-step notes

87. **Chrome extension demo — YouTube summarization.** The instructor opens the Claude Chrome extension on a YouTube video and asks for "notes for this video in just one point / one point summary." Claude tries to extract the transcript from the page; when a video's transcript isn't available/enabled, it falls back to trying other data sources. The instructor notes that if you go to YouTube directly and use Claude there (rather than through the general extension), transcript-based summarization tends to work more reliably.
88. **Chrome extension demo — live cricket score.** He asks "what is the current score of the IPL match today," and the extension opens a live score page itself, reads it, and reports back an analysis of the current match situation (score, required run rate, who is likely to win) — all without the instructor manually navigating anywhere. Each action needs an "Approve Plan" click before Claude proceeds, which several students note is a friction point compared to a fully autonomous run.
89. **Chrome extension demo — sending a WhatsApp group message.** With WhatsApp Web already open in a browser tab, the instructor asks Claude (via the extension) to send the current match score to a specific WhatsApp group ("Life Skills with Durgasoft"). Claude finds the group in the open tab, drafts the message, asks for confirmation ("Can you confirm you want to send this message to

Life Skills with Durgasoft?"), and after approval the message is posted automatically — visible to the group members in real time.

90. **Comparison with RPA/Selenium.** A student compares this to Selenium/RPA tools they already use. The instructor's key distinction: with Selenium you must write a script (he estimates 50+ lines of code for a comparable task and you also pay for RPA tool licenses); with the Claude extension, you only need to grant two permissions — access to your system files and access to your Chrome browser — and describe the task in plain English, no coding required.
91. **Transition to data analysis.** The instructor introduces a sample Excel file ("Durgasoft online course sales," 120 records, generated artificially via ChatGPT for demo purposes — not real data) with columns like order ID, date, student name, city, course name, instructor, fee, discount, final fee, payment mode, and status. He uploads this Excel file into a Claude chat and runs a series of prompts to show different analysis angles.
92. **Prompt: CEO summary.** *"You are CEO of Durgasoft. Analyze this sales data and explain total revenue, total students, top-selling course, lowest-selling course, best instructor, best city. Provide the answer in simple business language."* Claude reads the file, runs calculations, and builds a full visual executive dashboard (bar charts by course, by city, by instructor; payment status breakdown) plus written CEO-style "key action items" (e.g., chase 30 pending payments immediately, boost the underperforming course, address regional weaknesses).
93. **Prompt: Find the hero course.** *"Analyze the data set and identify which course gets maximum revenue, which course has maximum enrollments, which course should Durgasoft promote more, and explain why."* Claude identifies the highest-enrollment vs. highest-revenue courses, spots a pricing gap between two similar courses, and recommends which one to push harder in marketing, with reasoning.
94. **Prompt: Instructor performance comparison.** *"Compare all instructors: number of students handled, revenue generated, average fee collected. Declare the best instructor of the month."* Claude ranks instructors and produces a comparison table with recommendations (e.g., reward the top performer, expand their course load).
95. **Prompt: Shark-Tank/investor-style analysis.** *"Imagine you are an investor. Would you invest in Durgasoft based on this data? Provide strengths, weaknesses, opportunities, and risks. Give your final decision."* Claude produces an "investor scorecard" (revenue momentum, market fit, pricing strategy scores), a conditional "yes, I'd invest, but fix these problems" verdict, and flags a concrete business risk: one instructor generating 58% of revenue is a single point of failure ("existential risk," not just a business risk), plus a 25% unpaid/pending-payment rate that needs fixing.
96. **Prompt: City-wise expansion plan.** Ranks cities by revenue and enrollment, flags priority target cities, and even suggests new cities to expand into based on market demand patterns.
97. **Prompt: Business dashboard creation.** *"Create a professional dashboard using total revenue, total students, top course, top instructor, revenue by city, revenue by course. Use charts and tables."* Claude generates a downloadable HTML page with KPI cards and charts. (The instructor notes this dashboard is a regular HTML file output, not a Claude "Artifact.")

98. **Prompt: Hidden insights / future prediction.** *"Find interesting insights from this dataset... Predict next month's revenue, expected student enrollment, and courses that may decline. Explain your logic."* Claude produces forward-looking projections (e.g., a specific forecasted revenue figure and enrollment range for the following month) along with reasoning based on trend growth rates.
99. **Prompt: Marketing budget allocation and "keep only 3 courses" persona.** Further prompts ask Claude to act as a marketing manager (allocate a fixed budget across courses/cities) and, more provocatively, as "Durgasoft himself" deciding to keep only 3 of the current 6 courses — Claude picks specific courses to keep/drop with clear reasoning based on revenue trends, tied explicitly back to the underlying numbers.
100. **Editing the generated dashboard.** The instructor asks Claude to change a label on the dashboard (the quarter shown at the top) directly through a follow-up prompt, and Claude updates the HTML output accordingly.
101. **Q&A: What's happening "under the hood."** A student asks whether this data analysis relies on MCP servers or connectors. The instructor and other students clarify: no special connector is needed for this — Claude is using its **built-in skills** (visible in the tool trace as things like "using JavaScript tool" / reading a "skill" file) to run calculations and generate outputs, similar to writing and running a small script internally.
102. **Comparison with GitHub Copilot.** Several students discuss GitHub Copilot in the chat — the general conclusion is that Copilot is mainly focused on code generation and code review (e.g., auto-reviewing pull requests) rather than broad document/data analysis, though it can do some of that "not up to the same mark," and Copilot has recently added Claude as one of its selectable underlying models because of its strength at coding.

Key takeaways

- Data analysis with Claude requires no SQL, Excel formulas, or manual charting — one well-written prompt against an uploaded spreadsheet can produce full dashboards, rankings, and written business recommendations in about a minute.
- You can reframe the same dataset through different "personas" (CEO, investor, marketing manager, business auditor, "you are Durgasoft") to get very different, business-relevant angles on the same numbers.
- This kind of automated analysis is powered by Claude's built-in skills (internal code execution), not by an external connector or MCP server.
- The Chrome extension can fully operate your browser (search, navigate, message contacts) but currently needs "Approve Plan" confirmation at each major step; that's a safety/consent feature, not a permanent limitation.
- Compared to Selenium/RPA scripting, the Chrome extension needs no code — only browser + system access grants and a plain-language instruction.
- Data analysis and data science are related but distinct: data analysis works with existing data to draw conclusions; data science is a broader/different discipline.

Demo 14: Introducing Claude Cowork — Your AI "Digital Employee" on the Desktop

This session introduces Claude Cowork (the desktop-based agent capability), explaining conceptually what it is, what it can do, its security model, and who should use it, followed by live demos of it creating real files on the instructor's computer.

What you'll learn

- What Claude Cowork is and how it differs from the regular Claude chat (it performs real actions on your computer, not just conversation)
- The full list of things Cowork can do: read folders, create real files, browse the web, install/use plugins (connectors), run on a schedule, and be triggered remotely ("dispatch")
- Cowork's security model: sandboxed execution, folder-level access control, one-time vs. persistent permissions
- Who should use Cowork and for what kinds of tasks
- Two live demos: generating a formatted Word document from scratch, and researching a topic online to build a multi-section Word report

Step-by-step notes

103. **What Cowork is.** The instructor defines it directly: *"Claude Cowork is an AI assistant that can do real work on your computer."* Instead of just answering questions in a chat window, Cowork can read files, create documents, browse the internet, connect with tools, and run tasks automatically — described as "a smart AI employee that works with you and for you."
104. **Example use cases described (before the demo).** Things like: "In my D drive there's a Durgasoft classes folder — create a file named Java Questions with 25 Q&A, red questions/blue answers, with page numbers and headers" or "Separate all the mixed Java and Python files in this folder into two separate folders." The point: any repetitive file/folder "labor work" can be delegated to Cowork instead of doing it by hand.
105. **What Cowork can do — the full list:**

- **Read folders**: it can read many files (Excel, PDF, Word) inside a folder you grant access to and summarize them.
- **Create real files**: it creates actual files on your system (Excel, PDF, Word, presentations) — not just text in a chat window.
- **Browse the web**: it can research topics online and pull findings directly into the documents it creates.
- **Install plugins**: this is effectively the same idea as connectors (Gmail, Slack, Calendar, Drive) — Cowork can talk to external apps too.
- **Run on schedule**: like a cron job — e.g., "every day at 9 AM, send me a website sales report" — Cowork runs automatically without you prompting it each time.

- **Dispatch**: you can trigger Cowork tasks remotely from your phone, like a remote control for the desktop app, even while away from your computer.

106. **Security model.**

- Cowork runs inside a sandboxed/protected virtual-machine-like environment, not directly against your system resources, reducing malware/security risk.
- You control access at the folder level — you explicitly grant Cowork permission to a specific folder, and it cannot touch anything outside that folder.
- Permission prompts can be answered "Allow once" or "Allow always" for a given folder/task.
- When connecting external tools (Gmail, Slack, Calendar), you can restrict Cowork to specific operations (e.g., read-only, inbox-only).

107. **Who should use Cowork.** Anyone doing repetitive work involving: files and folders (organizing, searching, summarizing), documents and reports (meeting notes, project reports, business documents), data and spreadsheets (Excel analytics, sales/enrollment reports), and recurring tasks (daily/weekly/monthly reports that can be scheduled).

108. **Example prompts described (conceptual, before live demo):**

- "Read 20 PDF notes and create exam revision notes."
- "Read 200 emails and provide important action items." (A student flags that this will consume a lot of credits/tokens — the instructor agrees this is a real tradeoff and recommends avoiding heavy PDF use where possible, and choosing a lower-cost model when available.)
- "Read Durgasoft's daily enrollments, collections, and inquiries and send a business summary every night at 9 PM" — a recurring, scheduled task.

109. **Live demo 1 — creating a Word document from scratch.**

- Setup: create an empty folder (e.g., "Durgasoft Cowork 2") and, in the Claude desktop app, start a **new task** (Cowork mode, distinct from "new chat") and point it at that folder. The first time, Claude asks for permission ("Allow this change to files in [folder]... includes all files and subfolders... Always allow / Cancel") — choose "Always allow" to avoid being asked repeatedly for that folder.
- Prompt used: *"Create a Word document called 'durgasoft-cowork-getting-started.docx' in this folder. The document should include the title 'Getting Started with Claude Cowork at Durgasoft', today's date, and a numbered list of five practical Cowork tasks related to student enrollments, email management, report generation, document creation, and daily automation. Format it professionally with a header, footer, and page number."*
- Result: Cowork creates the actual .docx file in the folder, with the requested title, date, formatted header/footer/page numbers, and five specific automation task ideas written out — no manual document editing done by a human.

110. **Live demo 2 — web research + report generation.**

- Prompt: *"Create a Word document called 'AI tools for students.docx' in this folder. Research the top 10 AI tools useful for students in 2026. For each tool include its purpose, key features, and one practical use case. Add a cover page, table of contents, today's date, page numbers, and professional colorful formatting."*
- Result: Cowork researches the web, then produces a multi-section Word document with a cover page, a table of contents listing all 10 tools (ChatGPT, Claude, Google Gemini, NotebookLM, Grammarly, Perplexity, etc.), and a dedicated section per tool covering purpose, key features, and a practical use case.

111. **Discussion points raised by students:**

- Cowork ultimately relies on an underlying LLM to do the reasoning — in principle the "Cowork interface" and "which LLM powers it" could become separate/interchangeable in the future (a student speculates about connecting cheaper/local LLMs to reduce cost).
- Cowork does not have video-generation capability — it's for real productivity/file/document tasks, not media generation.
- A student notes this could be used to auto-generate user stories from functional/technical specification documents in real projects — the instructor confirms this is a realistic use case.
- Cowork currently runs on the **desktop app only** (not the web version) — this is why a mobile "dispatch" trigger is valuable, since it lets you kick off a task remotely even though execution happens on the desktop app.
- A student working on Linux mentions there's no native Linux desktop app yet (Windows/Mac only at this point), though some community-built Debian-based install options were being explored.

Key takeaways

- Cowork = Claude acting as a "digital employee" that performs real, file-level and document-level work on your actual computer, not just chatting.
- Core capabilities: read folders, create real files, browse the web, use plugins/connectors, run on a schedule, and be dispatched remotely from mobile.
- Security is handled via sandboxing plus folder-scoped, explicit permission grants (Allow once / Allow always) — Cowork cannot silently touch files outside the folder you grant it.
- Heavy operations (many PDFs, hundreds of emails) consume more credits/tokens — plan prompts accordingly and prefer Word/Excel over PDF where possible.
- Cowork currently requires the desktop app; the web version doesn't yet support it.
- Any recurring "reporting" or "document generation" job that a human currently does manually and repeatedly is a strong candidate to hand off to Cowork.

Demo 15: Cowork in Action — Real Business Documents, Web Scraping, and the CCOR Prompting Framework

This session is a rapid-fire showcase of practical Claude Cowork prompts: extracting content from web pages into documents, generating sample data and full analytical reports, building entire sets of business templates in one prompt, and a preview of a structured prompting framework (CCOR) for working with Cowork.

What you'll learn

- How to use Claude (chat and Chrome extension) to extract entire web page content into a clean Word document
- How to generate realistic sample data and a full analytical Word report from a single prompt
- How to have Cowork build a complete folder of business templates (registration form, fee receipt, welcome email, batch info) in one shot
- How to have Cowork build an entire "management system" folder (student database, fee tracker, faculty schedule, revenue dashboard, etc.) from a single detailed prompt
- A first introduction to the CCOR prompt structure (Context/Command, Constraints, Output format, Rules) as a way to write better Cowork prompts
- Practical limitations noticed: browser-extension doc creation vs. chat-based doc creation, voice input, and why very detailed prompts get better results

Step-by-step notes

112. **Recap of Demo 14's two examples.** The instructor briefly re-shows the "Durgasoft Cowork getting started" Word doc and the "AI tools for students" research report created in the previous session, reinforcing that Cowork created both files entirely on its own once given a folder and a prompt.
113. **Real-world story: data-driven negotiation.** Before the demos, the instructor recounts asking Claude to analyze Durgasoft's sales data and act as an investor — Claude flagged that 54% of revenue comes from one instructor and recommended reducing that dependency to 30%, plus building systems/processes rather than relying on one person. He shares this as a real example of how these tools already influence his actual business decisions.
114. **Extracting a landing page into a Word document (Chrome extension).** Using the Claude Chrome extension on an actual course-landing-page website, the instructor's prompt is essentially: *"Prepare a doc file with content from this page without missing anything. You can make it more elaborate with proper colorful headings."* The extension reads the whole page (headings, before/after claims, module breakdown, FAQs, testimonials, mentor bio) and produces a structured document — though in this run it couldn't directly generate a downloadable file through the browser and instead returned the content in chat, which the instructor then had to save manually.

On a second attempt from a saved example, the equivalent full FAQ/testimonial extraction worked and produced a complete downloadable doc.

115. **Extracting a course page (chat + URL, not the extension).** For a Udemy-style course page, the instructor compares two approaches: (a) the Chrome extension trying to expand/read all sections directly in the browser — this hit some content/output-length limitations and only got course headings, not full lecture text; (b) pasting the page **URL** into a regular Claude chat and asking it to "create a doc file with the content from this URL" — this fetched and organized the content (course overview, statistics, highlights) more successfully, though the instructor notes prompts need to be specific (e.g., "extract the section-wise syllabus in detail") to get complete results.

- A student compares this to web-scraping/RPA tools; the instructor agrees it's conceptually similar to Selenium-style scripted automation, but here you describe the task in plain English instead of writing a script.

116. **Prompt: generate sample data + build an analysis report (one prompt).** In the shared Cowork folder: *"Create an Excel file called student-enrollments.xlsx with sample data for 100 students. Include student name, course name, enrollment date, fee paid, payment mode. Then analyze the data and create a Word report called enrollment-summary-report.docx containing: total enrollments, course-wise enrollments, total revenue, top-performing course, charts and graphs. Format the report professionally with headings, page numbers, and conclusions."*

- Result: Cowork generates a realistic 100-row Excel dataset from scratch, then a full Word report with total revenue, average fee, course-wise enrollment counts, top-performing course ("Web Development," 17 enrollments), payment-mode breakdown, multiple charts (course-wise enrollments, revenue by course, payment mode distribution, monthly enrollment trends), and a written conclusions/recommendations section (e.g., focus marketing on high-demand courses, improve payment-mode flexibility).
- The instructor's point: this replaces needing to separately know Tableau, Power BI, or SQL reporting — the entire "generate data → analyze → visualize → write conclusions" pipeline happens from one natural-language prompt.

117. **Prompt: build a complete folder of admissions templates.** *"Create a folder called Durgasoft Admissions. Inside, create: student registration form, fee receipt template, welcome email template, batch information. Make all documents professionally formatted and ready for actual student admissions."*

- Result: Cowork creates the folder and all four files: a registration form (personal info fields + signature lines), a fee receipt template, a batch information sheet (course modules, attendance policy), and a welcome email template addressed to "Dear [Student Name]" with placeholders for course, dates, and contact info.
- The instructor notes you can iterate further, e.g., "add our logo to every template file," and Cowork will apply that change across all the generated files.

118. **Prompt: generate a full course brochure.** *"Create a professional brochure document called 'Python with AI brochure.' Include course overview, syllabus, faculty profile, duration, fee details, benefits, contact information. Use attractive headings, icons, colors, tables, and professional formatting."*

- Result: a full multi-page brochure with a 12-week curriculum breakdown, named (placeholder) instructors, pricing tiers, and contact details — written out to roughly 30,000+ characters in one generation pass.

119. **The "big" prompt — full management system in one shot.** *"You are the operations manager of Durgasoft. Create a complete training institute management package including: student database, fee collection tracker, faculty schedule, daily revenue dashboard, admissions form, welcome letter, monthly management report. Organize everything inside a folder called 'Durgasoft Management System' and interconnect the reports wherever possible."*

- Result: Cowork creates the whole folder with all listed files (student database with ID/name/email/phone/course fields, a fee collection tracker with transaction/due-date fields, a faculty schedule, a daily revenue report, and a monthly management report with key metrics) — demonstrating that a single, well-specified prompt can bootstrap an entire small back-office system.

120. **Voice input.** A student asks about giving voice commands. The instructor shows there is a "press and hold to record" voice option, but notes it's just converted to text before being processed (not a separate always-listening/interactive mode), and typing is generally more reliable and predictable than speech-to-text for this kind of detailed instruction.

121. **Introducing the CCOR prompting framework (preview only).** The instructor references a memory aid from childhood learning (RTC-style acronyms) and introduces **CCOR** as a way to structure Cowork prompts: **C**ontext/**C**ommand (what task to do), **O**utput format (what the result should look like), **R**ules/restrictions (constraints to follow). He says he will cover this framework with worked examples, along with "dispatch," in the next session — this video only introduces the acronym.

Key takeaways

- A single, detailed, well-structured prompt to Cowork can generate an entire folder of real business documents (forms, receipts, templates, brochures, dashboards) in minutes — work that would otherwise take hours.
- For extracting web page content into a document, either the Chrome extension (acting directly in the browser) or a regular chat with the page's URL can work — the URL-in-chat approach tended to be more reliable for longer/paginated content in this session, but prompts need to be specific about depth of detail.
- Claude can generate realistic sample datasets and then immediately analyze them (calculations, charts, written conclusions) in the same request — no separate BI tool needed.

- Iterative refinement works: you can ask Cowork to add a logo, change a date/label, or expand a section after the first draft is generated.
- Voice input exists but is just transcribed to text before processing — it isn't a distinct interactive mode, so typing detailed prompts remains the more reliable method.
- The upcoming CCOR framework (Command/Context, Output format, Rules) is presented as a structured way to write more effective Cowork prompts, to be covered with examples in the next session.

Demo 16: Balanced Life, Debugging n8n Workflows with Claude, and the CCOR Prompt Format

This session opens with a motivational talk about work-life balance, then shifts into a real story of debugging an n8n automation workflow purely by copy-pasting error screenshots to Claude, and closes with a new prompt framework called CCOR (Context, Command, Output format, Rules).

What you'll learn

- Why balancing health, family, friends, hobbies, spirituality, finance, and work matters (a life-skills discussion, not a technical lesson)
- How to reframe "AI might take my job" as an opportunity instead of a threat
- A real example of solving a broken n8n automation entirely by pasting screenshots and console logs into Claude, without reading or understanding the generated code
- The CCOR prompt format: Context, Command, Output format, Rules — a simpler alternative/companion to the earlier RTC framework for use with agentic AI tools

Step-by-step notes

1. Life balance discussion (motivational, non-technical)

The instructor tells a story about a person who kept raising his earning target (1 crore, then 5, then 10, then 100 crores) while neglecting family and health, and died before enjoying the money — illustrating that "you can't take money with you." The lesson: balance seven areas of life — health, hobbies, spirituality, friends, love/family, money, and work — instead of only chasing money and work. He asks students to rate each area 1–10 for themselves and discusses each one (health, hobbies, friends, spirituality, finance) with several students sharing personal reflections.

2. Reframing AI-related job fears as opportunities

A student expresses worry about layoffs caused by AI. The instructor's advice (delivered as a "charged"/Claude-style answer) lists concrete reframes:

- Old thinking: "AI is taking jobs." New thinking: "AI creates the biggest opportunity for people who learn faster than others." One employee + AI can do the work of five people, so companies keep people who use AI effectively, not everyone.
- Opportunity 1: Become 5x productive by using AI for coding, documentation, and testing.
- Opportunity 2: Build a second income (courses, YouTube, consulting, freelancing) — the very question "what if my salary stops?" is a gift because it pushes you to build alternatives.
- Opportunity 3: Learn skills that used to be expensive (video editing, coding, writing) — AI now lets one person do the work of a small team.
- Overall lesson: instead of asking "will my job survive," ask "what opportunity is AI giving me that didn't exist 5 years ago."

3. Real example: debugging a broken n8n workflow using only Claude

The instructor describes building an n8n automation to capture student registration confirmation emails (name, mobile number, course). The workflow initially failed to capture course details.

- He pasted the problem to Claude, which suggested trying a Gmail trigger node.
- When that didn't work, Claude suggested switching to an **HTTP Request node** instead of the Gmail trigger.
- Claude asked him to execute nodes and share **browser console logs** for debugging.
- The cycle repeated many times: take a screenshot → paste to Claude → get a fix suggestion → try it → if it fails, screenshot again → repeat.
- After roughly 3–3.5 hours of this copy-paste cycle, the workflow finally worked. The instructor emphasizes he never read or understood the generated JavaScript code — his only job was to relay errors and paste solutions.
- He shares a second example: connecting an external Sony camera to Zoom by screenshotting driver installation prompts and asking Claude what to click next, solved in about 20 minutes without technical knowledge.
- Takeaway: with full confidence in Claude/ChatGPT, you can attempt tasks outside your expertise (e.g., implementing SSO with Keycloak) by screenshotting errors and logs and following AI's step-by-step fixes.
- A discussion follows about developers feeling guilty for relying on AI-generated code — the instructor's answer: using AI tools makes you more productive, not "less skilled" (analogy: taking a bike/train instead of walking doesn't make you "stupid," it makes you smart). AI-generated code from tools like GitHub Copilot is often more optimized than manually written code, and many companies now scan/refactor code through Copilot regardless.

4. The CCOR prompt framework

Introduced as a simple, repeatable way to write effective prompts, especially for agentic/coworker-style tasks (building on the earlier RTC/RTCO framework used for general chat):

- **C – Context**: Give background information (e.g., "You are helping Durgasoft promote a new Python course").
- **C – Command**: Clearly state the task ("Create a marketing email for the course").
- **O – Output format**: Specify how the result should look (e.g., "Provide subject line, email body, and call-to-action separately").
- **R – Rules/Restrictions**: State constraints (e.g., "Use simple Indian English, keep it under 200 words, include a contact number").

Example prompt shown in the session for a full workflow design:

Context: You are the operations manager of DASoft. The institute offers Python, Java, DevOps, Data Science, AWS and AI courses. Student inquiries come through email, phone, website forms, and WhatsApp.

Command: Design a complete workflow to convert inquiries into enrollments.

Output format: Inquiry collection, follow-up process, demo session, fee discussion, enrollment process, post-enrollment activities.

Rules: Use a step-by-step process, include automation opportunities, focus on increasing enrollments.

The framework's field labels (Context/Command/Output/Rules) aren't rigid — what matters is that all four types of information are present so Claude has no ambiguity.

Key takeaways

- Balance your life across health, family, friends, hobbies, spirituality, money, and work — money and work alone are not "success."
- Reframe AI-driven disruption as opportunity: become more productive, build a second income, and learn newly-affordable skills.
- You don't need to understand the code Claude generates to fix a broken automation — screenshots, console logs, and iterative "not working, try again" prompts are a legitimate debugging workflow.
- Using AI tools to code faster makes you more productive, not less skilled — don't feel guilty about it.
- Use the CCOR framework (Context, Command, Output format, Rules) to structure prompts, especially for multi-step or agentic tasks.

Demo 17: Understanding the claude.md File (Project-Specific Instructions for Claude)

This session is a deep dive into the `claude.md` file — a project-specific configuration/instructions file used with Claude Cowork — explained through analogies (legal codes, web.xml config files, exam syllabi) and demonstrated live with a student-enrollment Excel analysis example.

What you'll learn

- What a `claude.md` file is and why it exists
- Why it must be named `CLAUDE.md` (capital CLAUDE, lowercase `.md`)
- That it is project-specific, not global — each project folder can have its own `claude.md`
- How Claude's memory and project context can accidentally leak information between projects, and how to control that
- A live before/after demonstration showing generic vs. highly targeted AI output based on whether a `claude.md` file exists

Step-by-step notes

1. What is a claude.md file?

- `MD` stands for **Markdown**, a plain-text formatting file extension.
- A `claude.md` file is a markdown file containing **instructions and rules for Claude to follow within a specific project**.
- Analogy: like a lawyer/judge referring to specific legal sections (Indian Penal Code, Constitution) for rules, or like a `web.xml` configuration file that a Tomcat web server reads for app-specific settings, or like an exam syllabus that defines what a student must study — `claude.md` defines what Claude must follow for a given project.
- Naming convention: recommended to use **CLAUDE** in uppercase and **.md** in lowercase (`CLAUDE.md`). Other casings may or may not be recognized depending on the model, so stick to the recommended form.

2. Why is it project-specific, not global?

- Every project folder in Claude Cowork can have its own separate `claude.md` file.
- Example: if you run one business for software training (DurgaSoft) and another for real estate, each needs completely different instructions — the software business talks about courses and students, the real estate business talks about plots, buyers, and pricing per square yard. A single global file wouldn't fit both.
- This is different from **global/account-level settings**, which apply everywhere, versus `claude.md`, which is scoped to one project folder only.
- Once written, a `claude.md` file:
- Acts as a **guide** telling Claude what to do and how to behave in that project.

- Helps Claude understand your company/context and workflow.
- Makes Claude give **consistent answers every time**.
- **Reduces the need to repeat instructions** in every single prompt — you only write the instructions once, and Claude applies them automatically every time you work in that project.

3. Live demo: Analyzing student enrollment data with and without claude.md

122. The instructor has an Excel file (`student_enrollments.xlsx`) with about 100 rows of student data, inside a Claude Cowork project folder.
123. First attempt (no `claude.md` file yet): he asks Claude to "Analyze student enrollments Excel file and provide recommendations." Because there's no claude.md, Claude cannot know anything about "DurgaSoft" specifically — but interestingly, the report that came back still referenced "DurgaSoft," which confused the class.
124. Investigation revealed the leakage was coming from Claude's **memory settings** (a separate feature that remembers info across chats) and from the **project folder name itself** ("DurgaSoft" was in the folder path), which Claude picked up and assumed was relevant, even incorrectly labeling a document with the organization's name without being told to. When asked directly, Claude admitted: "I inferred 'DurgaSoft' from your project — I should not have assumed that without asking you first."
125. Lesson: **memory settings can carry over context you didn't intend**, and Claude may guess organization names from folder names. The instructor recommends explicitly telling Claude "don't assume any names" in prompts if this is a concern, and shows how to disable/pause memory and reset it from Settings → Profile → Capabilities → Memory.
126. Second attempt: creating a **new, clean project folder** and analyzing the same Excel file without any assumptions — this time, with memory paused, the analysis came back generic (no incorrect "DurgaSoft" references), confirming the source of the earlier leakage.
127. Third attempt — now with a `claude.md` file added to the project folder containing:

DASoft business context. You are working for DASoft. Main goal: increase enrollments and revenue. Use simple Indian English. Focus on revenue-generating opportunities. Compare courses based on enrollments. Highlight courses needing promotion. Suggest WhatsApp, YouTube and email campaigns. Give management-level recommendations.

Asking Claude to analyze the same Excel file now produced a **much more specific, business-relevant report**: it flagged duplicate enrollment records, recommended boosting mobile app development enrollments, compared course-wise revenue, flagged the underperforming/top-performing courses, suggested a monthly trend analysis, recommended fee increases, WhatsApp QR-code campaigns, and top-priority actions — all clearly shaped by the rules defined in `claude.md`.

4. Real-world use cases discussed by the class

- Insurance: uploading vehicle damage photos/videos so Claude can help assess claims, and even check whether photos/videos were tampered with or geolocation-verified.

- Healthcare: uploading medication lists or ECG reports so Claude can flag unusual prescriptions or confirm results are within normal range (used personally by the instructor and a student for hospital visits).
- Personal finance: uploading bank/credit card statements and asking Claude to review them for issues.
- Business fraud detection: giving Claude a full sales dataset revealed that one student had "purchased" 21 courses under one login — Claude flagged this as a business risk (possible account sharing across multiple people), something a human would be unlikely to notice by eyeballing an Excel sheet.
- Nutrition tracking: logging a week of meals and asking Claude for dietary suggestions.
- Kids' education: uploading exam papers/report cards over time for progress tracking.

Key takeaways

- `claude.md` = a project-specific instruction file that tells Claude the rules, tone, goals, and context for a particular project, so you don't have to repeat them every time.
- Use the exact filename `CLAUDE.md` (capital CLAUDE, lowercase md).
- Each project should have its own `claude.md` — instructions for one business/domain should not bleed into another.
- Claude's memory settings and project folder names can leak unintended context between chats — be aware of this and reset/pause memory when starting a genuinely new analysis.
- Defining clear rules in `claude.md` turns generic AI output into highly specific, actionable, business-relevant output.
- Real-world applications go far beyond coding: insurance claims, healthcare record review, personal finance, fraud/anomaly detection in business data, and more.

Demo 18: Revision — Claude Chat, Cowork, Artifacts, Skills, Connectors, Memory, CCOR, and claude.md (Quiz Format)

This is a structured revision session covering everything taught over the previous ~10 sessions about Claude's features, delivered partly as explanation and partly as a live 20-question multiple-choice quiz generated by Claude itself.

What you'll learn

- A consolidated recap of Claude's three "flavors": Claude Chat, Claude Cowork, and Claude Code
- When to use Claude vs. ChatGPT for coding/technical tasks vs. general content tasks
- A full recap of: Memory settings, Artifacts, Skills, Connectors, Chrome extension, the CCOR prompt format, and claude.md
- How to generate a self-quizzing MCQ revision session using Claude itself

Step-by-step notes

1. Recap: Claude's interfaces and models

- Claude offers three main flavors: **Claude Code**, **Claude Cowork**, and **Claude Chat**.
- Two ways to access Claude: **web** (chat only) and **desktop app** (chat, plus coding capability).
- Claude has multiple models, roughly tiered like car models: a **basic/economy model (Haiku)** for low-effort daily tasks, a **regular/mid-tier model (Sonnet)** for typical work, and a **premium/high-end model (Opus)** for tasks requiring deep analysis.
- The instructor shares that for coding, JavaScript generation, and complex analysis (e.g., debugging n8n workflows), he now instinctively reaches for **Claude** over ChatGPT; for general content generation, notes, quizzes, and simple text, he still often uses ChatGPT — implying each tool has its own strength area.

2. Recap: Memory settings

- In Settings → Profile → Capabilities → Memory, there are two related features:

128. **"Search and reference chats"** — lets Claude search your past chat history for relevant details while answering a new question.
129. **"Generate memory from chat history"** — Claude builds a running summary of what it has learned about you (e.g., your job, communication style, ongoing projects) and applies that automatically to future chats.

- This summary is regenerated nightly and does **not** apply to Projects, which maintain their own separate memory/context.
- You can view, edit, and clear ("reset") this memory manually.
- By default (memory disabled), each new chat is independent and does not carry over context from previous chats.

- You can also explicitly say "please memorize this about me" to add something to memory directly.

3. Recap: Artifacts

- An artifact is a **reusable output displayed in a separate workspace** (visible on the right side of the chat window) — e.g., an interest calculator, a loan comparison calculator, HTML pages, code, documents, flowcharts, or dashboards.
- Artifacts can be downloaded, published, and shared with others.
- You don't have to explicitly say "create an artifact" — Claude usually recognizes reusable content and creates one automatically. But if it doesn't, you can explicitly ask: "create this as an artifact" to force it.

4. Recap: Skills

- A **skill** is a reusable, predefined set of instructions/capabilities that extends what Claude can do (e.g., the built-in Canva design skill, Skill Creator skill, Web Artifact Builder, MCP Builder, Theme Factory, Brand Guidelines).
- You can create your own custom skill via Settings → Customize → Skills → "+" → "Create" → "Write skill instructions" (give it a name, description, and instructions).
- Skills are defined **once** but can be **used unlimited times**, just by naming the skill in a prompt (or using `/` to see a list of all available skills).
- Skills apply across **all projects** (unlike `claude.md`, which is scoped to one project only).
- Skills can be downloaded and shared with others, and others can upload a shared skill file into their own Claude account to use it.
- Example given: Claude has no built-in text-to-speech capability, but by adding a custom "Gemini TTS" skill, Claude can now hand off text to Gemini to generate speech — extending Claude's abilities via skills (sometimes combined with connectors/MCP for the actual external communication).

5. Recap: Connectors

- **Connectors** let Claude connect to external apps and data sources with your permission — e.g., Gmail, Google Drive, Google Calendar, Slack, Canva, Notion, Figma, Strava, Zoom.
- Example: connect Gmail to summarize the last week's emails, then use the Google Drive connector to save that summary as a PDF in Drive.
- Managed via Settings → Customize → Connectors → "+ Browse connectors" (custom connectors also exist but weren't covered in depth).

6. Recap: Claude for Chrome (browser extension)

- Lets Claude read and understand the content of the web page you're currently viewing, and act on it — e.g., summarizing a page, or finding the best flight from Hyderabad to Bangalore by reading search results directly in the browser.

7. Recap: CCOR prompt format

- **C**ontext, **C**ommand (or Challenge), **O**utput format (or Outcome), **R**ules (or Requirements/Restrictions) — a simple structure for writing clear prompts, especially for project/agentic work. Equivalent in spirit to the earlier RTC(O) framework used for general chat prompts.

8. Recap: claude.md and Projects

- `claude.md` acts as a **project handbook**: coding standards, folder structure, naming conventions, business rules, and other must-follow instructions, defined once and followed consistently by Claude in that project.
- A **Project** in Claude is an isolated, reusable workspace that groups related chats, files, instructions, and knowledge together (e.g., a separate project for marketing, a separate one for real estate, a separate one for course creation). You only need to add files/instructions once per project, and they stay available inside it.

9. Live self-quiz exercise

The instructor asks Claude to generate a 15-question multiple-choice quiz covering: Claude Chat, Cowork, Artifacts, Skills, Connectors, Memory settings, CCOR prompt format, and claude.md — one question at a time, waiting for the student's answer, validating it, giving a learning conclusion, then moving to the next question. The class answers together. Key correct answers reinforced during the quiz:

- Claude Chat = a conversation workspace for learning, writing, coding, analysis, and problem-solving.
- Claude Cowork = a "digital teammate/colleague" that can work with files, perform multi-step tasks, and produce real deliverables (not just answer questions).
- Artifact = a separate, editable workspace for documents, code, diagrams, dashboards, and reports.
- Skill = a reusable capability/workflow/calculator/dashboard defined once, used many times.
- Connector = a bridge that lets Claude access and work with data from connected apps (with permission).
- Memory settings = lets Claude remember your preferences, teaching/writing style, and recurring instructions across chats (disabled by default; must be enabled).
- Chrome extension = lets Claude understand and work with the web page you're currently viewing.
- CCOR's "C" = Context; second "C" = Command/Challenge; "O" = Outcome/Output format; "R" = Rules/Requirements.
- claude.md = project-specific instructions file, a "project handbook."
- Project = a dedicated, organized workspace for a specific topic (chats, files, instructions, knowledge together).
- Best combination for automatically analyzing student enrollment emails: **Connector (Gmail) + Claude Chat** — the connector supplies the raw data, Claude Chat provides the analysis/intelligence.

Key takeaways

- Claude has three flavors (Chat, Cowork, Code) and three model tiers (Haiku = basic/fast, Sonnet = regular, Opus = premium/deep analysis).
- Memory settings, Artifacts, Skills, Connectors, and the Chrome extension are all distinct features that each extend Claude's usefulness in a different way — know which one solves which problem.
- Skills are reusable across all projects; claude.md is scoped to a single project.
- CCOR (Context, Command, Output, Rules) is the recommended structure for clear, unambiguous prompts.
- Quizzing yourself using Claude-generated MCQs is an effective way to revise and retain concepts.

Demo 19: Web Scraping with Apify — Connecting Claude to Real-Time Web Data for Lead Generation

This session introduces **Apify**, a web-scraping tool/platform, and walks through connecting it to Claude via a connector (MCP), then demonstrates several real business use cases: competitor research, job-market research, and lead generation for a training institute.

What you'll learn

- What Apify is and what it's used for (scraping live data from websites, social media, maps, and e-commerce sites)
- How to create an Apify account, generate an API token, and connect it to Claude Desktop as a connector
- How to use Apify's "actors" (pre-built scraping tools) through natural-language prompts in Claude Cowork
- Multiple real-world business use cases: competitor analysis, job-market research, student lead generation, finding businesses without websites, and finding YouTube channels for collaboration

Step-by-step notes

1. What is Apify?

- Apify (apify.com) is a tool/platform that **collects real-time data from websites** — social media, Google Maps, e-commerce sites, TikTok, Instagram, Facebook, YouTube, and more.
- Apify's pre-built scraping tools are called **"actors"** — there are 36,000+ actors available, each specialized for a specific site/type of data (e.g., Google Maps Scraper, Instagram Scraper, TikTok Scraper, Website Content Crawler).
- Apify supports API access, scheduling, integrations, and **MCP (Model Context Protocol) for AI agents**, which is how it connects to Claude.
- Once Apify collects raw data, Claude can analyze it, turn it into reports, summaries, tables, emails, or business decisions.

2. Setting up an Apify account and connecting it to Claude

130. Go to **apify.com** and sign up/log in with your Gmail account.
131. Once logged in, go to **console.apify.com** (the Apify control console).
132. Navigate to **Settings → API and integrations**.
133. Click **"Create a new token"**. You can optionally set an expiration date or limit token permissions — for testing, the instructor left both unset and just clicked **Create**.
134. **Copy and save the API token immediately** — like most API keys, it's shown only once.
135. In **Claude Desktop**, go to **Settings → Customize → Connectors → "+" → Browse connectors**, search for **"Apify"**, and click **Install**.

136. When prompted, **paste your Apify API token** and save. (Note: installation can take a little time, and sometimes you may need to retry or re-enable the connector if it doesn't attach to the MCP server right away.)
137. Once installed and enabled, the Apify connector shows as active/running in Claude's connector settings, confirming it's ready to use.

3. Using Apify inside Claude Cowork — example prompts and outcomes

The instructor creates a new Cowork project and runs several example prompts, each combining "Use Apify [specific scraper] to find X" with a request for a specific data table:

- **Competitor research**: *"Use Apify Google Maps Scraper to find the top 50 (or 20) software training institutes in Hyderabad. Collect institute name, phone number, website, Google rating, number of reviews, address. Create a table and identify top 10 competitors of DurgaSoft."* — This produced a full Excel-style table of local competitors within about 2 minutes, complete with contact details, ratings, and an analysis identifying the top 2–3 most "formidable" competitors based on review volume, ratings, and overlapping course focus (e.g., data science/AI).
- **Job-market research**: *"Use Apify to find the companies currently hiring Python developers in Hyderabad. Collect the company name, careers page, HR email (if publicly available), company website, and location. Create a clean Excel report and rank companies based on hiring activity."* — Returned companies like TCS, Infosys, Wipro, HCL, Capgemini, Amazon, and Microsoft with their careers pages and hiring activity scores, ranked from highest to lowest hiring activity.
- **Student lead generation**: *"Use Apify Google Maps Scraper to find engineering colleges in Hyderabad. Collect college name, website, phone number, email address, and location. Create a lead list for promoting DurgaSoft online courses."* — Produced a table of colleges (JNTU, Keshav Memorial, Narayanamma, TKR, Deccan, etc.) with contact info that could immediately be used to send outreach emails or WhatsApp messages.
- **Finding businesses without websites** (potential clients for web/digital-marketing services): *"Use Apify Google Maps Scraper to find businesses in Hyderabad that do not have a website listed. Collect business name, phone number, Google rating, address, category. Create a report of businesses that may need website development or digital marketing services."* — Businesses without a website listed on their Google Maps profile are flagged as sales leads for web-development or digital-marketing outreach.
- **YouTube collaboration research**: *"Use Apify to find YouTube channels related to Python, AI, data science, and career guidance. Collect the channel name, subscriber count, channel URL, topic, and contact information. Create a list of the best channels for collaboration and promotion."* — Returned a ranked list of channels (e.g., Programming with Mosh) with subscriber counts and collaboration priority.
- **Fun/business-idea example**: *"Use Apify Google Maps to find the top 100 tea shops in Hyderabad. Collect shop name, rating, reviews, address, phone number. Analyze the data and tell me which area has the most tea shops, which shop has the highest rating, what rating is most common, and if I want to open a new tea shop, which area has less competition."* — Demonstrates that the same

scraping + analysis pattern works for any small-business market research question, not just tech-related ones.

4. Important caveats discussed

- Apify can only access **publicly available data**. If a page/profile requires login (e.g., private LinkedIn profiles, gated real-estate listing sites like Magic Bricks or 99acres that sell their data), Apify cannot scrape it.
- API usage has limits, and Apify has paid pricing plans for heavier use.
- After gathering leads (emails, phone numbers), Claude/Cowork can go further — draft and send outreach emails, trigger WhatsApp messages, or feed the leads into an **n8n workflow** that automatically emails every contact and alerts an admin whenever someone replies, effectively automating end-to-end lead generation and follow-up.

Key takeaways

- Apify is a web-scraping platform with thousands of ready-made "actors" for scraping Google Maps, social media, e-commerce, and more — connect it to Claude Desktop via Connectors, using an Apify API token.
- Once connected, you can ask Claude in plain English to scrape targeted data (competitors, job postings, leads, YouTube channels, etc.) and get a clean, structured Excel-style report in minutes — tasks that would otherwise take hours or days manually.
- Apify only accesses publicly available data — login-gated content and private profiles are off-limits.
- Combining a connector (data collection) with Claude Chat/Cowork (analysis and reporting) turns raw web data into actionable business intelligence — market research, lead lists, and competitor analysis can all be done from a chat prompt.
- This same scraping + analysis + outreach pattern can be extended with n8n to build fully automated lead-generation pipelines.

Demo 20: Using Claude for Stock Investment Decision Support and "Simpler Way" Thinking

This session shows a detailed custom prompt for personal stock-investment decision support (built with a strict scoring model and HTML artifact output), followed by a broader discussion on using AI to simplify everyday decisions and activities.

What you'll learn

- A full worked example of a "Personal Stock Investment Decision Assistant" prompt for Claude, including required structure, safety rules, and sourcing requirements
- How changing personal inputs (risk level, investment horizon, portfolio concentration) changes Claude's investment recommendation
- The "simpler way / less effort" questioning technique for simplifying any daily activity or decision
- Practical class discussion on using AI (ChatGPT/Claude) for stock research, chart analysis, and staying updated on financial news

Step-by-step notes

1. The stock investment decision-support prompt

The instructor shares a long, carefully structured prompt (for personal/educational use only, since he is not SEBI-certified and the tool is not financial advice) with these key elements:

- **Purpose statement**: "You are my personal stock investment decision support assistant. Your job is to help me decide whether I can invest in a particular Indian stock or not, based on my personal situation, risk level, investment horizon, portfolio allocation, and the company's fundamentals, valuation, financial health, growth, quality, ownership pattern, and red flags."
- **Required final output**: one of **Invest**, **Do not invest**, **Wait and watch**, or **Need more data** — always via a scoring model, never based on "gut feeling."
- **Strict safety rules**: never promise guaranteed profit, never give a target price/stop-loss/intraday view, never say "this will become a multibagger" — this is a decision-support checklist, not trading advice.
- **Sourcing rules**: every financial metric must show its source (e.g., "ROE — source: annual report"). If data is unavailable, Claude must say "data unavailable" rather than fabricate or estimate. Use at least two sources where possible: NSE India, BSE India, company annual reports, investor presentations, earnings call transcripts, **screener.in**, moneycontrol, tickertape, Tijori Finance. If sources disagree, show both values and flag "source mismatch — verify manually."
- **Output format requirement**: the final output must be a single self-contained **raw HTML artifact** (no markdown, no code blocks) with **8 tabs**, defaulting to the "Final Decision" tab.
- **Step 1 — required intake questions** Claude must ask before starting analysis:

138. Which stock (company name or NSE/BSE ticker)?

139. Investment horizon (3, 5, 10 years, or custom)?
140. Risk level (low/medium/high)?
141. Investment amount planned?
142. Is this your first stock investment (yes/no)?
143. Do you have an emergency fund for 6 months (yes/no)?
144. Current portfolio concentration — what % of your total stock portfolio will this one stock represent after buying?
145. Investment style (long-term wealth creation, dividend income, learning purpose, or high-risk growth)?

- **Step 2 — silent research**: Claude collects and cross-checks company data (valuation, growth, financial health, ownership, business/management quality, peer comparison, news/red flags) **without showing the raw research process** to the user, then applies the scoring model to reach a final decision.

2. Live demo: analyzing TCS stock

- The instructor runs this prompt in a **new Claude Chat** (not Cowork, since it's pure analysis — no file operations needed), recommending **Opus 4.5** (or Sonnet) with **extended thinking** enabled for better-quality financial reasoning.
- He answers the 8 intake questions: stock = TCS, horizon = 5 years, risk = low, amount = a sample figure, first investment = yes, emergency fund = yes, portfolio concentration = 100% (since it's his first investment), style = long-term wealth creation.
- Claude researches and produces an artifact with tabs including: Snapshot (company name, BSE code, sector, current market price, 52-week high/low, market cap, face value, revenue, net profit, dividend, recent key news), Personal Fit, Valuation, Growth, Financial Health, Ownership, Peers, and Final Decision.
- With **low risk selected**, the final decision came back as **"Wait and watch"** (score in the 60–74 range) — largely because of recent negative news (TCS stock had fallen ~9% in a single day due to AI-disruption fears in the IT sector, despite otherwise strong financial health).
- The instructor then **changes only the risk level to "high"** and reruns — the score rises to 77/100 and the final decision changes to **"Invest."** This demonstrates that the same company can get a different recommendation purely based on the user's personal risk tolerance, not just the company's fundamentals.
- The class discusses real context around TCS: a 2.5-year price decline from ~4600 to ~2100, driven by AI disrupting the IT services business model and experienced staff attrition — illustrating that such prompts give a structured starting point, but real-world business/sector context still matters.
- A second live example is run for **HAL (Hindustan Aeronautics)** with different inputs (5-year horizon, medium risk, not first investment, 70% portfolio concentration, long-term wealth creation) — resulting in a "Wait and watch" outcome (score 69/100) with full company snapshot and news context.

3. The "simpler way, less effort" thinking technique

- The instructor introduces a mental habit inspired by Toyota's development process, which reportedly always asks two questions for any activity: **"How can I do this with less effort?"** and **"How can I do this in an easier way?"**
- He connects this to Elon Musk's "first principles thinking": don't accept a process as fixed just because "that's how it's done" — only physical laws are truly fixed; everything else can potentially be simplified or changed.
- Applied example: a student's 25 km one-way bike commute (3 hours/day, ~60 hours/month) is analyzed by AI, which suggests: work from home 2–3 days/week if possible, start earlier (6:30 am) or later (10:30 am) to avoid peak traffic, use commute time for audiobooks or learning, and consider whether the real underlying problem is "the office depends on me too much" rather than the commute itself.
- General advice: for any repetitive daily activity or decision, ask Claude/ChatGPT to suggest a simpler, lower-effort way to do it, providing your own personal context (job, role, constraints) for a tailored answer.

4. Class discussion on AI-assisted stock research

- A student shares that using ChatGPT for chart analysis (with saved filters like 50-day/200-day moving averages) cut his weekly stock research time from 4–5 hours down to 30–40 minutes, and that uploading long PDF reports and asking for bullet-point summaries saves significant reading time.
- The instructor recommends trying **Claude** specifically for deeper technical/analytical work (like n8n workflows and JavaScript generation) since he finds it stronger there, while ChatGPT may still be preferred for quick content tasks.
- The class discusses that AI cannot predict unpredictable market-moving events (e.g., a single tweet from a political figure disrupting an index), so AI-based analysis should be treated as a decision-support aid, not a guarantee.
- Suggested habit: run this stock-analysis prompt regularly (e.g., every morning) for stocks you're tracking, and iteratively refine your prompt over time — asking Claude "what did you miss, and how should the prompt be updated?" after each real outcome — so accuracy and relevance improve over repeated use.

Key takeaways

- A well-structured prompt (with mandatory intake questions, a scoring model, required sourcing, and a fixed output format) can turn Claude into a genuinely useful personal stock-research assistant — but always frame it as decision support, not guaranteed financial advice.
- The same company/stock can get different recommendations depending purely on your personal inputs (risk tolerance, horizon, portfolio concentration) — the tool is personalized, not a universal "buy/sell" signal.

- For any repetitive or time-consuming daily activity, ask "how can I do this with less effort / in an easier way" — this simple two-question habit (borrowed from Toyota and Elon Musk's first-principles thinking) can uncover real simplifications.
- AI-assisted research can massively cut down manual analysis time (one student reported going from 4–5 hours/week to 30–40 minutes), but cannot predict unpredictable market shocks — always combine AI output with real-world judgment.

Demo 21: Scheduling Automated Tasks with Claude Cowork (the "Scheduler")

This session teaches how to make Claude's Cowork (desktop) feature run tasks automatically and repeatedly — similar to cron jobs — either by describing the schedule directly inside your prompt, or by using Cowork's built-in Scheduled Tasks tab.

What you'll learn

- What "scheduling" means inside Claude Cowork and why it's useful
- How to write a prompt that tells Claude to repeat a task on its own (e.g., every 3–5 minutes)
- How Claude organizes its own output into folders and files automatically
- How to use the separate "Scheduled" tab in Cowork to set up a task without writing timing instructions in the prompt
- The difference between manual, hourly, daily, weekday, and weekly schedule options
- Important limitations of Cowork scheduling (desktop-only, computer must stay awake, Pro/paid feature)
- The real-world idea of "cron jobs" and how this concept maps to automated content/news systems

Step-by-step notes

146. **The example goal**: build a "Cricket and Movie News Update Scheduler" — an agent that automatically fetches cricket news and movie news, alternating between the two, and saves each update into organized folders and files.

147. **Writing the prompt (Option 1: put the schedule inside the prompt itself)**

- The instructor drafted a prompt (with help from an AI chat assistant) describing:
- Role: "You are my automated news research agent."
- Frequency: "Run every 5 minutes" (later changed to every 3 minutes for faster testing), alternating: first run = cricket news, second run = movie news, third run = cricket news, and so on.
- Content rules: cover Indian cricket/IPL/international match updates and Telugu/Hindi/Tamil movie & OTT news; avoid fake news, rumors, gossip, and repeated news.
- Output format for each update: date & time, category, news title, short summary, why the news is important, who benefits, and the source.
- File/folder rules: create one folder named "news updates" containing two subfolders — "cricket news" and "movie news." Save each execution's result as a separate new file. Never mix cricket and movie news in the same file. Never repeat the same news. Keep language simple. Always record the exact execution time.

148. **Running it in Cowork**

- Open Cowork, start a "New Task," and give Claude access to a folder (e.g., a folder named "Claude Scheduler").

- Paste the prompt in. Claude begins working: it creates the "news" folder, then the "cricket news" and "movie news" subfolders inside it, and starts writing files automatically — no manual folder/file creation needed.
- Every 3 minutes, Claude alternates: it writes a cricket news file, then next cycle a movie news file, and so on (visible in the Scheduled Tasks list with timestamps like :00, :06, :12, :18…).
- Opening the generated files shows realistic structured news entries (date/time, title, summary, importance, beneficiaries, source, and an "action point").
- To stop the recurring runs, you pause the task from the Cowork interface.

149. **Important observation**: even though the instructor never touched the dedicated "Scheduled" tab, Claude automatically registered the recurring prompt as a scheduled task — because the instruction to "run every X minutes" was written directly in the prompt.

150. **Writing the prompt (Option 2: use the Scheduled tab, no timing in the prompt)**

- Second example: an "AI Tools Update" agent that checks for new AI tools/features/products, without specifying a run interval in the prompt.
- Go to the **Scheduled** tab in Cowork → click **"New Task"** → choose **"Set up manually"** (as opposed to "Create with Claude," which builds a scheduling prompt for you).
- Fill in: Task name (e.g., "AI news update"), Description ("AI news daily updates"), the actual prompt text, which project/folder to work in (grant folder access), and optionally a specific model.
- Under **Frequency**, choose from: **Manual** (only runs when you click "Run now"), **Hourly**, **Daily** (pick a time, e.g., 6:00 a.m.), **Weekday** (Monday–Friday only), or **Weekly** (pick a specific day and time, e.g., every Monday 6 a.m. or every Tuesday 7 a.m.).
- Save the task. You can test it immediately using "Run now," even if it's scheduled for a future time — this doesn't change the actual daily schedule.

151. **Result of the AI tools example**: Claude organized results into multiple categorized files/folders automatically — e.g., "AI news," "AI tool feature updates," "business useful tools," "new AI tools," and "student useful tools" (things like resume builders, interview practice tools, job-matching tools).

Key takeaways

- Cowork scheduling works two ways: (1) describe the repeat interval directly in your prompt ("run every 3 minutes…"), or (2) use the dedicated **Scheduled** tab and pick Manual/Hourly/Daily/Weekday/Weekly.
- Claude handles all the file and folder organization on its own once instructed clearly — you don't need to create folders manually.
- This scheduling concept is compared to **cron jobs** (a Linux/AWS term for tasks that run automatically on a timer).
- Cowork scheduling currently only works through the **desktop app**, not the web version, and your computer must stay awake (not shut down or asleep) for scheduled tasks to run.

- Cowork itself is a paid/Pro feature — there is no separate extra charge just for using the scheduler, but Cowork access itself requires a paid plan.
- Practical use cases discussed: automated news aggregation for a website, competitor-batch monitoring for a training institute, daily AI-tools digests, and generally any repetitive "check something and report/save it" task.
- Because Claude has both browser access (to search the web) and system access (to create files/folders), it can fully automate research-and-report workflows end to end.

Demo 22: Claude Dispatch — Controlling Your Desktop Claude App from Your Phone

This session introduces "Dispatch," a (beta) feature that lets you send commands from the Claude mobile app and have them executed on your desktop/laptop Claude app — essentially a remote control for Cowork.

What you'll learn

- What the Dispatch feature is and why it exists
- The relationship between the mobile app and the desktop app when using Dispatch
- How to pair the mobile app with the desktop app and send a test command
- Practical scenarios where Dispatch is useful
- Key requirements and limitations (same account, desktop must be awake, Pro-only feature)

Step-by-step notes

152. **What is Dispatch?** It lets you give a task from your mobile phone, and Claude executes that task on your desktop or laptop app (not on the phone itself). The instructor describes it as "a remote control for the Cloud (Claude) Cowork" — similar to using a TV remote or an AC remote, except here your phone controls your desktop Claude app.
153. **Example use case**: While outside the office, you could tell Claude (from your phone), "Open my laptop files, find today's student enrollment report, summarize it, and prepare a short WhatsApp message." The command originates on the phone, but the actual work (opening files, reading them) happens on the laptop.
154. **Why it's useful**:

- Delegate computer-only tasks (finding files, preparing reports, checking documents, summarizing sheets, drafting content) while you're away from your desk.
- Continue the same work across devices — start a request on your phone, check/finish the result later on the desktop.
- Useful when traveling, in a car, outside the office, or in a meeting — situations where you only have your phone but need something done on your computer (e.g., unpublishing a workflow, checking a file) without calling someone else to do it manually.

155. **Requirements to set it up**:

- You must install the **Claude mobile app** on your phone.
- The mobile app and the desktop app **must use the same account/email** — this is mandatory, since Dispatch links the two based on matching login.
- Your desktop/laptop must be **awake** (screen can be locked, but the machine must not be shut down or asleep) for the desktop app to receive and execute the dispatched task.

- Dispatch currently appeared to be a **Pro-only** feature (one participant without a Pro subscription didn't see the option).

156. **Live demo of pairing and testing**:

- On the desktop Cowork settings, there's a **Dispatch** section/tab with options like "keep awake," "mobile notifications," "browser actions," "computer/code permissions," described as: "Dispatch to cloud and check from anywhere."
- On the mobile app, open the three-dot menu (near Chat/Projects/Artifacts/Code) and tap **Dispatch**.
- Choose **"Pair with the desktop"** and type a message (e.g., "hello") to trigger pairing/enabling.
- Once paired, any message typed on the mobile Dispatch screen (e.g., "This is a test message from my mobile," or "What is today's cricket match result?") appears and gets processed on the connected desktop app — confirming the mobile-to-desktop link works.

157. **Limitation noted**: Dispatch only controls actions that Claude itself is capable of doing (i.e., whatever Cowork/Claude can already do on the desktop) — it does not give you separate control over arbitrary third-party apps directly; it works through what Claude is authorized to access on that desktop (files, connected apps, browser, etc.).

Note: A large portion of this session was a live, informal walkthrough by a student showing a personal website/admin panel he built using Claude and a Google Sheets backend. Since it was mostly personal project chat rather than core instructional content, it is not detailed here — the instructor's guidance was simply to finish the project and prepare a focused case-study demo for a future session.

Key takeaways

- **Dispatch = mobile remote control for your desktop Claude/Cowork app.** You type/ask from the phone; execution happens on the desktop.
- It requires: the Claude mobile app installed, the same account/email on both mobile and desktop, and the desktop machine to be awake (not shut down/asleep).
- Great for delegating "computer work" while away from your desk — no need to call someone else to operate your machine for you.
- It's currently a beta feature and appears to require a Pro subscription.
- A deeper, hands-on use-case walkthrough was promised for the following session.

Demo 23: Foundations Review — Semantic vs Keyword Search, and 10 Generative AI MCQs

This session is a spoken revision quiz covering earlier "core foundations of generative AI" material, framed as 10 multiple-choice questions with explanations. It opens with a real n8n workflow story illustrating why semantic search matters more than keyword search.

What you'll learn

- The practical difference between keyword search and semantic search, illustrated with a real automation bug
- Core generative AI vocabulary: generative AI, prompts, LLMs, training data, hallucination, tokens, fine-tuning, and RAG
- Why fine-tuning is costly while RAG is cheap and easy to update
- Why human review of AI output always matters

Step-by-step notes

158. **Real-world problem: keyword search vs. semantic search**

- The instructor built an n8n workflow (using only JavaScript code, no AI agent) that should: when a student inquires about "Java" courses, search durgasoftonline.com for matching batches and email the matched batch details to the student.
- Workflow structure: Student inquiry (trigger) → HTTP Request node (fetches all batch info from durgasoftonline.com) → a filtering step (finds Java-related batches) → send email to student.
- **The bug**: a batch called "Spring Boot and Microservices" is genuinely Java-related, but because the filtering logic used plain keyword matching (looking for the literal word "Java"), it never matched and was excluded from the email — even though a human would recognize it as relevant.
- **Root cause**: JavaScript/code-based filtering can only do keyword search — matching literal text, not meaning. To match by meaning, you need an AI agent (e.g., ChatGPT/OpenAI) doing semantic search.
- **Key distinction**: Keyword search matches literal words; semantic search matches meaning. Inside transformers, the attention mechanism effectively performs meaning-based (semantic) comparisons between tokens, not literal keyword matching.

159. **MCQ Review (10 questions, answered live with the class)**

160. **What does generative AI mean?** — AI that creates new content (text, images, code, audio), not AI that just stores data or performs calculations. All major LLMs (ChatGPT, Gemini, etc.) are generative AI because they generate new output based on your request rather than copy-pasting existing content.

161. **Best example of generative AI?** — "ChatGPT writing an email draft" (not a calculator, not Google Maps, not an alarm clock). Generative AI is for creating content, not just displaying fixed information.
162. **What is a prompt?** — The question or instruction you give the AI. Better prompts lead to better AI output.
163. **Main purpose of an LLM (large language model)?** — To understand and generate human-like language. LLMs are the backbone of tools like ChatGPT because they work with natural language (NLP).
164. **What is training data?** — The data used to teach the AI model patterns of language and concepts. Data quality directly affects output quality — better training data means better output.
165. **What is AI hallucination?** — AI confidently giving a wrong or fake-looking answer. It happens because the model is predicting/guessing, not because it "knows" the truth. Always verify important AI answers — especially anything related to money, health, legal matters, or current/live information.
166. **Best way to reduce wrong answers from generative AI?** — Give clear context and verify important facts. This ties back to the earlier-taught **RTCC** prompt format (Role, Task, Constraints/Context, output format/Cconstraints) — the clearer and more complete your prompt, the clearer and more reliable the output.
167. **What is a token?** — A small piece of text — a word, part of a word, or punctuation — not necessarily a whole word. Example: "color" vs "colour" would become two entirely different tokens if word-based tokenization were used, breaking semantic matching; similarly, long compound words (like "internationalization") or spelling variants would create inconsistent tokens if tokenized only by whole words. That's why sub-word tokenization is used instead of whole-word tokenization. Longer prompts and longer answers use more tokens.
168. **Fine-tuning vs. RAG** —

- **Fine-tuning**: retraining an existing LLM further on your own domain-specific data (e.g., retraining a general model on pharmacy-industry data) so it performs better for that specific task/domain. It is expensive and difficult — every time your data changes, you must retrain, and internal token relationships/semantic distances shift.
- **RAG (Retrieval-Augmented Generation)**: you do NOT retrain the LLM. Instead, you tell it: "Don't answer from your own knowledge — refer to my provided documents/PDF (stored in a vector database) and answer from that." Example given: a cricket-rules PDF was vectorized and stored, and the LLM was instructed to answer only from that document (this was demonstrated earlier in an n8n workflow).
- **Why RAG is preferred in industry**: it's much cheaper and easier to update — just replace the old PDF/document with a new one, no retraining needed. This is also why many companies use RAG to let employees search internal/sensitive company knowledge bases without exposing that data publicly (avoiding the security risk of publishing sensitive data openly, as fine-tuning a public model would require exposing that data to the training process).

169. **Why is human review important when using generative AI?** — Because AI output can sometimes be wrong, biased, incomplete, or outdated (hallucination). AI is a powerful assistant, but final responsibility must remain with humans. A participant shared a real example: a locally-run Ollama model confidently said the wrong team won the 1983 Cricket World Cup, then contradicted itself when asked for the scorecard — a live illustration of hallucination and inconsistency, often tied to outdated or incomplete training data.

Key takeaways

- Code/keyword-based filtering can miss relevant results because it only matches literal words; use an AI agent for semantic (meaning-based) search when relevance matters more than exact wording.
- Generative AI = creates new content by learning patterns and predicting likely output, not by storing/copying data.
- A token is a small text unit (word, sub-word, or punctuation) — not always a full word — which enables more consistent, meaningful processing across spelling variants and long words.
- Hallucination = confidently wrong AI answers; always verify anything important (money, health, legal, current events).
- Fine-tuning retrains the model itself (expensive, hard to keep updated); RAG feeds the model your documents at answer-time (cheap, easy to update, and industry-preferred for private/sensitive data).
- Good prompting (clear context, constraints, and fact verification) reduces the risk of poor or wrong AI answers.

Demo 24: Foundations Review Part 2 — AI, ML, Deep Learning, and How GPT Actually Works

A continuation revision quiz covering the fundamentals of AI/ML/deep learning and a deeper technical walkthrough of how GPT-style models process text internally — tokens, embeddings, attention, context windows, and positional encoding.

What you'll learn

- Clear plain-English definitions of AI, machine learning, and deep learning, and how they relate to each other
- What GPT stands for and the high-level flow of how it processes an input into an output
- Why GPT gives different answers to the same question sometimes
- The role of embeddings, attention, context windows, tokenization, and positional encoding
- Why token limits matter in practice

Step-by-step notes

170. **What is Artificial Intelligence (AI)?** — Machines behaving a little bit like humans by using data, logic, and learning — but they never actually become human. "AI is not magic. AI is data + logic + learning."
171. **What is Machine Learning (ML)?** — Teaching a machine to learn from data instead of writing fixed rules for every situation. ML learns from patterns in past data rather than relying only on hardcoded rules.
172. **What is Deep Learning?** — A type of machine learning where the computer learns using many layers, inspired by the structure of the human brain (like neurons). Deep learning is a sub-part of machine learning, learning "layer by layer."
173. **What is Generative AI?** — AI that creates new content (text, image, audio, video, code) by learning patterns and predicting the next best piece of content, rather than just storing/retrieving data.
174. **What is a Language Model?** — A model that predicts the next word (more precisely, the next *token*) based on language patterns in a sentence — essentially a "next-word/token prediction engine."
175. **What does LLM mean?** — Large Language Model. It's called "large" because it's trained on huge amounts of data and has a very large number of parameters — giving it strong language "learning power."
176. **Why does ChatGPT feel intelligent?** — Because it predicts the next word accurately using context and probability — not because it has human-like thinking or emotions. It "feels" smart because of accurate prediction, not genuine understanding.

177. **What does GPT stand for?** — Generative Pre-trained Transformer: **G**enerative (creates new output), **P**re-trained (already trained on huge data), **T**ransformer (uses the transformer/attention architecture).
178. **High-level GPT processing flow**: Input text → **tokenization** (input tokens) → **embeddings** (tokens converted to numeric vectors with meaning) → **transformer attention** (understands context/relationships between tokens) → output generated.
179. **Why does GPT give different answers to the same question sometimes?** — Because it's a probability-and-prediction model with some controlled randomness, not a fixed calculator — it doesn't return one hardcoded answer every time.
180. **Why are embeddings important?** — They convert text tokens into numerical representations that carry meaning, similarity, and relationships between words, so the model can process them mathematically. Embeddings are **multi-dimensional** (roughly 1,000–1,500+ dimensions in practice), not just 1D, 2D, or 3D — this large dimensionality is what allows rich meaning to be captured.
181. **What is the main role of attention in transformer models?** — To focus on important words and understand context — described as "the focusing power" and "the heart of the transformer."
182. **What is the context window?** — The limited, recent portion of the conversation that GPT can "see" while replying — like short-term memory, not permanent/unlimited memory of everything ever said.
183. **Why does hallucination happen in GPT?** — Because GPT can generate a confident-sounding answer based on learned patterns even when that answer is factually wrong — it's predicting plausible text, not verifying truth. Always verify important answers (facts, money, legal, health, current events).
184. **Why are LLMs called "large"?** — Because they're trained on huge amounts of data and contain billions of parameters — large data gives "experience," large parameters give "learning power."
185. **Why is text broken into tokens before GPT processes it?** — Because computers process text more efficiently after it's broken into smaller standardized units (tokens), rather than working with full raw sentences directly.
186. **What happens after tokens are converted into embeddings?** — The transformer processes those embeddings using attention to understand context. (Embeddings give meaning in numeric form; attention connects those meanings together.)
187. **Why are token limits important?** — Because both input and output consume tokens, and every model has a fixed context limit — you're not given unlimited processing per conversation. This limitation is part of why prompt engineering (writing clear, short, focused prompts) matters. (A joke was shared about developers who've grown so reliant on AI-generated code that when their token credits run out, they've "forgotten how to code manually" — illustrating over-reliance on AI tools.)
188. **Main difference between a normal language model and an LLM?** — An LLM is trained on much larger data and has many more parameters, making it a more powerful, large-scale version of a language model.
189. **What is positional encoding used for?** — To tell the transformer model the order/position of tokens in a sequence. This matters because word order changes meaning — e.g., "Dog bites a man"

vs. "Man bites a dog" use identical words/tokens but mean completely different things. Positional encoding is what allows the transformer to distinguish word order, not just word identity.

Key takeaways

- AI → Machine Learning → Deep Learning is a nested relationship: ML is a way of achieving AI, and deep learning is a specific many-layered technique within ML.
- GPT = Generative Pre-trained Transformer; its core loop is tokenize → embed → apply attention → generate output.
- Embeddings turn words into meaningful multi-dimensional numbers; attention figures out which words matter to each other; positional encoding preserves word order (which changes meaning).
- The context window is short-term/limited memory, not permanent memory of the whole conversation history.
- Hallucination happens because GPT predicts plausible-sounding text based on patterns, not because it "checks" facts — human verification is always required for important answers.
- Token limits (on both input and output) are a real, practical constraint — this is a core reason prompt engineering (being clear and concise) matters.

Demo 25: Foundations Review Part 3 — n8n Automation Basics (Nodes, Triggers, and Workflow Activation)

A short revision quiz covering the fundamental building blocks of n8n workflows: trigger nodes, wait nodes, if nodes, set/edit fields nodes, credentials, HTTP request nodes, and what happens when a workflow is activated.

What you'll learn

- The purpose of each core n8n node type used throughout the course
- How the "wait" node enables real drip-style email/marketing sequences
- What credentials are for and why they're needed
- What "activating" (publishing) a workflow actually does

Step-by-step notes

190. **Trigger node** — Its purpose is to start a workflow when something happens (an event occurs). Examples given: Chat trigger, Gmail trigger, Form trigger, n8n Form trigger. In n8n, a trigger starts the workflow when an event happens — such as a new email arriving, a form submission, or a scheduled time being reached.
191. **Wait node** — Pauses the workflow for a specific amount of time, or until a condition is met, then continues.

- **Real example (marketing/drip-email funnel)**: When a student inquiry email arrives asking about an n8n course, the workflow immediately sends batch details. Then, using a Wait node, it waits 24 hours and sends a second follow-up email (e.g., interview questions or learning objectives about n8n), waits another 24 hours for a third email, and so on for up to 7 days — creating an automated "nurture" email sequence. This is the same pattern used by e-commerce sites (e.g., Amazon) that follow up roughly 24 hours after you search for a product with a reminder or price update.

192. **If node** — Checks a condition and routes the workflow down a different path depending on whether the condition is true or false (a branching/conditional node).
193. **Set node / Edit Fields node** — Used to create, rename, or modify data fields within the workflow — for example, adding a new field mid-workflow, renaming an existing field, or changing a field's value. Useful for preparing/cleaning data as it flows through the workflow.
194. **Credentials** — Used to safely store login details or API keys for connected apps/services (e.g., Gmail, Google Sheets, Slack, OpenAI), so n8n can securely connect to those services without exposing sensitive login info directly in the workflow.
195. **HTTP Request node** — One of the most commonly used nodes. It sends an HTTP request to a URL and captures the response — used for calling external APIs or website services to get or send data.

- **Example shown**: an HTTP Request node configured with the URL `durgasoftonline.com/s/pages/newbatches` using the **GET** method, used to fetch new batch information from the DurgaSoft website whenever a student inquires about (for example) Java batches.

196. **Workflow activation (publishing)** — When you activate/publish a workflow, it becomes ready to run automatically whenever its trigger condition occurs — you no longer need to run it manually. (Note: in older n8n versions this was called "Activate/Deactivate"; in newer versions it's called "Publish/Unpublish," but the concept is the same.)

Key takeaways

- Every n8n workflow needs a **trigger node** to start it (e.g., new email, form submission, scheduled time).
- The **wait node** is what makes multi-day drip campaigns and delayed follow-ups possible (pause for time or until a condition).
- The **if node** handles conditional branching; the **set/edit fields node** handles shaping/cleaning data mid-workflow.
- **Credentials** securely store the login/API details n8n needs to connect to external apps (Gmail, Sheets, Slack, etc.) — don't hardcode secrets directly in nodes.
- The **HTTP Request node** is the primary way to pull data from or send data to external websites/APIs (commonly used with GET requests to fetch information).
- **Activating/publishing** a workflow is what makes it run automatically in the background whenever its trigger condition is met, instead of requiring manual execution.

Demo 26: Prompt Engineering Revision (Quiz-Style Recap) and Prompt vs Context Engineering

This session is a revision class. The instructor runs a quick quiz-style review of everything taught earlier about prompt engineering, then closes with a discussion on how "context engineering" differs from (and builds on) prompt engineering.

What you'll learn

- A recap of core prompt engineering ideas: what a prompt is, why clear prompts matter, and what happens with confusing prompts
- The RTCO (Role, Task, Context, Output format) prompt formula
- Control techniques: tone control, length control, step-by-step (chain-of-thought) prompting, format control, "only output this" instructions, and word choices like "exactly" and "strictly"
- Why conflicting instructions confuse AI, and why reusable prompt templates and structured output are useful
- The difference between prompt engineering and context engineering, and how a "skill" (e.g. a Claude Skill) fits in as a reusable instruction set

Step-by-step notes

197. **What is a prompt?** A prompt is a clear instruction given to AI to get the output you want. If the instruction is confusing, the AI has to guess, and it may give a wrong or random output — so never give AI the chance to "guess."
198. **Comparing prompts.** "Explain Java" is a weak prompt because it doesn't say who the answer is for or how it should be written. "Explain Java basics for freshers in simple English with one example" is a strong prompt because it is specific about audience, style, and format.
199. **The RTCO formula** (recapped from earlier sessions):

- **R – Role**: tells the AI how it should behave (e.g., act like a teacher, HR interviewer, or assistant).
- **T – Task**: what you want the AI to actually do.
- **C – Context**: background information so the AI understands the situation (e.g., "the student is a fresher who just completed BTech, at a beginner level").
- **O – Output format**: how the final answer should be structured.

200. **Tone control.** This decides the "style" of the AI's response — friendly, professional, funny, motivational, etc. For office emails, a professional tone is best because it gives clean, formal communication.
201. **Length control.** This decides how long the answer should be (short, medium, long, or a fixed word count, e.g. "in 200 words"). Controlling length matters both for clarity and to control token cost — you don't want a 200-page answer when you need one line.

202. **Step-by-step prompting (Chain of Thought / COT).** Asking the AI to explain step-by-step gives a clearer, more structured, and easier-to-understand explanation.
203. **Format control.** This tells the AI how the output should look — bullet points, table, checklist, steps, or story. Examples given:

- **Table** is best for comparisons (e.g., differences between two languages).
- **Checklist** is best for tracking completed vs. pending tasks (to-do lists).
- **JSON** is best for automation tools and APIs because it is clean, structured, and machine-readable.

204. **"Only output this" instruction.** This is very important for automation workflows. If you tell the AI "only output this" (e.g., only valid JSON, no extra text), it avoids adding extra commentary or suggestions that would break an automation expecting a clean JSON response. Example of a good instruction: "Reply only in valid JSON, do not add extra text."
205. **Control words like "exactly" and "strictly."** Use "exactly" when you want a fixed number of results (e.g., "exactly three points"). Use "strictly" when you want the AI to follow a given format without changing its structure.
206. **Avoid conflicting instructions.** If one line of your prompt says "be brief" and another says "give a detailed explanation," the AI gets confused and the output becomes inconsistent — the AI might favor either short or long output unpredictably. Keep instructions specific and non-contradictory.
207. **Reusable prompts (templates).** A single, well-structured prompt format can be reused again and again for different topics — you just change the topic name and keep the same structure. This saves time and produces consistent output.
208. **Structured output.** Making the AI follow a fixed format every time makes the output professional and reusable, especially when scaled across many prompts.
209. **Prompt Engineering vs. Context Engineering** (class discussion):

- **Prompt engineering** = writing a clear, well-structured instruction (the "how" of asking).
- **Context engineering** = giving the AI the full background — data, examples, memory, files, tools, and instructions — so it can respond correctly (the "what" you give it before asking).
- Example: "Explain Python loops for beginners in simple English" is good prompt engineering. But context engineering goes further: "The student is a non-IT fresher, knows only basic English, is preparing for interviews, use Indian examples, avoid complex words, first explain the concept, then give an example, then ask an MCQ — now explain Python loops."
- Context engineering is considered a broader, more advanced concept than prompt engineering; prompt engineering can be thought of as a subset of context engineering.
- A "skill" (such as a Claude Skill) is related to context engineering — it's a reusable set of instructions/metadata you teach the AI once, so it can perform that activity multiple times without you re-explaining it each time. For example, a company might build a "database skill" containing metadata and credentials so any team member can reuse that context repeatedly.

Key takeaways

- A prompt is only as good as its clarity — vague prompts lead to random or wrong output.
- The RTCO formula (Role, Task, Context, Output format) is a reliable structure for writing prompts.
- Different control techniques (tone, length, format, step-by-step, "only output this," "exactly," "strictly") let you fine-tune AI responses for specific needs, especially automation.
- Never give conflicting instructions in the same prompt.
- Context engineering is a broader concept than prompt engineering: it means preparing the AI with full background information (data, files, tools, examples) before asking it to do a task, rather than just writing one clear instruction.
- Reusable prompt templates and "skills" save time by letting you define instructions once and reuse them repeatedly.

Demo 27: Using Claude's "Dispatch" Feature to Control Your Desktop from Your Mobile

This session introduces the **Dispatch** feature in Claude Desktop, which lets you send instructions from the Claude mobile app while the actual work is executed on your desktop/laptop computer — even when you're away from it. The instructor demonstrates this with three real use cases: generating an attendance report, creating a class report, and generating student certificates.

What you'll learn

- What the Dispatch option is and why it's useful
- How to pair the Claude desktop app with the Claude mobile app to use Dispatch
- How to give prompts from your phone that make Claude do real work on your desktop (reading Excel files, creating Word docs, generating certificates)
- Practical safety/security notes about what Dispatch can and cannot access

Step-by-step notes

210. **What is Dispatch, conceptually?** Imagine you're traveling and can't be at your desktop, but you need work done there (e.g., preparing MCQs from a notes file, or generating certificates from a student list). Instead of waiting, you can send an instruction from your phone to Claude, and Claude performs the actual work on your desktop computer while you're away.

- Your phone becomes a **remote controller**.
- Your computer does the **actual work**.
- Claude acts as your **digital assistant**, executing tasks on your computer on your behalf.

211. **Requirements to use Dispatch:**

- Your desktop must be powered on (it can be locked, but must be on).
- You need the Claude desktop app installed and set up with the Dispatch option.
- You need the Claude mobile app installed on your phone.
- **Both the desktop and mobile app must be logged in with the same Claude account** — otherwise they won't sync/connect.
- Dispatch is a Pro-tier feature; it may not be visible for free-tier accounts.

212. **How to set up Dispatch:**

213. On the desktop Claude app, find the **Dispatch** option (near settings — look for "Keep awake," "Mobile notifications," etc.).

214. Click Dispatch; it may prompt you to **pair with your mobile device**.

215. On the mobile Claude app, open the left-side menu (three lines) — you'll see options like Chats, Projects, Artifacts, Code, and **Dispatch**.

216. Once paired, you can type instructions into the Dispatch chat from your phone, and they execute on the desktop.

217. **Use Case 1 — Attendance report from a distance:**

- Prompt used: *"Cloud [Claude], open the Durga Soft folder on my desktop. Check the file student attendance.xlsx. Prepare a simple report with the total students, present students, absent students, and the top five regular students. Save it in the same Durga Soft folder as today class report."*
- Claude read the Excel file, analyzed attendance data, and generated a new Word document report with total students (30), present (27), absent (3), attendance rate (90%), and a list of the top 5 regular (100% attendance) students, plus names of absentees.
- The generated file appeared in the desktop folder, and the same summary could also be viewed directly in the mobile app.

218. **Use Case 2 — Auto-generating MCQs from notes:**

- Prompt used: *"Cloud, open the [folder] on my desktop. Open prompt engineering notes. Create 10 simple MCQs from the notes. For each question, provide four options, correct answer, and a one-line explanation. Save the output as prompt engineering mcq.doc file."*
- Claude read the existing notes document and generated a new file with 10 multiple-choice questions, each with four options, the correct answer, and a short explanation — saved as a separate file in the same folder.

219. **Use Case 3 — Auto-generating certificates for a batch of students:**

- Prompt used: *"Cloud, open Durga Soft folder on my desktop. Open student names.xlsx file. Use certificate template. Create one certificate for every student name. Replace the placeholders. Create a new folder called certificates inside Durga Soft folder. Save all generated certificates inside the certificates folder. Use clear file names like certificate001_RaviKumar.xxx and so on."*
- Claude read a list of 10 students from Excel, used an existing certificate template (with placeholders for name, course, date, certificate ID), created a new "certificates" folder, generated one certificate per student, and saved them all with clear filenames — all from a single prompt.
- The instructor noted this used to require substantial manual work; a single well-written prompt handled the entire batch job.

220. **Q&A / important clarifications from the class:**

- Dispatch is one-directional: you can control the desktop from mobile, but not the reverse (mobile actions from desktop).
- There is some latency (a delay of a minute or two) when using Dispatch from mobile, mainly due to syncing between mobile and desktop — this is normal and not a bug.
- Dispatch is currently supported on Mac and Windows; there's no native Linux version (though the instructor mentioned there might be indirect ways via VMware).
- **Security**: Claude only accesses the specific folder(s) you grant it permission to use — it asks for permission before touching any file or folder, and you can choose one-time or repeated access. It does not touch other folders on your system.

- The range of Dispatch is not limited like a TV/AC remote — you can trigger work on your desktop even from a different country, as long as both devices are online and logged into the same account.

Key takeaways

- Dispatch turns your phone into a remote control for Claude Desktop, letting you delegate real desktop tasks (reading files, generating documents, creating certificates) from anywhere.
- The actual computation/work always happens on the desktop machine — the phone just sends the instruction.
- Both devices must be logged into the same Claude account for Dispatch to work, and the desktop must stay powered on.
- Claude only accesses folders you explicitly permit, making it reasonably safe from a security standpoint.
- This feature is well suited for on-the-go admin/data tasks: attendance reports, generating documents from templates, and batch-creating files like certificates.

Demo 28: Combining Claude with Canva for Design, Posters, and Social Media Automation

This session shows how to connect Claude to Canva (a popular graphic design tool) using Claude's Connectors feature, so that Claude can generate posters, Instagram posts/carousels, YouTube thumbnails, logos, and certificate templates directly through simple text prompts — and even schedule these designs to be created and posted automatically.

What you'll learn

- What Canva is and why combining it with Claude is powerful for content creation
- How to connect (or verify the connection of) Canva to Claude using Connectors
- How to write prompts that make Claude design posters, Instagram posts, YouTube thumbnails, logos, certificate templates, and carousels via Canva
- How to edit, download, and share Canva-generated designs
- How this combination can be used for daily automated content generation (e.g., an "AI news" poster emailed every morning)

Step-by-step notes

221. **Why combine Claude with Canva?** Canva is a widely used design tool for posters, YouTube thumbnails, Instagram content, and video editing. Normally, using Canva well still requires you to pick templates and manually edit them (roughly 10–15 minutes minimum per design). By connecting Claude to Canva, you can just describe what you want in plain English, and Claude will generate the content and use Canva to design it — cutting the time down dramatically.
222. **Setting up the Canva connector:**
223. You need a Canva account (canva.com / canva.in) since Claude needs access to it.
224. In Claude, go to **Customize → Connectors** (or click the **+** symbol in the chat box) and look for **Canva** in the list of available connectors (along with Gmail, Google Drive, Appy Pie, Chrome, etc.).
225. If not already connected, click Canva and log in to authorize the connection.
226. Once connected, you do **not** need to manually "activate" the connector each time — simply mentioning the tool/task in your prompt (e.g., "Create a Canva poster...") is enough for Claude to automatically recognize and use it.
227. Note: Claude itself has no built-in image generation; the Canva connector is what supplies the actual design/image capability. (Other options like Google Gemini can be used inside Claude for image generation in different workflows, but this session focuses on Canva specifically.)
228. **Example prompt 1 — Simple poster:**

"Create a simple Canva poster for Durga Soft. Topic: AI tools workshop. Use professional design, attractive title, three bullet points, and call to action 'join today.' Create it in Canva and give me the Canva link also."

- Claude generates multiple poster options (in this demo, four designs) with an "Open in Canva" link for each.
- You can open the design directly in Canva to manually tweak text, positioning, or colors.
- From Canva, use **Share → Download** to export the poster (e.g., as PNG).

229. **Example prompt 2 — Instagram post:**

"Create a Canva Instagram post for Durga Soft about learning AI tools in simple English. Use modern style, clear text, and a strong call to action."

- Works the same way: multiple design options are generated, and you can edit and download the one you like.
- It's recommended to start a **new task/chat** each time you want a fresh design, rather than continuing an old thread (reduces confusion from previous context/colors carrying over).

230. **Example prompt 3 — YouTube thumbnail:**

"Create a Canva YouTube thumbnail with the title 'Top 5 AI tools you must learn.' Use bold text and attractive design."

- A short, simple prompt is often enough — Claude fills in the design details.

231. **Example prompt 4 — Certificate template:**

"Create a Canva certificate template for course completion. Add placeholders for student name, course name, date, and certificate ID."

- This certificate template (created via Canva) is what earlier "Dispatch" demos (Demo 27) used to auto-generate individual student certificates.

232. **Example prompt 5 — Logo design:**

"Create a simple and professional logo in Canva for [Brand Name]. This brand is related to [business type]. Use [color1] and [color2] in the logo. Add a suitable icon related to [theme]. The logo should look clean, modern, trustworthy, and easy to remember. Show the brand name clearly. Create three different logo options in Canva, suitable for website, Instagram, YouTube, and visiting card. After creating, share the Canva design link."

- Generates multiple logo concepts you can review and refine in Canva.
- Note: Canva has an AI generation quota per account/period — if exhausted, Claude may offer to generate similar designs directly (e.g., high-quality PNGs) without Canva.

233. **Example prompt 6 — Instagram carousel (multi-slide):**

"Create an Instagram carousel in Canva with eight slides on the topic 'life skills we learn from books.' Use a clean, modern, attractive design. Use simple English. Keep each slide short, clear, and visible. Include ideas from these books: [list]..."

- Claude generates the actual written content (titles, explanations, book references) for every slide **and** uses Canva to design the visual layout — this combination (content + design) is the key value: Canva alone can't generate the content, and Claude alone can't do polished visual design.
- A **carousel** in Instagram means a multi-page post (swipe through several slides), as opposed to a single-page **post**.

234. **Automating recurring content (scheduling):**

- You can combine Canva + Claude with scheduling instructions in a single prompt, e.g.: *"Every day at 7:00 a.m., choose one viral topic, generate content, create a carousel using Canva, and post it on Instagram."*
- A more advanced example demonstrated: *"AI news update every day at 7:00 a.m. — search for the latest AI news from the last 24 hours, pick the top 3–5 useful updates, create a Canva poster titled 'Today's AI News' with date, headline, short explanation, and a 'why it matters' section, export as an image, and email it daily at 7:00 a.m. with the subject 'Daily AI News Poster [date].'"*
- This shows Claude combining web research, content writing, Canva design, and scheduled email delivery — all from a single well-structured prompt.

Key takeaways

- Claude does not generate images/designs on its own — the Canva connector supplies actual design capability, while Claude supplies the content and instructions.
- Once Canva is connected via Connectors, you don't need to manually select it each time — just mention the tool and task in your prompt.
- A single detailed prompt can replace 10–15+ minutes of manual Canva work (choosing templates, editing text/images).
- You can always open the "Open in Canva" link to manually fine-tune a design, then download/export it (PNG, or via Share/Download options).
- The real power is combining Claude's content generation with Canva's design generation, and layering in scheduling so posters/posts/emails can be produced and delivered automatically every day.
- Canva's free tier has AI generation quota limits; Canva Pro is a paid upgrade (mentioned as roughly ₹500/month in the session) for more usage.

Demo 29: Motivational Interlude on Consistency (Kaizen/1% Rule) and an Introduction to Claude Design

This session opens with class housekeeping (Anthropic certification encouragement) and then spends most of its time on a motivational talk about the power of small, consistent daily actions (the "1% better every day" principle). It closes with a short first look at a new Claude feature called **Claude Design**.

What you'll learn

- The "1% improvement every day" (compounding) principle and why consistency beats sporadic large effort
- Practical, real-life examples of applying this principle (savings, reading, fitness, learning)
- A first introduction to **Claude Design** — a newer Claude capability (introduced around April 17, roughly two months before this session) for creating posters, slide decks, logos, Instagram carousels, one-page notes, website/app mockups, and animations, without needing Canva or design skills
- Where to find Claude Design in the Claude web app interface

Step-by-step notes

235. **Course/certificate housekeeping:** Students were encouraged to complete Anthropic's official free courses (e.g., "Anthropic 101") and share their completion certificates to the instructor's email so they can be featured for motivation on the channel.

236. **The 1% rule (Kaizen / compounding improvement):**

- If you improve just 1% every day, after 365 days you become roughly **37.7 times** better (calculated as 1.01^365 ≈ 37.7).
- The key message: don't wait to do "big" things — instead, do small, consistent things every single day. Over time, small consistent actions compound into massive, sometimes unimaginable results.
- Examples given: saving at least ₹1 every day (leads to real savings and eventually a "financial freedom" mindset once momentum builds), reading at least one page a day, doing at least 2–5 minutes of some skill-building activity a day.
- A student shared a real example: preparing only 2 minutes a day still helped them pass a competitive exam (Bihar Public Service Commission), showing small daily effort compounds into real outcomes.
- The instructor connected this directly to the course itself: the "5-minute class" format has been running daily for 5+ months without a single day's break, and has taken students from zero Claude knowledge to earning certificates — proof that small, consistent 5-minute sessions add up.
- The "tortoise and the hare" analogy was used: consistency (not raw speed or intensity) wins in the end.
- Life-skills side notes shared: sitting/eating on the floor (instead of chairs/dining tables) increases body movement and blood circulation and, per the instructor, may modestly improve health/longevity — mentioned as an example of small daily habits, not core AI content.

- Book recommendations shared for self-improvement: *How to Win Friends and Influence People* and *Stop Worrying and Start Living* (both by Dale Carnegie), plus a mention of a "Law of Attraction" video series the instructor has produced separately.

237. **Applying the same "small steps" principle to AI/automation learning:** The instructor extended the analogy to building automation — e.g., committing to build one N8N workflow every week consistently would yield 52 workflows in a year, again reinforcing "small but consistent" over "big but sporadic."

238. **Introduction to Claude Design:**

- Recap: the previous day's session (Demo 28) covered using **Canva** (an external tool) connected to Claude for design work. This session introduces an alternative: **Claude Design**, a built-in Claude capability that doesn't require Canva at all.
- Claude Design can create: posters, logos, Instagram carousels, presentations/slide decks, one-page notes, website/app mockups (prototypes), and even animations.
- **How to access it:** Go to **claude.ai** in a web browser. In the left-hand panel, look for an icon that looks like a paint/palette symbol (the instructor jokingly compared it to the Microsoft Paint icon) — hovering over it shows the label "Design." This sits alongside other modes like **Claude Code** and **Claude Co-work**.
- Note: at the time of this session, Claude Design was only visible for **Pro** subscribers (some free users in the class could not see the option), and it was **not yet available in the desktop app** — only in the web version.
- When you open Claude Design, you're offered several starting points: **Design (blank/file)**, **Slides**, **Prototype**, **Product wireframe**, **Doc**, **Animation**, or a **Blank canvas**. There's also a gallery of "Examples" (e.g., globe loaders, shader wallpapers, particle effects, app onboarding flows) that you can use as prompt inspiration/templates.
- Anthropic released an official product video showcasing Claude Design's capabilities, which the instructor played briefly for the class. The instructor promised a full hands-on walkthrough (posters, slides, logos, mockups) in the following session (Demo 30).
- A student asked about choosing between Claude Artifacts, Claude Co-work, and Claude Code for building a full website. The instructor clarified: **Artifacts** are good for lightweight, single-page website demos (not a full production app); **Claude Design** is only for visual mockups/prototypes (not functional code); for a fully functional multi-feature web app, you would use the normal Claude (chat) prompting or **Claude Code** to actually write and connect backend code (e.g., to a database like Firebase).

Key takeaways

- Consistent small daily actions (even 1–2 minutes) compound dramatically over time — 1% daily improvement compounds to ~37x over a year.
- This mindset is directly modeled by the course itself (daily 5-minute sessions, uninterrupted for months).

- **Claude Design** is a newer, built-in Claude feature (introduced mid-April) for generating posters, slides, logos, carousels, notes, and app/website mockups — without needing an external tool like Canva.
- Claude Design is accessed via the paint-palette icon on claude.ai (web), and was Pro-only and web-only (not desktop) at the time of this recording.
- Claude Design produces prototypes/mockups and visual designs, not functional code — for real backend/functional development you still need Claude Code or standard prompting.

Demo 30: Hands-On Walkthrough of Claude Design — Posters, Logos, Carousels, Website Mockups, and Mobile App Prototypes

This session is a full, practical demonstration of **Claude Design**, following up on the introduction in Demo 29. The instructor walks through a special "design prompt formula," then builds a series of real design outputs — an Instagram poster, a logo, another poster, an Instagram carousel, a website homepage mockup, and a mobile app onboarding flow — entirely from text prompts, without using Canva.

What you'll learn

- A prompt formula specifically tailored for design tasks (extending the earlier RTCO / CCOR formulas)
- How to access and use Claude Design step by step
- How to generate posters, logos, Instagram carousels, website mockups, and mobile app prototypes using natural-language prompts
- How to export/download designs (PDF, PowerPoint, HTML, image/PNG) and send them to Canva for further manual editing
- The practical business value of fast prototyping (compared to hiring a designer or web developer)

Step-by-step notes

239. **Recap of prior prompt formulas:**

- **RTCO**: Role, Task, Context, Output format (used for general prompting).
- **CCOR** (also referenced as "car" format in agentic contexts): Context, Command, Output format, Rules/Restrictions.
- For **design-specific prompts**, extra elements are needed beyond the usual formula: you must additionally specify **style/design mood**, **color scheme**, and **visual elements** (icons, imagery) — things you normally don't need to mention in a regular chat-based prompt, but which matter a lot for visual design.

240. **The Design Prompt Formula (as demonstrated):**

- **Goal** — what are you creating (e.g., a poster)?
- **Audience** — who is it for (e.g., students)?
- **Format** — what type of output (e.g., Instagram poster)?
- **Style** — visual style (e.g., clean and modern)?
- **Colors** — specific color theme (e.g., blue and white)?
- **Content** — what information/text should appear (e.g., 5 practical tips)?
- **Visuals** — any icons or imagery to include?
- Example full prompt: *"Create a clean Instagram poster on time management for students. Use simple English, bold title, five practical tips, modern design, blue and white color theme, and add small icons for clock, calendar, focus, mobile control, and study plan."*

- Note: this design-specific formula is a good practice, not mandatory — you don't need every element in every prompt, but including style/colors/visuals helps a lot for design tasks specifically (similar to how image/video prompts need camera angle/movement details, which are different from normal text prompts).

241. **Accessing Claude Design:**
242. Go to **claude.ai** in a browser (only available on web, not desktop, at time of recording).
243. Click the paint-palette icon in the interface → labeled **Design**.
244. Choose a starting point: **Design (file)**, **Slides**, **Prototype**, **Product wireframe**, **Doc**, **Animation**, or **Blank canvas** — or start from a design system, screenshot, code base, or Figma file.
245. Type your prompt and send it. Note: Claude Design consumes **more credits** than normal chat, and in this demo the instructor was using the **Opus** model, which is relatively costly — so use it thoughtfully.
246. **Demo — Instagram poster ("time management"):**

- Used the prompt from step 2 above.
- Claude generated a 1080×1080 poster (Instagram's standard aspect ratio was inferred automatically from "Instagram poster" in the prompt — no need to specify pixel dimensions).
- By default, the design was downloadable only as an **HTML document**; export options included **PDF, PowerPoint, standard HTML, and a ZIP project archive** — but not a direct image file unless requested.
- To get a downloadable image (PNG), you must explicitly ask in the prompt, e.g. add: *"Provide as a downloadable image (PNG) file."* Once added, Claude regenerated the poster as an actual PNG image, ready to post.

247. **Demo — Logo design ("SkillGrow" learning app):**

- Prompt: *"Create a simple modern logo concept for a student learning app named SkillGrow. The logo should look friendly, modern, and educational. Use blue and green colors. Include an idea of growth, learning, and [confidence]. Provide three logo concepts with meaning."*
- Claude generated **three distinct logo concepts**, each with a short explanation of its symbolism (e.g., "sprout with an arrow" representing growth and direction; "open book with a leaf" representing knowledge; "ascending steps blooming into a leaf" representing skill progression).
- Designs can be **sent directly to Canva** (via a "Send to → Canva" option) for further manual editing, or edited briefly in-line within Claude Design itself (though in-place editing is limited — essentially one manual edit pass).
- Export/download options observed: PDF, GIF, HTML, PowerPoint; to get an image file, again you need to explicitly request it in the prompt.

248. **Demo — Second poster ("Consistency Beats Motivation"):**

- Prompt: *"Create an Instagram poster on the topic 'consistency beats motivation.' Audience is students and job seekers. Use simple English, bold heading, five short points, one powerful quote, and a clean modern design style. Use a yellow, black, and white color theme."*
- Generated in under a minute; demonstrates that by default, most designs come out as editable HTML blocks, and the actual downloadable image format must be explicitly requested in the prompt.

249. **Demo — Instagram carousel ("5 life skills every student needs"):**

- Prompt: *"Create a seven-slide Instagram carousel on five life skills every student needs. Use simple Indian English. Each slide should have one title, short explanation, one example, and one small icon idea. Make it useful for students, employees, and beginners."*
- Claude generated 7 complete slides (1080×1080 aspect ratio, Instagram-carousel style) covering: time management, communication, money management, problem-solving, and adaptability — each with a title, explanation, and example.
- The instructor emphasized the time savings: manually writing this content might take 2–3 hours, and manually designing each slide (colors, CSS/HTML, layout) could take a professional designer 3–4+ hours — versus about a minute with Claude Design.

250. **Demo — Website homepage mockup ("LearnSmart"):**

- Used the **Prototype** starting option (described as equivalent to a "markup"/mockup of a website or app — showing what each screen/page should look like, without functional back-end code).
- Prompt: *"Create a clean homepage design idea for an online learning platform called Learner Smart. Audience: students and job seekers. Include hero section, course categories, success stories, benefits, call-to-action button, and simple modern layout."*
- Claude Design asked clarifying questions before generating (a distinctive feature of this mode): whether there's an existing brand/design system, what visual "vibe" fits (friendly, professional, calm, bold, or "decide for me"), a color direction, how many homepage directions to explore, whether it should be a static markup or interactive prototype (with hover/click states), and what course categories to include. You can answer each, or let Claude decide.
- Result: a full, polished one-page website mockup (hero section, course categories, testimonials, pricing, "career starts today" CTA) generated in about 10 minutes.
- Key point from the instructor: this is a **mockup/prototype**, not functional code — but it can be used as a direct reference/blueprint to then generate actual front-end and back-end code (e.g., via Claude Code), dramatically compressing typical freelance timelines (the instructor contrasted this with a quote of "30 days and ₹70,000" from a human developer for a comparable website).

251. **Demo — Mobile app onboarding flow ("fitness app"):**

- Prompt (long-form): *"Design a beautiful five-screen mobile app onboarding flow for a fitness app. App theme: dark theme, premium mobile app look, modern fitness style, clean layout, smooth gradient, attractive typography, simple and motivating text. Target users: beginners who want to

start their fitness journey."* — followed by explicit descriptions for each of the 5 screens (e.g., Screen 1: "Start your fitness journey — build strength, energy, and confidence every day," with a workout illustration; Screen 2: goal selection, etc.).

- Claude Design again asked clarifying questions (color theme choice, illustration/visual style, how many welcome-screen variations, whether screens should be static or "tappable"/interactive with progress dots, typography style) before generating the mockup.
- This produces a set of app screen mockups that function as a **UI/front-end design reference** — actual backend/app code generation would be a separate step using Claude Code.

252. **Practical workflow suggested for freelancers/founders:**

- Use Claude Design to rapidly produce mockup screens and iterate with a client (they can request color/text/layout changes directly in the prompt).
- Once the client approves the design, feed those mockup screens back to Claude (or Claude Code) as a reference and ask it to generate the actual front-end/back-end code.
- This whole cycle (design + code generation) can realistically be done in a fraction of the time (and cost) that manual design + development would take — the instructor used this to argue that clients quoting long timelines (e.g., "30 days, ₹70,000") for simple websites may not be using AI tools effectively.

253. **Housekeeping notes on usage limits:** Claude has session and weekly usage limits, viewable under **Settings → Usage** (shows time until the current session resets and percentage of weekly limit used). The instructor hit a session limit during the demo, illustrating that Claude Design (especially with Opus) can consume significant usage quota.

Key takeaways

- Claude Design uses a design-specific prompt formula: goal, audience, format, style, colors, content, and visuals — richer than a typical chat prompt because visual design needs those extra specifics.
- Claude Design can generate posters, logos, Instagram carousels, website homepage mockups, and mobile app onboarding screens directly from prompts — no separate design tool (like Canva) is required, though you can still send results to Canva for manual fine-tuning.
- By default, generated designs come as editable HTML/PDF/PowerPoint; to get a plain downloadable image (PNG), you must explicitly ask for it in the prompt.
- Claude Design produces **mockups/prototypes**, not functional code — turning a mockup into a real working website or app still requires a separate step (e.g., Claude Code) to generate front-end and back-end code.
- Claude Design sometimes asks clarifying questions (colors, style direction, number of variations, interactivity) before generating, which helps tailor the output.
- The overall time savings are dramatic: tasks that might take a professional designer or developer hours to days (writing content, designing a poster/carousel/mockup) can be done in roughly a minute to a few minutes with a well-written prompt.

- Claude Design was Pro-only and web-only (not available on desktop or, at the time, on mobile) and consumes meaningful usage credits, especially with premium models like Opus.

Demo 31: Claude Design for Mockups, Prototypes, and Animations, Plus Tips to Save Claude Credits

This session opens with a motivational talk about investing in self-improvement, then moves into a hands-on demo of Claude's "Design" feature for building app mockups, prototypes, slide decks, and short animations, followed by a detailed set of techniques for reducing Claude credit usage.

What you'll learn

- What the Claude Design feature can do (mockups, prototypes, posters, slides, animations)
- How Claude Design differs from Claude Code and normal Claude Chat
- Why Claude chats consume credits faster the longer they get
- Five practical techniques to save Claude credits during everyday use

Step-by-step notes

1. Opening motivational point (context only)

The instructor spent the first part of the session explaining why continuous self-improvement matters (comparing salaries of different roles, the rise and fall of companies like HP, Nokia, and Kodak due to lack of innovation). The core message: invest time daily in upgrading your skills, especially AI skills, because "the person who knows AI will replace the person who doesn't know AI."

2. Using Claude Design to build a mobile app prototype

254. Open claude.ai (desktop version). Look for the "Design" option — it appears at the bottom left of the interface (this is a relatively new feature, may require relaunching/updating the app to see it).
255. Start a new Design task and describe what you want, e.g., a fitness app prototype.
256. Claude Design will ask clarifying questions (e.g., preferred color/gradient combination, how interactive the design should be) before generating.
257. Claude generates multiple mobile app screens (e.g., onboarding screen, "choose your goal — lose weight or build muscle," etc.) as a working prototype you can click through.
258. This is design/prototyping only — not full coding implementation. It's comparable to what a UI designer would produce in Figma, and can also generate the corresponding front-end code from the mockup afterward.

3. Time and cost comparison

- Without AI: creating a UI mockup/prototype like this normally takes a team (designer, developer, tester) roughly 20–30+ billable hours, or 2–4 days in practice.
- With Claude Design: a comparable prototype can be produced in about 2–3 minutes.

4. Creating animated videos with Claude Design

259. Select the "Animation" option within Design (a separate flavor from Design/Coworker — not a "skill," but a distinct built-in capability).

260. Write a scene-by-scene prompt, e.g.: "Create a 10-second animated video explaining the importance of AI, why some jobs are changing. Dark theme, modern futuristic style. Scene 1: 0–3 seconds... Scene 2: 3–6 seconds..." and so on.
261. Claude generates a short animation (video without audio — it's animation/motion, not a full video).
262. To add audio: you can explicitly ask in the prompt for narration text, then use another tool (e.g., ask Gemini/Flow, or a tool like Suno) to generate matching audio, or have Claude generate a script for a separate text-to-speech step.
263. Claude Design can also generate slide decks (PPT-style) and posters/logos using the same design capability.

5. Model selection matters

- The instructor initially used Opus 4.8 for speed in the demo, but noted it consumes far more credits than other models, especially for a 10-slide/10-second video task.
- You can check usage under Settings > Usage, though exact token counts per response are not shown — only overall usage.

6. Tips to save Claude credits (the core teaching of this session)

Tip 1 — Don't keep asking follow-up questions in the same long chat.

Every new question in an existing chat forces Claude to re-read the entire chat history before answering. At scale, up to ~98% of credit usage can go toward re-reading old context rather than generating the new answer. Recommendation: start a new chat/task for anything not directly related to the current thread.

Tip 2 — Edit your prompt instead of sending a follow-up correction.

If you asked for "2 MCQs" but actually want 5, don't type "give me 3 more" as a new message (this reads and re-processes the whole history). Instead, use the **Edit prompt** option on your original message to change it to "5 MCQs" and resend — this keeps it as effectively the first message in the chat and avoids the growing-context cost.

Tip 3 — Don't let a single chat run too long; summarize and start fresh.

After roughly 15–20 messages, a chat becomes expensive because every new message re-reads the whole history. When this happens:

264. Ask Claude: "Summarize our current chat with all important points, the decisions, and the next steps."
265. Copy that summary.
266. Start a new chat and paste in: "This is the summary of my previous chat. Continue from here."

This preserves context while resetting the token overhead. You can also switch models when starting the new chat (e.g., switch to Haiku for lower-stakes continuation work) — the model doesn't have to stay the same throughout.

Tip 4 — Batch small related questions into a single message.

Instead of sending separate messages like "Explain Claude projects," then "give five examples," then "create MCQs," then "give a classroom analogy" one after another (each forcing a re-read of prior messages), combine them into a single prompt: "Explain Claude projects in simple English, give five examples, create five MCQs, add one classroom analogy." This produces a similar answer for far fewer credits.

- Caveat: only batch *small* tasks. Don't batch large/heavy tasks together (e.g., "explain 10 topics + create 100 MCQs + write a poster prompt + write an email + create a full course plan" in one prompt) — this reduces output quality. Balance batch size against task complexity.

Tip 5 — Set a standing instruction in Claude's settings.

In Settings → General → "Instructions for Claude," you can add a permanent instruction such as: "Be concise and practical. I'm trying to save Claude account usage and reduce token consumption. Avoid unnecessary repetition. Use simple English. When the chat becomes long, advise me to start a new chat and give me a short continuation summary. Also suggest better prompt methods when they can save tokens." Claude will follow this across all chats and even proactively suggest more targeted follow-up questions.

7. Other reminders from this session

- PDFs and Claude "are enemies" — avoid uploading/generating unnecessary PDFs, as they consume more credits. If you need Claude to read a PDF, copy its text content into a Word/text file first and share that instead.
- Reference chat history/memory settings increase credit usage because Claude has to search across all past chats — use with awareness of the cost trade-off.

Key takeaways

- Claude Design can generate app prototypes, posters, slide decks, and short animations directly from a text description, cutting design/prototyping time from days to minutes.
- Claude Design is a separate specialized capability from Claude Code and Claude Chat — not a "skill" add-on.
- Every message in a chat causes Claude to reprocess the full chat history, so credit usage compounds as chats get longer.
- Five key credit-saving habits: (1) start new chats/tasks for unrelated work, (2) edit prompts instead of sending corrections as new messages, (3) summarize and restart long chats, (4) batch small related questions into one message, (5) set a standing "be concise" instruction in Claude's settings.
- Avoid uploading PDFs directly to Claude; convert to text first.

Demo 32: Automation Workflows with N8N, Semantic vs Keyword Search, Model Selection, and More Credit-Saving Tricks

This session shows two real N8N automation workflows built for Durgasoft (a one-rupee download confirmation email flow and a course-inquiry autoresponder), explains the difference between keyword search and semantic/AI search, and continues the previous session's theme with a deeper breakdown of Claude's models, effort levels, memory import, and Projects feature for saving credits.

What you'll learn

- How to design a simple N8N workflow that sends automated confirmation emails and logs data to Google Sheets
- The practical difference between keyword-based search (Google, plain code) and semantic search (LLMs)
- When to use Haiku, Sonnet, or Opus, and what the "effort" and "thinking" settings do
- How to import your ChatGPT memory/context into Claude
- Why using Claude "Projects" saves credits compared to open-ended chats

Step-by-step notes

1. Why the "pay ₹1" download requires an automated email (N8N workflow #1)

- Durgasoft made prompt-engineering material cost ₹1 (not free) specifically so that a real payment event exists, which lets an N8N workflow trigger an automatic confirmation email.
- After payment, the workflow automatically:

267. Sends an email with links to free YouTube playlists (GenAI, Prompt Engineering, Agentic AI, N8N Automation, Claude AI).
268. Saves the student's info as a new row in a Google Sheet for record-keeping.

- If the material were free, there'd be no reliable trigger event and no confirmation email would go out.

2. Course inquiry autoresponder workflow (N8N workflow #2)

269. A student fills out an inquiry form (e.g., typing a course name like "Agentic AI" or "AI").
270. An HTTP Request node pulls current batch/course data from the Durgasoft website.
271. A filter step tries to match the entered course name against available batches.
272. If a match is found, an HTML email is generated with batch time, faculty, Zoom link, and a "Register Now" button, and sent to the student.
273. The student's inquiry is also logged to a Google Sheet.
274. **The flaw demonstrated live:** typing "AI related" or "Generative AI" (when the actual course is named differently) returned "no active batch found," because the matching logic was implemented in plain JavaScript, which only does exact/keyword matching — it doesn't understand meaning.

3. Keyword search vs. semantic search

- **Google / plain code = keyword search.** It matches exact words/substrings. If the course is called "Agentic AI" and you type "AI related," code-based matching won't find it because that exact phrase isn't present.
- **ChatGPT / Claude / LLMs = semantic search.** They understand meaning, so "AI related" would correctly be understood to include GenAI, Agentic AI, Prompt Engineering, etc.
- Semantic search effectively covers keyword search too (exact matches are also understood), so there's little need for a separate hybrid approach for most cases, though the group noted some AI systems do use hybrid ranking (e.g., combining page-rank-like signals with semantic understanding).
- Trade-off: using an LLM for this kind of matching costs more (token/API cost) than writing custom matching code, so there's a genuine "cost vs. capability" decision — for small-scale internal tools, plain code or even a dropdown list of valid course names can be a cheaper practical fix.
- The group also observed that people increasingly search for things on ChatGPT/Claude instead of Google, because AI tools give meaning-based (not just keyword) answers, and modern models can also do live web search when needed so their knowledge doesn't stay frozen at a training cutoff.

4. Why automation and workflows matter (business point)

- Building automated workflows (like the two above) means the business logic keeps running 24/7 without manual effort, freeing people to focus on higher-value work.
- The instructor's advice: don't just work hard hour to hour — build systems and pipelines that work for you continuously.

5. Real-world context shared in the session

- A large company reportedly lost significant money after giving employees unlimited Claude access, leading to a massive, unexpected credit/usage bill — reinforcing the need for usage discipline and account-level limits.
- A separate news item was discussed about a services company's (Accenture-related) profit miss attributed to AI impact, which the group linked to broader stock market movement in IT shares — illustrating real business stakes around AI adoption and cost management.

6. When to use Claude vs. other AI tools (cost strategy)

- Recommendation: do preliminary/simple work in other models (ChatGPT, Gemini) where credits aren't a constraint, and reserve Claude only for the final, most crucial/complex part of a task.
- Example: if a task has parts A, B, and C, and only C truly needs deep analysis, do A and B in ChatGPT and bring only C to Claude.

7. Claude's three models and when to use each

- **Haiku** — cheapest, fastest. Use for simple tasks: summaries, data extraction, basic Q&A, formatting, classification, bulk/routine work. Recommended for ~90% of routine work (combined with Sonnet).

- **Sonnet** — balanced, mid-tier. Use for: drafting, coding, research summaries, document analysis, multi-step tasks, MCP/tool-related work. This is the default model most people use in Claude Code.
- **Opus** — most powerful and most expensive. Reserve for only the top ~10% of high-value work: final review, important decisions, complex reasoning, strategy planning, high-stakes writing, final quality checks. The instructor demonstrated that asking the same question to Haiku, Sonnet, and Opus produces noticeably different depth of analysis (e.g., a "pharmacist selling medicine" vs. a "drug/health consultant" level of response) — Opus gives more expert-level, multi-factor answers.

8. Effort levels and "Thinking" mode

- Newer Claude interfaces show an **Effort** selector: Low, Medium, High, Max.
- Low = daily/normal work.
- Medium = important work.
- High/Max = only for critical work (selecting Max shows a warning about excessive token usage and longer response times — "use sparingly for the hardest tasks").
- **Thinking (extended thinking)** should stay disabled by default. Only enable it when deep analysis is genuinely required, since it consumes significantly more credits when active.

9. Setting a standing "be concise" instruction (recap and further detail)

- Go to Profile → Settings → General → "Instructions for Claude."
- Paste an instruction like: "Be concise and practical. I'm trying to save Claude account usage and reduce token consumption. Avoid unnecessary repetition. Use simple English. When the chat becomes long, advise me to start a new chat and give me a short continuation summary. Also suggest better prompt methods when they can save tokens."
- Demonstrated live: after adding this instruction, the same question ("Explain about Java OOPs") produced a noticeably shorter response, and Claude even appended a "token saving tip" suggesting narrower follow-up questions — a claimed ~70% reduction in response size.

10. Importing memory/context from another AI provider

275. Go to Profile → Settings → Capabilities → "Import memory from other AI providers."
276. Click **Start Import**. Claude gives you a prompt to copy.
277. Paste that prompt into a new chat in your other AI tool (e.g., ChatGPT). It will export your stored memory/context — identity, career, projects, preferences, instructions — in a structured format.
278. Copy that exported output back into Claude's import box and click **Add Memory**.
279. Result: Claude now has the same personalized context ChatGPT had about you (name, role, work style, projects, preferences), enabling more customized responses. Note: this step is about personalization, not credit savings.

- The same import process works in reverse (ChatGPT importing from Claude).

11. Claude Projects save credits

- Using a Project scopes Claude's context to only that project's chats, files, and resources — it does not search across your entire chat history the way memory-enabled general chat does.
- Because the search space is isolated/smaller, credit usage is lower than long, memory-enabled general conversations.
- To use: go to Projects → create/open a project (e.g., "How to use Claude") → all chats, files, and task lists specific to that project live there and are the only context Claude considers.

12. Recap of all credit-saving tips from this session

280. Add a standing "be concise" instruction in Settings.
281. Avoid long, sprawling single chats — summarize and start a new chat instead.
282. Choose the right model for the task (Haiku for simple, Sonnet for balanced, Opus only for high-value work).
283. Don't default to Claude for everything — use ChatGPT/Gemini for simpler work and reserve Claude for critical tasks.
284. Watch Effort level — don't accidentally leave it on High/Max.
285. Keep Thinking mode off unless deep analysis is truly needed.
286. Use separate accounts for different life contexts (e.g., one for personal/family use, one for professional use) so memory/context doesn't mix and produce confusing cross-context answers.
287. Use Projects to scope and isolate context, reducing unnecessary re-reads.

Key takeaways

- Plain code and Google-style search only match keywords; only AI models do true semantic (meaning-based) search — this is why AI-based matching succeeds where a simple JavaScript filter fails.
- N8N workflows (HTTP requests, filters, email generation, Google Sheets logging) can fully automate confirmation emails and course-inquiry responses, running continuously without manual work.
- Match your model to the task: Haiku for simple/bulk work, Sonnet for balanced daily work, Opus only for the top ~10% of critical, high-value tasks.
- Disable "Thinking" mode and keep Effort on Low/Medium unless a task genuinely needs deep reasoning — both settings significantly increase credit consumption when enabled.
- Claude Projects and a standing "be concise" instruction are two of the most effective, low-effort ways to cut credit usage.
- You can import your existing AI memory/context from one provider (e.g., ChatGPT) into Claude so it "knows you" without starting from scratch.

Demo 33: Introduction to Claude Code — What It Is and Why It Changes How You Build Software

This session introduces Claude Code (Anthropic's AI coding assistant) at a conceptual level using an AI-generated explainer, discusses real examples of its impact (including an IBM COBOL migration story), and closes with an extended discussion on why AI fluency is becoming essential for careers, plus a set of MCQs to reinforce the concepts.

What you'll learn

- What Claude Code is and what it can do differently from a normal chat
- Where Claude Code can run (terminal, IDE, desktop, browser, Slack, GitHub, CI/CD)
- Why Claude Code always asks for permission before making changes
- A real-world example of Claude Code's impact on legacy system migration
- Why "confidence in using AI" is becoming a core career skill

Step-by-step notes

1. What is Claude Code?

Using ChatGPT to generate a plain-English explanation, the instructor walked through Claude Code's definition:

- Claude Code is an AI coding assistant from Anthropic.
- It helps developers **write, edit, debug, and understand code**.
- It works like a coding partner inside your **terminal, IDE, desktop app, browser, or Slack**.
- You give it instructions in normal English (e.g., "Find the bug in this project," "Create a login page").
- It can **read your project files and understand the full codebase** (not just a single snippet).
- It can **edit files directly** — you don't need to copy-paste code back and forth.
- It can **run terminal commands, test code, install packages, and fix errors**.
- Useful for: creating new features, fixing bugs, explaining old/legacy code, writing tests, refactoring, and handling Git-related tasks.
- **Important safety behavior:** it asks for permission before changing files or running commands — this is a deliberate safety measure, especially important for production systems.
- It runs locally in your terminal and connects to Claude's model APIs — no separate backend server or remote code index is required.

2. Example prompts for Claude Code

- "Check this project and explain how the login system works."
- "Fix the payment page error and run the tests."

These show Claude Code's typical loop: read the project → locate the issue → edit the code → run tests to confirm the fix.

3. Real-world example: legacy code understanding

- A common historical pain point: a developer inherits old code (possibly in an old language like COBOL or Pascal) and has to manually trace through it line by line to understand what it does — often a slow, frustrating process, especially for beginners.
- Claude Code (and similar AI coding tools) can now read and explain unfamiliar/legacy code instantly, removing this bottleneck.
- **IBM COBOL example (as discussed):** Anthropic/Claude reportedly released a plugin capable of understanding COBOL code running on IBM mainframes and helping migrate it away from that legacy stack. The instructor stated this news event coincided with a roughly 23% single-day drop in IBM's share price, illustrating how disruptive this migration capability could be to the mainframe/legacy-system business model. (Note: this is relayed as reported news in the session, not independently verified in the transcript.)

4. Claude Code as a "smart junior developer"

- Analogy used: Claude Code is like a smart junior developer sitting inside your laptop — you give tasks in English, it reads your project, writes and fixes code, and runs commands, but **you are still the boss** (you review and approve).
- A student raised a good caution: even with Claude Code, you still need enough basic understanding to guide it properly and review its output — otherwise "it becomes the boss" instead of you.
- A domain-specific example was shared: a student used a Claude "life science" domain plugin to quickly gain working knowledge of "proteomics" (a domain they'd never worked in) and delivered a client project within one week — something that might otherwise have taken months to ramp up on.

5. What Claude Code can do (comprehensive list from the session)

- Understand the full project structure, not just isolated files.
- Explain which file does what.
- Help beginners learn like a teacher; help experienced developers work faster.
- Create new files, update existing files, remove unnecessary code.
- Help with bug fixing by reading error messages and checking related files.
- Run commands: install packages, run tests, start the project, check for errors.
- Help with Git-related tasks: understanding changes, creating commits, reviewing.
- Work inside VS Code, JetBrains IDEs, terminal, desktop app, web, Slack, and CI/CD workflows.
- Can be mentioned directly in GitHub issues/pull requests (e.g., "@Claude") to analyze problems or suggest changes.
- Customizable via settings/config, and supports skills and custom commands.
- Useful for students (learning project structure, debugging, testing) and for companies (reducing routine coding work, speeding up bug fixes, faster code review, improved developer productivity).

- One participant compared it directly to GitHub Copilot — the instructor agreed it performs a similar role.

6. MCQ-style review (concepts reinforced through quiz questions in the session)

- Claude Code is mainly used for: writing, understanding, and fixing code (not photo editing, video, or music).
- Claude Code works mainly inside coding tools like terminal, IDE, and project files.
- Claude Code reads and understands project files, so it can explain project structure and help with changes — and it understands the whole codebase, not just a snippet.
- You give Claude Code instructions using normal English instructions (not machine language or audio commands).
- Before changing files or running important commands, Claude Code should **ask for user permission** — this is the safe, correct behavior.
- The best way to use Claude Code safely on a real project: **give clear instructions, review changes, and approve commands carefully** (not "let it change everything without checking").

7. Broader discussion: AI fluency as a career skill

- The instructor's core message: knowing how to work effectively with AI tools is becoming more important than knowing low-level coding syntax by heart.
- Real anecdotes shared by participants:
- A PM reportedly said he no longer asks candidates to code in interviews — he asks about architecture thinking and AI-tool fluency instead.
- Some companies (e.g., Nvidia, per a participant's account) now run an "AI round" as part of interviews.
- A participant used Claude to simulate a mock interview ("roast me like an interviewer") to prepare for real interviews.
- The instructor's advice for personal use: use AI for everything in daily life — career advice, financial guidance, health/fitness plans, emotional support, learning new topics — not just professional coding tasks.
- On kids/education: encourage children to use AI tools to understand difficult material (e.g., explaining a hard book "for a 10-year-old," or explaining a tricky math problem step by step) rather than avoiding AI, while being mindful not to let AI simply do their assigned homework for them.
- A cautionary counterpoint was raised by a participant: does over-reliance on AI reduce our own thinking ability? The instructor's response: you still need to form the thought/requirement yourself in order to guide the AI effectively — AI amplifies your direction, it doesn't replace the need to have one (illustrated with the Henry Ford "mastermind team" anecdote — knowledge doesn't need to be memorized if you know how to access the right expertise).
- On confidentiality/regulated environments: some organizations (banking, government, defense) restrict use of public AI tools and instead use isolated/in-house hosted models for sensitive work — but this is expected to become less of a barrier over time.

Key takeaways

- Claude Code is an AI coding assistant that can read, understand, write, edit, debug, and run code across your whole project — not just answer isolated code questions like a normal chat.
- It always asks for permission before changing files or running commands, which makes it safer to use even on production-adjacent work.
- It's available across many environments: terminal, VS Code, JetBrains IDEs, desktop app, browser, Slack, GitHub, and CI/CD pipelines.
- Even with powerful AI coding tools, you still need enough understanding to give clear instructions and review/approve the changes — you remain responsible for the outcome.
- The instructor's overarching message: build genuine confidence in using AI tools daily, in both professional and personal contexts, since AI fluency is increasingly what differentiates people in interviews and careers — not memorized coding trivia.

Demo 34: Quick Recap Quiz — Claude Chat vs Claude Code vs Claude Coworker vs Claude Design

A short, focused revision session using multiple-choice questions to consolidate the differences between the four Claude "modes" covered in recent sessions: Claude Chat, Claude Code, Claude Coworker, and Claude Design.

What you'll learn

- The distinct purpose of each of Claude's four main modes/products
- How to quickly decide which mode to use for a given task
- A clear side-by-side comparison to avoid confusing the tools

Step-by-step notes

1. Session format

This was a short, quiz-driven revision session (the instructor mentioned time constraints due to travel) covering material from the previous few sessions via multiple-choice questions, each followed by the correct answer and a brief explanation.

2. Claude Code — recap

- Correct definition: mainly used to help developers work with **code files, terminal commands, and projects**.
- It's a coding assistant inside your development workflow — helps with coding, fixing errors, explaining code, and modifying files.
- Useful because it works with the **real project structure**, not just a normal chat window.

3. Claude Chat — recap

- Best for: **general conversation, explanation, writing, learning, and brainstorming** — not for running terminal commands, editing local files, or connecting external design tools directly.
- Described as being like a "smart teacher or discussion partner" — useful for learning topics, creating content, improving prompts, and getting explanations.
- Important distinction: Claude Chat does **not** directly control your local files the way Claude Code does. For file/directory creation, you need Claude Code or Claude Coworker.

4. Claude Coworker — recap

- Useful when you want Claude to work with **connected tools**: Gmail, calendar, documents/files, and work apps.
- Described as a "work assistant connected to your apps" — helps with real work tasks like checking emails, calendar entries, files, and workflows.
- Difference from Claude Chat: Chat explains ideas in the abstract; Coworker can act within your actual connected work context (e.g., check your calendar, read your email).

5. Claude Design — recap

- Useful for **creating visual designs**: posters, slides, layouts, and creative concepts (not for writing back-end code).
- Focused on visual creativity, not coding or general work tasks — helps make posters, presentations, graphics, and design ideas quickly.

6. Quick decision guide reinforced via the quiz

- Need to fix a bug in a Java project, update files, and run test commands in the terminal? → **Claude Code**.
- Need Claude to check your email, understand calendar meetings, and help with documents? → **Claude Coworker**.
- Need to create a course poster, slide layout, or Instagram creative for students? → **Claude Design**.
- Need general explanation, brainstorming, or writing help? → **Claude Chat**.

7. Simple summary table taught in the session

Mode	Primary purpose
Claude Chat	Thinking, explaining, writing, learning
Claude Code	Coding, debugging, editing files, project-level work
Claude Coworker	Connected work apps (Gmail, calendar, documents)
Claude Design	Visual creation (posters, slides, layouts, graphics)

Key takeaways

- Claude Chat, Claude Code, Claude Coworker, and Claude Design are four distinct modes with different jobs — using the right one for the right task avoids wasted effort and credits.
- Claude Chat cannot directly touch your local files or connected apps — for that you need Claude Code (for code/project files) or Claude Coworker (for Gmail/calendar/documents).
- Claude Design is specifically for visual/creative output (posters, slides, layouts), not for coding or general conversation.
- When deciding which Claude mode to use, ask: "Am I coding a project? Am I working with connected apps? Am I creating a visual design? Or am I just having a conversation?" — the answer points directly to the right tool.

Demo 35: Hands-On with Claude Code — Building, Enhancing, and Debugging a Python Project from the Terminal

This session is a live, hands-on walkthrough of Claude Code: the instructor builds a simple Python "student marks analyzer" project from scratch using natural-language prompts, then iteratively enhances it (moving hardcoded data into a CSV file) and demonstrates automatic bug detection and fixing — all without writing code manually.

What you'll learn

- The traditional manual steps of software development (write, run, debug, test, commit) and how Claude Code takes over each of them
- How to set up and run Claude Code from the terminal
- How to generate a working Python project from a plain-English requirement
- How to ask Claude Code to enhance/refactor an existing project
- How Claude Code detects and fixes bugs automatically
- Why Git matters when using Claude Code, and prerequisites for installing Claude Code via the terminal

Step-by-step notes

1. The traditional software development workflow (without AI)

Normally, a developer/programmer is personally responsible for:

288. Writing the code (e.g., a `.py` file).
289. Running/executing it.
290. Debugging when errors occur.
291. Writing test cases (e.g., using `pytest`) and running them to confirm the code works, before handing off to a client ("all test cases passed").
292. Using Git for commits and version control.
293. Making any requested enhancements by directly editing the code.

2. What changes with Claude Code

- With Claude Code, *you* describe the requirement and rules in plain English, and Claude Code becomes responsible for: writing the code, running it, testing it, analyzing files (Excel/CSV etc.), fixing errors, and enhancing the project on request.
- Your only two responsibilities become: **(1) grant access/approve actions when prompted, and (2) give clear prompts** describing what you want.
- Claude Code can be used in two ways: from the **desktop app**, or from the **terminal** (command prompt). In this session, the instructor primarily demonstrated the terminal version because the desktop version required a Git installation that wasn't ready yet.

3. Building a project from scratch — step by step

294. Create an empty project folder (e.g., "Claude demo" folder on the desktop).
295. Open Claude Code (terminal version in this demo) and point it at that folder as your working directory.
296. Give a clear, detailed prompt describing the full requirement in one go. Example used in the session:

"Create a simple Python project called student marks analyzer. Requirement: use a list of students with marks in English, maths, and science. Calculate total marks and average marks. If the average is 35 or above, the result is pass, otherwise fail. Grading: 90 and above A+, 75–89 A, 60–74 B, 35–59 C, below 35 fail. Print the report in the terminal. Save the report into report.txt. Keep the code simple for beginner students."

297. Claude Code will:

- Analyze the (empty) folder state.
- Ask for approval before creating files (e.g., "Do you want to create student_marks_analyzer.py? [y/n]").
- Ask for approval before running shell commands (e.g., running the Python file).

298. After approving each step, Claude Code creates the `.py` file **and** runs it itself, producing output directly in the terminal and saving a `report.txt` file — all without the user writing or manually executing any code.
299. You can open the generated `.py` file and the `report.txt` to verify the output (in the demo, the report correctly listed each student's marks, total, average, grade, and pass/fail status).

4. Enhancing the project (refactoring hardcoded data)

- Problem identified in the generated code: student data was **hardcoded** directly inside the Python file — not a good practice, since adding a new student would require manually editing the code file (via input/CSV would be better than hardcoding).
- Fix via prompt (no manual coding):

"Improve this project by moving student data into a separate CSV file called students.csv. Read from it. Also add basic error handling if marks are missing or invalid."

- Claude Code then:

300. Creates `students.csv` (asks for approval).
301. Modifies the Python file to read from the CSV instead of using hardcoded values (asks for approval to overwrite).
302. Re-runs the updated program to confirm it still works.

- Result: the `.py` file no longer contains any student data directly — all data now lives in `students.csv`, and future students can be added just by editing the CSV, not the code.

5. Automatic debugging demonstration

303. The instructor manually introduced bad data into `students.csv` (e.g., a missing English mark for one row, and a non-numeric value "ABC" in place of a math mark for another row).
304. Prompt given to Claude Code:

"Run the project, find the error, and explain the reason in simple English, and fix the code so that invalid records are skipped safely."

305. Claude Code ran the project, identified the bad rows, explained the issue, and modified the code so those specific invalid rows are safely skipped rather than crashing the whole program.
306. Verified in the output report: the two bad rows (row 8 — missing English mark; row 9 — invalid "ABC" math value) were skipped, while all valid rows were processed correctly.

- Key point: you do **not** need to tell Claude Code exactly what the bug is or how to fix it — a general instruction like "there's an error, please fix it" is enough for it to locate and resolve the issue itself.

6. Why Git matters with Claude Code

- The desktop version of Claude Code prompted for a Git installation when trying to select a project folder — Git enables version control (tracking changes, collaboration, and being able to revert to older versions of code).
- This matters because if Claude Code changes your code and something breaks, or you want to go back to an earlier version, Git/version history is how you recover it.
- For purely local/practice use, Git may not be strictly mandatory, but it becomes important for real, collaborative, or production projects.
- A participant also raised a valid caution: when Claude Code refactors existing code (especially in larger projects with multiple files/classes touched by multiple people), it can inadvertently break functionality by changing the flow — so manual review/testing and careful, scoped changes are still important, especially in shared codebases.

7. How to install and run Claude Code from the terminal (prerequisites)

307. Install **Node.js** (required because Claude Code is installed via NPM). Download the LTS Windows installer and install it normally.
308. Verify installation: run `node -v` in the terminal to confirm the Node.js version.
309. Verify NPM is available: check the NPM version.
310. Install Claude Code via NPM: `npm install @anthropic-ai/claude-code` (referred to in the session as installing "Anthropic Claude Code" via the NPM package).
311. Confirm installation: run `claude --version` to see the installed Claude Code version.
312. Once installed, you can run Claude Code from the terminal and use natural-language prompts exactly as demonstrated (create project, enhance project, debug project).

8. Model used by default

- The instructor noted that Claude Code, when run from the terminal in this demo, appeared to default to the **Sonnet** model (no explicit model selection prompt appeared). Participants confirmed that Sonnet is generally recommended for coding tasks, and that switching models (e.g., to a newer/updated version) can sometimes produce better code generation results, depending on the complexity of the task and the prompt's clarity.
- One participant shared a related real-world troubleshooting story: when working with a Spring Boot/Gradle project with company-specific framework libraries, Claude struggled due to version mismatches and inability to read certain internal JAR/plugin files — the instructor's advice was to explicitly tell Claude Code about the specific problem you're facing (e.g., "these are the versions I'm using, generate compatible code" or "analyze the code and detect version compatibility yourself") rather than expecting one perfect upfront prompt to solve everything — treat it as an iterative conversation.

Key takeaways

- Claude Code shifts nearly the entire traditional developer workflow (writing, running, testing, debugging, committing code) onto the AI — your job becomes describing requirements clearly and approving actions.
- A single well-structured, detailed prompt can generate a complete, working, beginner-level Python project — including calculations, grading logic, terminal output, and a saved report file.
- You can iteratively improve a project with follow-up prompts (e.g., "move hardcoded data into a CSV file") without writing any code yourself.
- Claude Code can find and fix bugs on its own when simply told "there's an error, please fix it" — you don't need to diagnose the bug yourself.
- Claude Code always asks for approval before creating/modifying files or running commands, giving you control over what actually executes.
- Git/version control matters when using Claude Code on real or shared projects, since automated refactoring can unintentionally break existing functionality — review changes and test carefully, especially in multi-developer codebases.
- To run Claude Code from the terminal, you need Node.js and NPM installed first, then install the Claude Code package via NPM and verify with `claude --version`.

Demo 36: Installing and Using Claude Code from the Terminal

This session is a from-scratch walkthrough of setting up Claude Code (the coding assistant CLI from Anthropic) on Windows, and using it to build, fix, and test a simple Python project called "Student Marks Analyzer."

What you'll learn

- What Claude Code is and why it changes how you write software
- How to install the prerequisites (Node.js, npm) and then Claude Code itself
- How to log in to Claude Code from the terminal
- How to write a good first prompt to generate a working project from scratch
- How to ask Claude Code to improve, refactor, and add error handling to existing code
- How to ask Claude Code to generate unit tests
- The difference between using Claude Code from the terminal versus the desktop app

Step-by-step notes

1. Understand what Claude Code does. As a developer, your normal job is writing code, testing it, debugging it, and enhancing it. Claude Code can do all of this for you — generate code, run it, find and fix bugs, and improve it. You are not required to write code by hand anymore; you just need to describe clearly what you want.

2. Install the prerequisites.

- Claude Code is distributed as a JavaScript/npm package, so you first need Node.js installed (this also gives you npm, the package installer).
- Download the Node.js Windows installer from nodejs.org and run it (Next → Accept terms → Next → Install). No special options are needed.
- Close and reopen your terminal/command prompt after installing so the new software is recognized.
- Verify the install by running `node -v` (should show a version number) and `npm -v` (should also show a version number). If these say "not recognized," Node.js isn't installed yet or the terminal needs to be reopened.

3. Install Claude Code.

- Open a command prompt inside the folder you want to work in (you can type `cmd` in the folder's address bar in File Explorer to open a terminal already pointed at that folder).
- Run: `npm install -g @anthropic-ai/claude-code`
- The `-g` flag means "install globally" so the `claude` command works from any folder.
- If you hit a "no space left on device" npm error, free up disk space and retry.
- Verify with `claude -v` (or `claude --version`) — it should print a version number once installed correctly.

4. Log in the first time.

- Type `claude` and press Enter to launch Claude Code in the terminal.
- The first time, it will ask you to choose a login method (Claude subscription account, or API/billing-based account) and open a browser link.
- Log in with your account in the browser; after a successful login you'll see a confirmation, and you can close that browser tab and return to the terminal — the terminal session is now authenticated. You will not be asked to log in again after this.
- No API key is required if you're using a Claude subscription login.

5. Create a project with a single prompt.

- Make sure your terminal's current folder is empty (or is the folder you want the project created in).
- Launch Claude Code with `claude`.
- Give a clear, detailed prompt describing exactly what you want. Example used in the video:

"Create a simple Python project called Student Marks Analyzer. Requirements: use a list of students with marks in English, Math, and Science. Calculate total marks and average marks. If average is 35 or above, result is pass, otherwise fail. Give grade: 90 and above A+, 75–89 A grade, 60–74 B grade, 35–59 C grade, below 35 fail. Print the report in the terminal. Save the same report in report.txt. Keep the code simple for beginner students."

- Claude Code will ask for permission before running shell commands or creating files — approve these prompts (type `1` / `yes`) to let it proceed.
- Once done, check the folder: Claude Code will have generated the Python file and the `report.txt` output file — you did not write any code yourself.

6. Improve the project — move data out of hardcoded code.

- A common beginner mistake is hardcoding data (like student names/marks) directly inside the Python file. This is bad practice because updating data means editing code every time.
- Ask Claude Code to fix this with a prompt like:

"Improve this project by moving student data into a separate CSV file called students.csv. Read marks from CSV and generate the same report. Also add basic error handling if marks are missing or invalid."

- Claude Code will create `students.csv`, modify the Python program to read from it, and add `try/except` error handling so that a student with missing/invalid marks (e.g., a missing English mark) is safely skipped instead of crashing the program.
- If Claude Code modifies a file that's open elsewhere, it will ask "Do you want to reload it?" — say yes to see the updated file.

7. Ask Claude Code to generate and run unit tests.

- Prompt example: "Add simple unit tests for the get_grade and get_result functions using pytest. Keep tests beginner friendly."

- Claude Code creates a `test_analyzer.py` file with test cases (e.g., checking that a score of 90 returns grade "A+").
- If pytest isn't installed, Claude Code will ask permission to run `pip install pytest` — approve it, and it installs and runs the tests automatically.

8. Terminal vs. desktop version.

- In the terminal, you can see every command Claude Code executes as it works (more transparency into the "flow").
- In the desktop app, the same actions happen but you don't see every underlying command as clearly.
- Both versions can do the same things — install packages, run code, fix bugs, write tests.

9. Connecting to GitHub (mentioned briefly).

- You can ask Claude Code directly in a prompt to connect to your GitHub account and push code — just provide your GitHub URL/credentials in the prompt, and it can save the repo location so it doesn't ask again each time.

Key takeaways

- Claude Code is a full coding assistant: it writes, runs, debugged, and tests code for you — your job is to describe requirements clearly in plain English.
- Before you can use Claude Code from the terminal, you need Node.js + npm installed, then install Claude Code globally via npm.
- First-time use requires a one-time browser-based login; after that the terminal remembers you.
- Detailed, specific prompts produce complete working projects in minutes — including data files, error handling, and tests.
- Moving hardcoded data into separate files (like CSV) and adding error handling are realistic, recommended practices — and Claude Code can apply them on request.
- Claude Code can generate unit tests (pytest) and install any needed packages itself, with your approval.
- The terminal version shows you every command being run; the desktop version hides more of that detail but does the same underlying work.

Demo 37: Building a Complete Student Fee Management System with Claude Code

This session builds a more realistic multi-file Python project (a student fee tracker using CSV files) entirely through natural-language prompts to Claude Code, showing how a project can be progressively enhanced — search feature, menu system, summaries, reports — without writing any code by hand.

What you'll learn

- How to describe a multi-file project requirement in one prompt and have Claude Code build it end-to-end
- How Claude Code can read data from CSV files, do calculations, and generate multiple output reports
- How to iteratively enhance an existing project (add search, add a menu, add summaries) using follow-up prompts
- How to ask Claude Code to explain a whole project, explain individual functions, or explain code line-by-line
- The general principle: describe what you want in plain English; you don't need to know how the code works

Step-by-step notes

1. Set up the project folder.

- Create a new, empty folder (e.g., "Student Payment Tracker") and open it as your working directory in the Claude Code desktop app (or select "open folder" to change the current working directory if a wrong project was already selected).

2. Write a detailed first prompt.

Example used:

"Create a beginner-friendly Python project called Student Fee Management System using only CSV files. Requirements: use students.csv for student details (student ID, name, mobile number, course, total fee); use payments.csv for payment details (payment ID, student ID, paid amount, payment date) — one student can have multiple payments. Calculate total fee, total paid amount, pending amount, and payment status. Payment status rules: Fully Paid if pending amount is zero; Partially Paid if paid amount > 0 and pending amount > 0; Not Paid if paid amount is zero. Show all student fee reports in the terminal. Save the full report into fee_report.txt. Save only pending-fee students into pending_fee_report.txt. Generate simple WhatsApp reminder message drafts for pending students and save into whatsapp_messages.txt. Add basic error handling for invalid amount values. Keep the code very simple for beginner students. Explain what files you created and how to run the project."

3. Let Claude Code build everything.

- Claude Code creates `students.csv` and `payments.csv` (sample data) and the main Python program (e.g., `fee_management.py`) — three files from one prompt.
- Ask Claude Code to run the program (or run it yourself from the terminal with `python fee_management.py`).
- Running it reads both CSVs, calculates each student's total fee, amount paid, pending amount, and status, then:
- Prints the full report to the terminal
- Saves it to `fee_report.txt`
- Saves a `pending_fee_report.txt` for only students with pending balances
- Generates `whatsapp_messages.txt` with ready-to-send reminder messages for students with pending fees (e.g., "Dear Priya Sharma, this is a gentle reminder... your pending course fee is ₹X... kindly clear your pending balance at the earliest...")
- Note: writing this program manually would normally take 3–4 hours; Claude Code generated it in about a minute.

4. Understand the project without reading code.

- You can ask Claude Code: "Explain the project files, file by file, in very simple English." It will describe each CSV and the Python file's purpose in plain language (e.g., "students.csv is like a register book in the school office... payments.csv is like a payment receipt book...").
- You can go further and ask: "Explain the important functions in fee_management.py in simple English." Claude Code explains each function (e.g., `read_students`, lines 24–43) in plain language, including what happens if the file is missing.
- You can also ask for a single-line summary of what an entire unknown file/program does — useful when inheriting someone else's code.
- The instructor's analogy: Claude (general chat) is like an MBBS general doctor who can answer generically, while Claude Code, focused on your actual codebase, is like a cardiologist — a specialist who gives precise, contextual answers about your specific code.

5. Add a search feature (enhancement prompt).

Example prompt:

"Add a search feature to this project. Requirement: allow user to search a student by mobile number. If the student exists, show name, course, total fee, total paid amount, pending amount, payment status. If the student does not exist, show 'student not found.' Keep the code simple and beginner friendly."

- Claude Code edits the existing Python file (not a new project), adds a `search_by_mobile` function, and the program now supports typing a mobile number to look up a student (typing `Q` quits the search loop).

6. Convert to a menu-based program.

Example prompt:

"Convert this project into a simple menu-based program: 1. Show all students fee report, 2. Search student by mobile number, 3. Show only pending fee student details, 4. Generate WhatsApp message drafts, 5. Exit."

- Claude Code rewrites the program into a clean menu loop; entering an invalid option (e.g., 7) is correctly rejected ("Invalid choice, enter a number between 1 and 5").

7. Add more features via prompts (each is a small, plain-English request):

- "Add course-wise summary: for every course show number of students, total fee amount, total collected amount, total pending amount."
- "Add date-wise collection report" — shows how much fee was collected on each date, with a grand total.
- Other prompts mentioned (not run live but shared in the instructor's document): add duplicate validation, add invalid-data handling, refactor/clean the code, generate pytest test cases, and generate a README file.

8. Generate a README.

Prompt: "Create a README file for your project. Include project title, purpose, files, features, how to run, sample input, sample output, and what students will learn — in very simple English."

- Claude Code writes a full `README.md` describing the project, files and their purposes, output files, and step-by-step run instructions.

Key takeaways

- A single, well-detailed prompt can generate an entire multi-file project (data files + program + logic).
- Claude Code can not only build software but also explain it — the whole project, individual functions, or line-by-line — making it easy to understand code you didn't write.
- Enhancements (search, menus, summaries, new reports) are just additional plain-English prompts on the same project; Claude Code edits existing files rather than starting over.
- Claude Code works with whatever programming language you specify; if unspecified, it may ask, or default to matching your existing project's language.
- Real productivity gain: work that would take days to explain/write manually (a menu-driven fee tracker with reports and reminders) was built and enhanced in minutes across a handful of prompts.
- The core skill required is "asking well" — being clear and specific about what you want; implementation is Claude Code's job.

Demo 38: Building and Publishing Web Applications with Claude Code

This session shows how to use Claude Code (desktop version) to build several different small web applications from scratch — a course landing page, an expense tracker, and a news app — using only natural-language prompts, including voice dictation, and discusses publishing apps to GitHub.

What you'll learn

- How to create a brand-new web app project folder and start a Claude Code session in it
- How to use voice input (dictation) to give prompts to Claude Code
- How to build a complete marketing landing page with reviews, syllabus, and contact info from one prompt
- How to add extra features to an existing web app (registration form, chatbot FAQ, WhatsApp chat button) with follow-up prompts
- How to build a colorful, interactive expense/income tracker web app
- How Claude Code can connect to real services (payment gateways, Firebase database, Gmail) when given credentials/access
- The idea of "spec-driven" development for larger, enterprise-scale projects

Step-by-step notes

1. Start a new project folder.

- Create a new empty folder (e.g., "XYZ") on the desktop, open it in Claude Code desktop as the working directory ("select folder" → "trust this folder").

2. Use voice input for prompts.

- Claude Code's desktop interface supports a microphone/dictation button. You may need to allow microphone access and hold the record button (press-and-hold to record) rather than a toggle click.

3. Build a landing page with one detailed prompt.

Example (dictated/typed):

"I want to create a landing page for my new Python batch starting July 2nd at 7 a.m. Add sample reviews, syllabus, videos from our YouTube channel, and contact information."

- Claude Code generates a complete static landing page (HTML) with sections for videos, reviews, syllabus, contact, and an "Enroll Now" call-to-action — including realistic-looking sample content (student counts, ratings, testimonials) based on context it inferred.
- The generated page opens directly in the browser from the project folder (it's a static site — mainly `index.html`).

4. Add more features with follow-up prompts.

- You can keep adding to the same project: "Add a registration form linked to a Google Form," "add a chatbot for FAQ," "add a WhatsApp chat button."
- Claude Code reads the existing `index.html`, determines what needs to change, and edits the file in one pass adding all three features (form section, FAQ chatbot widget, WhatsApp floating button) plus needed styles/scripts.
- Each time it makes changes, you can see how many lines were added/changed (shown as +/- counts) — useful for understanding the scope of an edit.
- The resulting chatbot can answer basic FAQ-style questions about the course (eligibility, mode of learning, how to enroll) using only the information present in the page — it's not a live/dynamic backend, just a simple scripted assistant based on the static content.

5. Discussion: spec-driven ("spec coding") vs. prompt-by-prompt for large projects.

- For small apps, prompting step-by-step is fine. For large, enterprise-level applications with many files, it's better to first generate a design "spec" document (via Claude or ChatGPT) and keep it updated — this preserves institutional knowledge so you (or someone else) can understand the system months later without digging through code.
- Claude also has a "team mode" concept where it can simulate multiple roles (e.g., designer, developer) collaborating.

6. Build a colorful expense tracker web app.

Prompt example:

"I want to create a web application to track all my expenses and income, with a colorful dashboard, and facility to add data and get reports and display summaries."

- Claude Code builds a single-file web app with: net balance, total income, total expenses, savings rate, an expense-by-category breakdown (e.g., housing, food, transport), transaction history, and a form to add new income/expense entries (with category selection). Adding a transaction updates the dashboard dynamically.

7. Real-world integrations mentioned (as capabilities, discussed with the group):

- Payment gateways: you can ask Claude Code to integrate a payment gateway (e.g., Razorpay) by providing your account credentials/API key — it can write the necessary JavaScript integration code.
- Firebase (Google's database): a line or two of instruction can have Claude Code wire up Firebase so that data persists even after closing/reopening the app (rather than being lost on refresh), using Firebase's generous free tier.
- Django-model-based backends: one participant described giving Claude Code an existing Django model and asking it to build a web page from it — Claude Code generated both backend and frontend code including URL routing.
- Gmail integration: one participant described asking Claude Code to read through the last month of transaction emails in Gmail, "bucketize" them by category (UPI, food bills, etc.), and present the results in a dashboard.

8. Build a colorful news app with one line.

Prompt example: "Colorful news app — with cricket, politics, cinema, and a special page for football." A working news-style app UI (with sections/categories) is generated from this single, short prompt. (Note: in the demo, the news data itself needed further prompting since it wasn't dynamically pulling live news.)

9. Publishing to GitHub (introduced, detailed in Demo 39).

- You need a GitHub account. Claude Code can publish an app there and provide a publicly accessible URL — a full walkthrough is given in Demo 39.
- Publishing this way is inexpensive compared to traditional agency costs (domain + hosting can cost as little as a couple thousand rupees, versus thousands of rupees and days/weeks of wait when using an agency).

Key takeaways

- Claude Code isn't limited to backend scripts — it builds full front-end web apps, including polished landing pages, dashboards, and interactive tools, from single natural-language prompts.
- You can dictate prompts by voice instead of typing.
- Iteratively adding features (forms, chatbots, buttons, integrations) works the same way as the earlier CLI demos: describe the addition, and Claude Code edits the existing files.
- Claude Code can integrate with real external services — payment gateways, databases (Firebase), email (Gmail) — as long as you provide the necessary access/credentials.
- For large/enterprise applications, consider having Claude generate and maintain a design "spec" document rather than relying purely on scattered prompts, so the system remains understandable over time.
- Everyone was encouraged to build and publish at least one small application to GitHub as homework.

Demo 39: Understanding Compounding, and Building/Publishing Compound-Interest Calculators with Claude Code

This session combines a personal-finance lesson on the power of compounding with a hands-on demo of building two web-based compound interest/savings calculators using Claude Code, then publishing them live to GitHub Pages with a public URL.

What you'll learn

- The financial concept of "pay yourself first" and why saving consistently (even small amounts) matters more than the amount saved
- How compounding accelerates value growth over long time periods, and why time invested matters more than the amount invested
- How to build a "daily saver" compounding calculator web app using a plain-English (even dictated) prompt
- How to have Claude Code push a project to GitHub and deploy it as a live, publicly accessible web page
- How to build a second, more flexible "monthly investment" compounding calculator app and use it to compare investment scenarios

Step-by-step notes

1. The financial mindset: save first, then spend.

- Recommended discipline: save at least a small amount every single day (even ₹1), making saving a daily habit rather than a monthly afterthought.
- When you receive salary, pay yourself first — separate at least 10% for savings/investment before spending on expenses. If you spend everything and save only what's left over (if anything), you're effectively working only to pay bills — described in the video as being "a slave" to your expenses.

2. The three-step wealth cycle taught:

313. **Save** — separate at least 10% of income for yourself first.
314. **Invest** — put the saved amount to work (SIP, fixed deposit, mutual funds, etc.) so it generates returns; idle savings are not real wealth.
315. **Reinvest returns** — do not spend the returns/interest you earn; reinvest them so they also start generating further returns. This is the actual mechanism of compounding (each year's returns become new "principal" that itself earns returns, creating exponential — not linear — growth).

3. Illustration of compounding growth (mentally, before building the app).

- Example: ₹10 lakh invested at 12% annual return, left untouched (returns reinvested each year) roughly doubles the count of "return units" every year in the analogy used (doubling pattern: 1, 2, 4, 8, 16 units growing to over 1,000 after ~10 years, and over a crore-scale multiplier after ~30 years).

- Key point: compounding looks slow/negligible in the early years but produces dramatic, non-linear results if continued for a long time (illustrated with Warren Buffett, who made the vast majority of his wealth after age 60, having started investing at age 10).

4. Build the "Daily Saver" web app.

- Create a new folder (e.g., "Daily Saver") and open a new Claude Code session pointed at it.
- Example prompt (can be dictated):

"I want to create a web application. My requirement is: every day I'm saving some amount like ₹1, ₹5, or ₹10. Every month (or every 2–3 months) that saved amount is invested. The interest rate and duration should be customizable. I want to see how much value I'll get after 20 or 30 years. I want a daily saver compounding calculator app."

- Claude Code generates a single `index.html` file implementing the calculator directly in the project folder (not as a throwaway "artifact" — it's saved to disk).
- Using the app: input a daily saving amount (e.g., ₹5,000/day → ₹1.5 lakh/month), a yearly compounding frequency, a 12% interest rate, and a duration (e.g., 30 years) — the calculator computes the final corpus (e.g., ₹46.45 crore after 30 years growing to ₹83 cr at 35 years, ₹147 cr at 40 years, ₹261 cr at 45 years in the example run), illustrating the accelerating effect of extra years.
- With a modest ₹100/day saved every 6 months for 30 years, the app showed the corpus growing to nearly ₹1 crore.

5. Publish the app to GitHub.

- Prerequisites needed on your system: Git, and GitHub CLI (gh). If GitHub CLI isn't installed, Claude Code can install it for you when asked.
- In Claude Code, simply prompt: "Push this app to my GitHub. Deploy this application and provide a public URL to access it."
- Claude Code runs the necessary commands (`git init`/gh commands), creates a local Git repository, creates the corresponding GitHub repository, pushes the code, enables GitHub Pages, and returns a live public URL you can open and share.
- If you're not logged in to GitHub CLI, Claude Code will prompt you to log in via a browser link the first time; after that it remembers your login.
- Note: GitHub Pages only supports static (client-side) applications/pages — for anything needing a real backend/database you'd need different hosting, but static apps like this calculator work fine.

6. Build a second, more flexible "Monthly Investment" app for scenario comparison.

- Create another new folder/session ("Monthly Investment App"), with a similarly dictated prompt asking for: an initial lump-sum input, a monthly/quarterly/yearly contribution amount, an interest rate, a compounding period (yearly/monthly/etc.), and a duration — producing a colorful, easy-to-understand calculator.

7. Use the app to compare real investment scenarios (illustrating "duration matters more than amount"):

- **Scenario A:** ₹10 lakh invested once, left for 40 years at 12% → final value ≈ ₹9.31 crore.
- **Scenario B:** ₹20 lakh invested (double the amount) but only for 20 years → final value ≈ ₹1.93 crore — dramatically less than Scenario A, despite investing twice as much money, because the time horizon was halved.
- **Scenario C:** ₹40 lakh invested but only for 10 years → final value ≈ ₹1.24 crore — even less, despite investing four times the original amount.
- **Conclusion drawn:** how much you invest matters far less than how long you stay invested. Time in the market compounds returns; a shorter duration severely limits the compounding effect, regardless of the principal amount.
- A second comparison used recurring monthly contributions (not lump sum): ₹1,000/month for 40 years yielded ≈ ₹92.05 lakh (total contributed: only ₹4.8 lakh — the rest is compounded interest); the same ₹1,000/month contributed for only 20 years yielded only ≈ ₹17 lakh, even though eventually the same monthly amount was used — again showing duration dominates.
- Another comparison (₹4,000/month for only 10 years, started later in life) versus ₹1,000/month for 40 years, starting immediately, reinforced the same lesson: starting early and staying consistent — even with a smaller amount — outperforms starting later with a larger amount.

8. Publish the second app as well.

- Same process: ask Claude Code to push it to GitHub, deploy it via GitHub Pages, and provide the public URL.

Key takeaways

- "Pay yourself first": save a fixed percentage (recommend at least 10%) of income before spending, and make saving a daily habit, not a monthly one.
- The wealth cycle is: Save → Invest → Reinvest returns (never "eat" your investment returns — reinvesting them is what creates exponential compounding).
- Compounding grows slowly at first but accelerates dramatically over long time horizons — the biggest gains typically happen in the later years, not the early ones.
- Time invested matters far more than the amount invested; starting early with a small amount beats starting later with a much larger amount.
- Claude Code can build fully functional, useful financial calculator web apps directly from a single descriptive (even dictated) prompt.
- Claude Code can push a project straight from your local folder to a new GitHub repository and deploy it live via GitHub Pages, giving you a shareable public URL — all through simple prompts, provided Git and GitHub CLI are available (and it can install GitHub CLI for you if missing).

Demo 40: Using Claude Code Inside VS Code, and Understanding Plan Mode

This session shows developers (and non-developers) how to install and use the Claude Code extension inside Visual Studio Code (VS Code), how that compares to the desktop app, and explains what "Plan Mode" is and why it helps save cost/tokens on larger projects.

What you'll learn

- How to install VS Code and the Claude Code extension inside it
- How to enable Claude Code for a specific project/workspace in VS Code
- The interface differences between Claude Code in VS Code versus the desktop app (model switching, effort level, auto-edit modes)
- Why developers benefit from using Claude Code inside an IDE (ability to manually tweak code)
- How to push VS Code Claude Code projects to GitHub and redeploy after manual edits
- What Plan Mode is, how it differs from Ask Mode and normal (auto) mode, and why it can reduce token/cost usage
- Brief mention of token/usage limits and switching between models (e.g., Claude vs. Codex)

Step-by-step notes

1. Install Visual Studio Code (VS Code).

- VS Code is a free code editor from Microsoft. Download it by searching "Download Visual Studio Code" and choosing the Windows installer.
- Run the installer (Next → Next → optionally check "create desktop icon" → Install → Finish).
- Context given: VS Code has become the dominant general-purpose code editor, replacing older language-specific editors like Eclipse (Java) or PyCharm (Python) for many developers, though IntelliJ IDEA remains popular specifically for Java.

2. Open a project folder in VS Code.

- Use File → Open Folder, and create/select an empty project folder (e.g., "VS Code Project 3").

3. Install the Claude Code extension.

- In VS Code's left sidebar, click the Extensions icon.
- Search for "Claude Code" and install the one published by **Anthropic** (there may be multiple similarly named entries — pick the official Anthropic one).
- After installing, you must also **enable it for your specific workspace**: go back to the Claude Code extension entry and click "Enable Workspace," then confirm "Trust the folder and continue."
- Once enabled, a Claude Code icon appears in the VS Code sidebar (similar to how GitHub Copilot's icon appears) — click it to open the Claude Code panel/interface inside VS Code.

4. Using Claude Code inside VS Code.

- The interface is very similar to the desktop app: you type a prompt describing what you want (e.g., "I want to create a web application to find the compound value... daily saver / interest calculator, etc.") and Claude Code builds it, same as in the terminal or desktop versions.
- If your project is empty/new, Claude Code may ask which language/tech stack you want (e.g., HTML/CSS/JS or something else); for an existing project, it infers the language automatically from context.
- Special options available via typing `/` in the prompt box include: switch model (default is Sonnet), set effort level (high/medium/low), enable/disable "thinking," attach memory, clear conversation, and view usage/token information.
- **Edit permission modes:** "Ask before edits" (Claude Code asks permission every time before making a change) vs. **Auto mode** (it makes changes without asking each time — faster but less supervised). There's also **Plan Mode** (explained below).
- High "thinking effort" and "thinking mode" consume more tokens and take longer; for straightforward tasks, lowering effort/disabling thinking speeds things up and saves tokens.

5. Why use Claude Code in an IDE like VS Code (developer advantage).

- If you are a developer/coder, having the code open directly in an IDE lets you manually open files and make small tweaks yourself, instead of always asking Claude Code to make every change — useful for quick, minor edits.
- For non-developers, this isn't necessary — you can just keep asking Claude Code to make all changes for you, exactly as in the desktop version.
- Example given: after Claude Code generated a compound interest calculator's `index.html`, the instructor manually edited the page's `<h1>` header text directly in VS Code, then asked Claude Code to "redeploy" — Claude Code detected the diff, described the change ("title updated to ... compound interest calculator"), and pushed it to the already-existing GitHub repo/live deployment.

6. Publishing/deploying from VS Code's Claude Code (same idea as Demo 39).

- Ask Claude Code (in the VS Code panel) to "push this into my GitHub account, deploy the application, and provide a public URL" — same process as the terminal/desktop versions, requiring Git and GitHub CLI (`gh`) to be available (Claude Code can install `gh` if missing, with your approval).
- After deployment, you get both a GitHub repository link and a live public URL for the deployed app.

7. Token usage note.

- Any operation where Claude Code performs work (updating code, running commands) consumes tokens/usage from your plan. Simply opening/viewing a file manually and editing it yourself (without asking Claude Code to do it) does not consume Claude Code tokens.
- Paid plans have tiers (e.g., $20 plan, up to $100 or $200 plans) and you can also buy additional API credits if you exceed your plan's limit. In a company setting, usage may be capped and you may need to wait for a reset if a shared limit is hit.

- You can switch to a different coding agent/model (e.g., Codex) if one hits its usage limit — this is separate from Claude's own model-switching option.

8. Plan Mode explained.

- Concept: like a building project needs an architect to draw up a full plan before construction starts, Plan Mode makes Claude Code first explore the codebase and produce a written plan (as a Markdown file) *before* making any actual code changes.
- To use it: select **Plan Mode** from the mode options before giving your prompt describing the application you want.
- Claude Code may first ask clarifying questions (e.g., "which tech do you want — HTML/CSS/JS, React, or other?"; "should the app look clean and minimal, dark theme, or colorful/financial dashboard style?").
- It then produces a plan document (e.g., "Daily Saver Compounding Calculator Plan") listing the context, tech stack, files to create, and UI layout — without yet writing the actual application code.
- Once you approve the plan ("accept this plan"), Claude Code proceeds to build the app based on that plan.
- **Why this matters / the cost-saving advantage:** Planning is the expensive, high-value step best done with a strong (costly) model like Opus. Once a solid plan exists, the actual implementation can be handed to a cheaper/faster model (e.g., Sonnet, Haiku, or even a different tool like Codex or another lower-cost LLM) to execute — saving overall token cost, similar to how an expensive architect designs a plan that any competent builder can then execute.
- Plan Mode is distinct from **Ask Mode**: Ask Mode is a pure question-and-answer/chat mode used to understand an existing project (e.g., "explain what this project does, where is the entry point, what are the steps") without creating any planning document or making changes. Ask Mode uses the fewest tokens; Plan Mode uses more (since it explores and writes a plan); full auto/implementation mode uses the most tokens.
- A participant also noted that Claude Code in VS Code automatically summarizes/compresses the conversation ("shrinks" the context) when a chat session's context window starts filling up during a long session — an automatic memory-optimization feature not present in GitHub Copilot.

9. Brief comparison: Claude Code vs. GitHub Copilot.

- Both are available as VS Code extensions and let you switch between different underlying models.
- Claude Code was described in the discussion as having more advanced features (e.g., automatic context/conversation summarization, more slash commands) compared to GitHub Copilot, though companies may standardize on Copilot for internal use.

Key takeaways

- Claude Code works the same way inside VS Code as in the terminal or desktop app — you still just describe what you want in plain English — but VS Code adds the ability to manually view and edit generated code yourself, which is useful if you have coding skills.

- Setting up Claude Code in VS Code requires two enable steps: installing the Anthropic Claude Code extension, and then enabling it specifically for your open workspace/folder.
- You can freely mix manual edits (as a developer) with Claude Code–driven edits in the same project, and ask Claude Code to redeploy after manual changes.
- Plan Mode has Claude Code produce a written implementation plan before writing any code — useful for bigger projects, for saving tokens/cost (plan with a strong model, implement with a cheaper one), and for getting clarity before committing to an approach.
- Ask Mode is for understanding/questioning an existing codebase without making any changes; Plan Mode is for designing a new build before implementing it; normal/auto mode is for direct implementation.
- Token/usage consumption only happens when Claude Code actively performs work (edits, runs commands) — manual edits you make yourself don't cost Claude Code tokens.

Demo 41: Building an Expense Tracker App with Claude Code and a Firebase Database

This session shows how to move beyond simple single-page tools by giving a Claude Code-built app real, persistent storage. The instructor sets up a free Firebase (Google) NoSQL database from scratch and connects it to a colorful expense tracker web app built entirely by prompting Claude.

What you'll learn

- Why saving money and tracking expenses matters (the instructor's "pay yourself first" philosophy) and how that motivates building a personal expense tracker app
- The difference between a relational (SQL) database and a NoSQL database, and why NoSQL (Firestore) is commonly used for these quick apps
- How to create a Firebase project, register a web app, and create a Firestore (NoSQL) database
- How to grab Firebase configuration (API key, project ID, etc.) and hand it to Claude Code in a prompt so it can wire up the database
- How to build a full app (HTML/CSS/JavaScript, charts, persistent storage) with a single natural-language prompt in Claude Code

Step-by-step notes

1. The "why": savings mindset (context for the app)

The instructor spends the opening explaining a personal-finance rule before getting technical: save at least a small amount every day, and whenever you get income (salary, etc.) set aside at least 10% for yourself first ("pay yourself first") before paying any expenses. The remaining 90% is what you use to live on. Because money must be tracked carefully to stick to the 90%, you need an **expense tracker** — and since the class already knows Claude Code, they can build their own instead of using a Play Store app.

2. Set up a Firebase database

316. Go to `firebase.google.com` and sign in with a Gmail account (no installation needed — it's fully cloud-based, similar to how n8n.io works online).
317. Click **Go to console**, then **Create a new Firebase project**. Give it a name (example used: "expense tracker seven").
318. You can enable or disable Gemini and Google Analytics for the project — neither is required for this app.
319. Once the project is created, register an **app** inside it. Choose **Web app**, give it a nickname, and skip enabling Firebase Hosting (the instructor plans to host on GitHub instead).
320. Firebase shows a script snippet with configuration values (API key, project ID, auth domain, etc.). Copy this — it will be pasted into the Claude Code prompt later.
321. In the left panel, go to **Build > Firestore Database** (a NoSQL database — no tables needed, just collections/documents/objects, similar to MongoDB).

322. Click **Create database**. Choose **Standard edition**, keep the default database ID, and pick a server **location** close to you (e.g., Mumbai for India, for faster response times).
323. Choose **Start in test mode** (allows open read/write for 30 days — good enough for learning; production mode is covered in a later demo).
324. Note: on the free tier you can typically only have a couple of Firestore databases active — if you hit a "billing required" or "database already exists" error, you may need to delete an old test database first.

3. Build the app with Claude Code

325. Open VS Code, use **File > Open Folder**, and create/select a new empty project folder (e.g., "expense tracker seven").
326. Install/enable the Claude Code extension for that workspace ("Trust folder and continue", then "Enable workspace") — this is the one extra step needed for the VS Code version versus the desktop Claude Code app; everything else works the same.
327. Open a new Claude Code session and give a prompt like:

"I want to create a colorful expense tracker with HTML, CSS, and JavaScript. I need day/week/month-wise charts. Use Firebase to store the data, and use the Firebase compat SDK. Here are the database configurations: [paste the Firebase config]."

328. Claude Code (running in **Auto mode**, which mostly proceeds without asking for approval each step) generates `index.html`, `style.css`, and `app.js` that talk to Firestore using the Firebase compat SDK.
329. Open `index.html` directly in the browser (Chrome) to test — no server needed for this basic version.
330. Add expenses (title, amount, category, date) through the app's form; they get saved to Firestore. Add a second expense and confirm both appear.

Key takeaways

- Building a personal finance habit (save daily, pay yourself 10% first, track the remaining 90%) is the motivation behind this whole app-building exercise.
- Firebase is a free, cloud-hosted database from Google — no local installation required, similar in spirit to how n8n is cloud-hosted.
- Firestore is a NoSQL database: data is organized as collections (~database), documents (~table), and objects/fields (~rows) — no schema or table creation needed.
- The full workflow is: create Firebase project → register app → create database → copy config → paste into a Claude Code prompt describing the app you want → Claude builds the whole working app.
- Test mode databases are meant for practice only and expire after 30 days (this becomes relevant again in Demo 44).

Demo 42: Adding Google Sign-In (Per-User Data) and Deploying to GitHub Pages

Building on the expense tracker from Demo 41 (which stored everyone's expenses in one shared list), this session upgrades the app to a "real" product: every user logs in with their own Google account and only sees their own private expenses, then the finished app is deployed publicly via GitHub Pages.

What you'll learn

- Why a shared, un-authenticated app isn't good enough for real use (privacy — everyone's data mixed together)
- How to enable Google Sign-In (SSO/authentication) inside Firebase
- How to tell Claude Code (in the prompt) to make the app track expenses per logged-in user
- How to configure Firebase's "Authorized domains" so sign-in works after deployment
- How to deploy the finished app to GitHub Pages and get a public URL
- Why the exact wording of your prompt to Claude matters, and why **Plan Mode** is useful for reviewing what will be built before Claude generates code

Step-by-step notes

1. Set up a new Firebase project for the private version

331. Create a new Firebase project (e.g., "expense tracker private two").
332. Register a new **web app** and copy its configuration (same process as Demo 41).
333. Create a Firestore database in **test mode** again (production mode covered later).

2. Enable Google authentication in Firebase

334. In the Firebase console, go to **Build > Authentication > Get started**.
335. Choose **Google** as a sign-in provider (Google is disabled by default — you must enable it).
336. Set a support email for the project and save. (Other providers like Facebook/Microsoft/phone number are also possible but weren't set up here.)

3. Update the Claude Code prompt for per-user data

Ask Claude to build essentially the same app as before, but add:

"...use Firebase Google SSO for authentication also. Individual user expenses should be tracked based on the logged-in user — i.e., user-wise expenses should be shown."

Claude generates the app so every expense record is tied to the logged-in user's ID, and each user only ever sees their own data.

4. Run and test locally

337. Because Google Sign-In requires a real server context (not just double-clicking the HTML file), Claude runs the app on a local HTTP server (e.g., `localhost:<port>`). If a port is already in use by another running app, change the port.
338. Open the app in the browser — it will show "Sign in with Google." After signing in, expenses can be added and are private to that Google account.
339. Sign out and sign back in with a **different** Google account to prove expenses are isolated per user — the new account starts with zero expenses, and switching back to the original account restores the original data.
340. In Firebase's Firestore data view, you can see a `users` collection with each user's `expenses` sub-collection, confirming the per-user structure.

5. Deploy to GitHub and fix the authorized-domain error

341. In Claude Code, simply ask: "Push this application to GitHub, deploy it, and provide a public URL."
342. Claude Code pushes the code, enables **GitHub Pages**, and returns a public URL (e.g., `django-guga.github.io/expense-tracker-private-2`).
343. Opening the public URL and clicking "Sign in with Google" initially fails with: *"This domain is not authorized for OAuth operations for your Firebase project."*
344. Fix: In Firebase, go to **Authentication > Settings > Authorized domains**, and **Add domain** — add the GitHub Pages domain (e.g., `django-guga.github.io`). After this, Google Sign-In works from the public URL too, and previously saved expenses reload correctly since they're all tied to the same Firestore database.

6. Prompting philosophy

- Whatever functionality you want (e.g., "family head can see everyone's expenses," "users can add their own custom expense categories"), you must describe it clearly in the prompt — Claude will build exactly what you ask, and if the result doesn't match your intent, you clarify and iterate.
- **Plan Mode** is recommended for bigger apps: Claude proposes a plan (fields, structure, etc.) for your approval before writing code, which also saves tokens.

Key takeaways

- A production-style app needs authentication (login) so that each user's data is private — Google/Gmail sign-in via Firebase Authentication is the simplest way to add this.
- Enabling a sign-in provider in Firebase (e.g., Google) is a manual one-time console step; Claude Code then wires the app to use it based on your prompt.
- After deploying to a new domain (GitHub Pages, or later a custom domain), you must add that domain to Firebase's **Authorized domains** list or authentication will fail.
- The whole cycle — build app, add database, add authentication, push to GitHub, deploy, get a public URL — can be done through prompting alone, without manually writing code.

- One custom domain example mentioned by a student cost around 560–625 rupees for a year via a different registrar; the instructor plans to demo buying and linking a custom domain the following day.

Demo 43: Buying a Custom Domain and Pointing It to a GitHub Pages App

This session completes the "own your app" journey by showing how to buy a real custom domain name and connect it to the app already hosted on GitHub Pages, including fixing the Google Sign-In authorization for the new domain.

What you'll learn

- How to buy a domain name cheaply using Namecheap
- Why simple domain "redirects" don't work for GitHub Pages, and why you need DNS **A records** instead
- How to configure DNS A records in Namecheap so a custom domain points to GitHub's servers
- How to add a custom domain inside GitHub Pages repository settings
- How to add the new custom domain to Firebase's authorized domains so login keeps working
- That this end-to-end flow (domain → DNS → GitHub Pages → Firebase auth) completes the full "idea to live app with your own domain" pipeline

Step-by-step notes

1. Buy a domain on Namecheap

345. Ask an AI assistant (ChatGPT/Claude) for domain registrar recommendations — Namecheap is suggested as the best balance of price, dashboard usability, privacy, and support for beginners (other options mentioned: Cloudflare Registrar, Porkbun, GoDaddy, Hostinger).
346. Go to `namecheap.com`, sign up, and sign in.
347. Use **Domain Name Search** to search for your desired name (e.g., "life skills with durga"). Prices vary a lot by extension — `.com` can be expensive (~$998 in the example search for a taken variant), while alternatives like `.online` were found for as low as ~89–113 rupees for the first year.
348. Add the cheap domain to cart, **turn off Auto-Renewal** (important — don't let it silently renew), confirm the order, and pay (e.g., via credit card; the example totaled about $3.08).
349. After purchase, the domain is reserved exclusively for you for one year (renewal required after that, with roughly a 30-day grace period before it's released to others).

2. Why redirect doesn't work — use DNS instead

- Namecheap has a simple **domain redirect** feature (forward your domain to a target URL), but GitHub Pages does **not** support being pointed to via redirect — doing so causes a "too many redirects" loop error.
- The correct approach for GitHub Pages is to configure **DNS records (A records)** that point directly to GitHub's IP addresses.

3. Configure DNS in Namecheap

350. In Namecheap, go to **Account Dashboard > Domain List > Manage** for your domain.
351. If a "Redirect Domain" entry already exists, **delete it** first (it conflicts with GitHub Pages DNS-based routing).
352. Go to the **Advanced DNS** tab and click **Add New Record**.
353. Add multiple **A records** (Host = `@`) pointing to each of GitHub Pages' fixed IP addresses (the instructor got these by asking an AI assistant for "GitHub Pages DNS records"; they are fixed, well-known GitHub IPs).
354. Add a **CNAME record** for `www` pointing to your GitHub Pages site (e.g., `yourusername.github.io`).
355. Save all changes.

4. Configure the custom domain in GitHub Pages

356. Go to your GitHub repository's **Settings > Pages**.
357. Under **Custom domain**, enter your new domain (e.g., `lifeskillswithdurga.online`) — make sure there's no trailing slash, or it can cause errors.
358. Save. GitHub will show a **DNS check** status; once it says "successful," the custom domain is properly linked (this can take a few minutes up to 24 hours, though it was fast in the demo).
359. Do not enable "Enforce HTTPS" until the DNS check is confirmed successful.

5. Fix Google Sign-In for the new domain

- After the domain works for basic page loading, clicking "Sign in with Google" fails with the same *"this domain is not authorized"* error seen before.
- Fix: In Firebase, go to **Authentication > Settings > Authorized domains > Add domain**, and add the new custom domain. Existing authorized domains (like `localhost` and the old GitHub Pages domain) don't need to be removed.
- After this, opening the custom domain, signing in with Google, and seeing your previously saved expenses all work correctly.

6. Extra Q&A points covered

- If something doesn't work, the instructor's troubleshooting method is: take screenshots of the errors/settings and ask an AI assistant (ChatGPT/Claude) to diagnose and give step-by-step fixes — this is how the DNS issue above was actually solved.
- Test-mode Firestore databases: there was discussion about whether test-mode data really gets deleted after 30 days; the instructor wasn't fully certain and said he'd verify (this is resolved with more detail on production mode in Demo 44).
- A student asked if this DNS-connecting process could be automated by Claude directly (rather than manually via the Namecheap/GitHub/Firebase consoles) — the instructor agreed it's technically possible but noted giving Claude your login credentials or payment details to do it automatically would be risky, so manual configuration is recommended for now.

Key takeaways

- Buying a custom domain is cheap (often under 100–300 rupees/year for non-`.com` extensions) via Namecheap.
- GitHub Pages requires **DNS A/CNAME records**, not simple domain redirects — redirects cause a "too many redirects" error.
- The complete pipeline is: buy domain (Namecheap) → configure DNS A records to GitHub's IPs → set custom domain in GitHub Pages repo settings → add the domain to Firebase's Authorized Domains list.
- Always disable auto-renewal on a domain you bought just for testing/learning.
- When stuck, screenshotting the problem and asking an AI assistant for the exact fix is a fast, practical debugging approach.

Demo 44: Firebase Test Mode vs Production Mode, Security Rules, and Prioritizing Your Time

This session explains the real difference between Firebase's test mode and production mode (including database security rules with a "super admin" example), then builds a Priority-Based To-Do List app using production mode. The instructor also gives an extended talk on time management — focusing only on high-value activities and delegating or eliminating low-value ones.

What you'll learn

- The practical differences between Firebase test mode and production mode
- What "security rules" are in Firestore and how to define custom access rules (e.g., a super admin role) via a Claude prompt
- How to build and test a priority-based to-do list app with role-based permissions
- A productivity framework: focus only on your highest-result activities, delegate/eliminate the rest

Step-by-step notes

1. Test mode vs production mode (concept)

Aspect	Test mode	Production mode
Purpose	Practice / quick testing / learning	Real users, real live app
Security	Weak — database open to many users	Strong — access controlled by rules
Data lifespan	Temporary (about 30 days)	Long-term / permanent
Example	Anyone can add/update to-do records	Only the logged-in user or an admin can update their own records

- The instructor asked an AI assistant to explain the two modes side-by-side for the class, and shared that summary directly (as reproduced above).
- The main reasons to move to production mode: (1) you want data to persist long-term, and (2) you need real security — e.g., a "super admin" who can see everyone's data, while normal users can only see their own.

2. Example rule scenarios

- A **family expense tracker** where the family head can view all members' expenses, but each member only sees their own — this is implemented as a security **rule** in the production database.

- A **priority to-do list** where a defined **super admin** can view (and optionally edit) everyone's tasks, while normal users can only see and manage their own tasks. Role-based rules like this are defined by describing them in the Claude prompt; Claude generates the corresponding Firestore security rules.

3. Productivity mini-lesson (context before building the next app)

- Don't try to do every activity — focus only on **highest-result / highest-priority activities**; delegate or eliminate low-value activities.
- Example: if your goal is "earn 100 crores," don't spend your valuable time on ₹10-level tasks (e.g., personally fetching milk/groceries) — that time should go toward big-impact work.
- Illustrative math: a 2-minute daily low-value task adds up to about 12 hours/year, or 120 hours over 10 years — time that could be spent learning a new skill or creating something valuable.
- Example used: Mark Zuckerberg wearing the same T-shirt style to avoid wasting decision-time on clothing selection — small recurring low-value decisions add up over years.
- The instructor's rule of thumb: rare/occasional low-value tasks are fine to do yourself, but *repeated* low-value tasks should be delegated (e.g., hiring help) so you can reclaim time for high-value work.
- This reasoning motivates building a **priority-based to-do/task tracker app** to actively track and manage your priorities.

4. Build the Priority-Based Task Tracker app (in production mode)

360. Create a new Firebase project (deleting an old one if you're on the free tier limit of ~2 projects).
361. Create the Firestore database in **production mode** this time (instead of test mode) — same steps as before (Standard edition, default database ID, nearest location like Mumbai) but selecting "production mode."
362. Enable **Google Sign-In authentication** as before (Authentication > Get started > Google > Enable).
363. In Claude Code, open a new project folder, enable the Claude Code workspace extension, and start a new session.
364. Prompt Claude with something like:

"I want to create a priority-based task tracker/to-do list app with HTML, CSS, and JavaScript. Require priority-wise charts; users can see activities by priority. Use Firebase to store data and the Firebase compat SDK, and use Firebase Google SSO authentication. Individual user tasks should be tracked by logged-in user. There should be a super admin (my email) who can see all users' tasks and change their priority. Since we're using Firebase production mode, also provide the Firestore security rules to update in Firebase."

365. Claude generates the app **and** a `firestore.rules` file containing the access rules (e.g., a user can only read/write their own tasks; the super admin can read/update any task but a normal user cannot).

366. Copy the generated rules from the `firestore.rules` file in the project and paste them into Firebase's **Firestore Database > Rules** tab, then click **Publish**.
367. Tell Claude "I've done the required setup — please run the app," and it starts a local server (adjusting the port if needed).
368. Test in the browser: sign in as a normal user, add tasks with priority levels (High/Medium/Low), filter by priority and completion status — data persists correctly.
369. Sign in as a **different** user — confirm that user only sees their own tasks (not the first user's).
370. Sign in as the **super admin** (the email specified in the prompt) — confirm they can see all users' tasks combined, and can change another user's task priority (which updates immediately in that user's view).
371. Discussion: whatever specific permission model you want (e.g., "super admin can only view, not edit" or "multiple people can be super admins") must be spelled out explicitly in the prompt — Claude implements exactly what's described.
372. Finally, ask Claude Code to push the project to GitHub and deploy it, the same as previous demos.

Key takeaways

- Test mode = temporary/insecure, meant only for learning; production mode = permanent data with real security rules — always use production mode for anything you intend to keep or share seriously.
- Security/access rules (e.g., "only the owner or a super admin can edit a record") are written as Firestore **rules**, which Claude can generate for you when you describe the desired permission model in your prompt — you then paste them into the Firebase console's Rules tab and publish.
- Role-based logic (e.g., a "super admin" who can view/edit everyone's data while regular users only see their own) is fully achievable by prompting Claude — describe the exact behavior you want.
- Productivity principle repeated throughout: identify your highest-value activities, spend your time only there, and delegate/remove low-value repeated tasks.

Demo 45: Designing an "AI-Powered Problem Solver" App (Using AI Inside an AI-Built App)

This session is more conceptual/design-focused than hands-on coding: the instructor proposes building an app where the *app itself* uses an AI/LLM internally to interview the user about a personal problem and generate a step-by-step, "impossible to fail" action plan. He live-demonstrates the underlying prompting logic using ChatGPT.

What you'll learn

- The idea of building an app that itself calls an AI/LLM (an "AI within AI" application) to solve open-ended personal problems
- A prompting technique for turning a vague personal problem into a concrete, incremental action plan through guided follow-up questions
- How to structure such a prompt: ask clarifying questions one at a time, diagnose the root cause, then produce a very small, sustainable daily action plan
- That this becomes a homework/challenge project for students: build this as a real web app (with Firebase storage, GitHub deployment, and a custom domain) using an LLM API (OpenAI/Claude) as the "brain"

Step-by-step notes

1. The concept

- Everyone has personal problems or unmet goals (health, career, finances, habits, relationships, etc.) — even highly successful/famous people have them, they just don't share them publicly.
- The proposed app should: (1) let the user describe their biggest problem in free text, (2) have the AI ask clarifying questions **one at a time** (not all at once) to understand the real root cause, (3) analyze the answers to identify the *actual* underlying issue (which is often different from the stated problem), and (4) generate a long-term action plan (could span 90 days, 6 months, a year — the AI decides) made of extremely small, easy, "impossible to fail" daily actions.

2. Live demo of the underlying prompting approach (using ChatGPT directly, not yet a built app)

The instructor typed a meta-prompt instructing the AI to act as this kind of problem-solving assistant:

"I have a bigger problem in my life... based on this problem you have to ask some questions to me one by one. By considering the problem and my answers, provide an action plan to solve that problem. The action plan should be very easy to follow — at least 1–2 minutes per day — impossible to fail."

He then walked through a live example using the problem **"I want to wake up at 5 a.m. every day but can't stay consistent":**

373. The AI asked when the person currently sleeps (~1 a.m.), and what their last activity before sleeping is (mobile phone scrolling).

374. It identified the real root cause: not "laziness," but insufficient sleep (only ~4 hours) caused by late-night mobile scrolling — a "night routine" problem, not a discipline problem.
375. It asked why they scroll at night (multiple choice: relaxation / not sleepy / habit / stress / "start for 5 min, becomes 1 hour") — the answer selected was a mix, with "starts for 5 minutes but becomes an hour" as the dominant driver.
376. It asked what the person wants to do in their first 5 a.m. minutes (chose: day planning) and their typical/worst wake-up time (8 a.m.), to gauge the size of the gap (3-hour jump from 8 a.m. to 5 a.m., which is too big to attempt directly).
377. It asked whether the phone could be kept away from the bed — the answer was "very difficult, I need it nearby" (e.g., for the alarm) — so the plan avoided that constraint entirely rather than forcing it.
378. Based on all this, the AI produced a **week-by-week micro action plan**, for example:

- **Week 1:** Wake up only 5 minutes earlier than usual (e.g., 8:00 → 7:55). Each night, write one line in a notebook by the bed: "Tomorrow morning my first 2-minute plan is [3 tasks]." After waking, spend 2 minutes writing today's most important task, one small personal task, and one thing to avoid wasting time on.
- **Week 2:** Reduce another 10–15 minutes (e.g., to 7:45), same night/morning micro-actions repeated to build the habit.
- **Week 3:** Continue reducing gradually (e.g., to 7:30); since keeping the phone away is "very difficult," a compromise step is introduced (e.g., keep the phone nearby but don't pick it up for a fixed short window).
- **Week 4 and beyond:** Continue stepping the wake-time earlier in small increments until 5 a.m. is reached naturally, with the underlying principle: **"Win the night, and the morning will automatically improve."**

379. The key theme throughout: never attempt a big, hard change all at once — break it into the smallest possible daily action that is genuinely "impossible to fail," and let consistency compound over weeks.

3. The assignment for students

- Build this idea as a real, working web application:
- Take the user's problem as free text input.
- Use an AI/LLM API (OpenAI's ChatGPT API or Claude/Anthropic's API) as the "brain" to ask clarifying questions and generate the plan (this requires an API key).
- Store conversation/problem context in a Firebase database (same pattern as prior demos).
- Deploy via GitHub with a custom domain, same as previous apps.
- The instructor stresses this isn't just a coding exercise — it doubles as genuinely solving real problems for users (and for the builder).
- A student ("Vijay") shared a similar app he had already built independently that used an LLM to interview a user about their job search struggles and generate a 30-day/365-day action plan — the

instructor used this as a live example of the same pattern already working (asking about job search steps, uncovering the real blocker "lack of clarity," and generating a daily 5-minute action plan).

Key takeaways

- The described app is an example of building an application that itself calls an LLM/AI API internally — not just an app *created* by AI, but one that *uses* AI as a core feature.
- Effective problem-solving prompts ask **one clarifying question at a time**, dig past the surface-level stated problem to find the real root cause, and then produce a plan made of very small, sustainable, incremental daily actions rather than one big difficult change.
- "Impossible to fail" plans work by making the very first step trivially easy (e.g., "wake up just 5 minutes earlier," not "wake up at 5 a.m. immediately").
- This app requires connecting to an external LLM API (OpenAI or Claude/Anthropic) with an API key, in addition to the Firebase storage and GitHub/custom-domain deployment pattern used in earlier demos.

Demo 46: Automating n8n Workflow Creation and Deployment Directly from Claude Code (MCP + Skills)

This session shows how to let Claude Code build an n8n automation workflow from a plain-English prompt, and then go a step further — automatically publish that workflow directly into a live n8n instance (no manual JSON download/upload) by giving Claude an n8n API key and connecting via MCP, with reusable project instructions stored in a `CLAUDE.md` file.

What you'll learn

- How to ask Claude Code to design an n8n workflow and export it as a JSON file
- How to manually import that JSON into n8n, test it, and publish it to get a live production form URL
- What MCP (Model Context Protocol) is and why it's needed for Claude to talk directly to other applications like n8n
- What a "skill" means for Claude Code, and how a `CLAUDE.md` file acts as a persistent system prompt/project configuration
- How to fully automate workflow creation **and** publishing in n8n directly from a Claude Code prompt, using an n8n API key plus n8n MCP/skill resources

Step-by-step notes

1. Build an n8n workflow via Claude Code and manually import it (first pass)

380. Open VS Code, create/open a new project folder (e.g., "agentic project one"), and enable the Claude Code workspace extension.
381. In a new Claude Code session, describe the desired n8n workflow in a prompt, for example:

"I want to create an n8n workflow with this requirement: student fills a form with their required course and email ID. Visit website durgasoftonline.com's 'New Batches' tab, identify batch information related to the requested course, and send that information by email (use a Gmail node). Provide a JSON file for this workflow."

382. Claude Code asks a couple of clarifying questions (e.g., how the workflow should extract course/batch info from the page — generic scraping vs. specific HTML selectors) and then generates a workflow JSON file in the project folder (e.g., `durgasoft-batch-info-workflow.json`).
383. In n8n, create a new workflow and use **Import from file** to load that JSON — this instantly recreates all the nodes that would otherwise take about an hour to build manually.
384. Fix small details as needed (e.g., confirm/update the target URL, set up Gmail node credentials).
385. **Execute workflow** to test it end-to-end: submit the form fields (name, email, requested course) and confirm the resulting email is received with the batch information.
386. Click **Publish**/activate the workflow to get a **production URL**, and share that form URL so others can use it (and receive their own results by email).

2. Automating the publish step: MCP, skills, and CLAUDE.md

- Manually downloading the JSON from Claude and uploading it into n8n is an extra step the instructor wants to eliminate.
- To let Claude directly create *and* publish into a live n8n instance, three things are needed:

387. **MCP (Model Context Protocol)** — the protocol AI tools use to communicate with other applications/services (here, n8n).
388. An **n8n API key** (n8n Pro/paid account required — free trial accounts don't expose API access for this).
389. n8n-specific **"skill" knowledge** for Claude — reference material about how n8n workflows/nodes work, so Claude doesn't have to re-derive this from scratch every time.

- "Skills" concept: once Claude is given the right reference material once, it can reuse that knowledge automatically on every future request, without you re-explaining it each time — similar to a person having existing subject-matter expertise instead of having to look everything up.
- This project-level reference material is stored in a special file called **`CLAUDE.md`** (Markdown) inside the project folder — Claude Code automatically reads this file every time it works in that project, treating it as a persistent **system prompt** / set of standing instructions.

3. Creating the CLAUDE.md file and connecting to n8n

390. Create a fresh project folder (e.g., "agentic project two") and enable the Claude Code workspace extension for it.
391. In a new Claude Code session, ask Claude to create the `CLAUDE.md` file itself, providing reference URLs, e.g.:

"I want to create a CLAUDE.md file for this project. I will use this to build n8n workflows and AI agents by prompting you. I'll give you the resources: an n8n MCP [GitHub URL] and an n8n skills reference [URL]. These contain everything you need to build n8n workflows. Create a system prompt for this CLAUDE.md file that makes you an expert at creating n8n workflows."

392. Claude fetches/reads those resources and writes a `CLAUDE.md` file containing project instructions — e.g., "You are an expert n8n workflow architect... treat the n8n MCP server as your hands and n8n skills as your expertise..."
393. Note: the CLAUDE.md file only needs to be created **once** per project — Claude Code then reads and applies it on **every** subsequent prompt in that project automatically.
394. Get your **n8n API key**: in your n8n instance, go to **Settings > n8n API > Create an API key**, name it (e.g., "n8n for Claude"), and copy it.
395. In the same Claude Code session, provide your n8n instance URL and the API key, and ask Claude to update the project's persistent configuration with this info (so you don't have to repeat it every time) — or supply it directly in a build prompt.

4. Full end-to-end automated build

396. With `CLAUDE.md` set up (containing n8n MCP/skill info) and the n8n instance URL + API key provided, give Claude a build prompt similar to before:

"I want to create an n8n workflow with this requirement: student fills a form with required course and email ID; visit durgasoftonline.com's New Batches page; identify requested-course batch info; send it by email. Publish this workflow in my n8n instance using the following information: [instance URL / API key already known via CLAUDE.md]."

397. Claude now uses the n8n MCP connection to: generate the workflow, call the n8n REST API to **create the workflow directly inside your n8n instance**, and **activate/publish it** — all without any manual JSON download or import.
398. Claude can also test the workflow itself: it submits a sample form entry, checks execution logs/results, and reports back whether it worked (e.g., diagnosing why a node failed and retrying).
399. The finished workflow appears live in the n8n instance under its own name (e.g., "batch info request"), fully published, with a working public form URL and Gmail delivery — all triggered from a single prompt.

5. Clarifications / Q&A covered

- The n8n MCP and n8n skill URLs given to Claude are **not** built-in knowledge — they're "extra specialized skills" explicitly supplied by the user, expanding what Claude already knows.
- `CLAUDE.md` content is the **system prompt** (standing instructions Claude always applies for this project); the specific "build this workflow" request each time is the **user prompt** (task-specific, changes every time).
- Content is only saved into `CLAUDE.md` when you explicitly ask Claude to save/update that file — simply mentioning URLs in the chat does not automatically persist them there.
- This entire flow requires an **n8n paid/Pro account** for API key access; free-tier accounts must still fall back to manually exporting/importing the JSON.
- Using VS Code (rather than the Claude Code desktop app) is convenient here because you can directly open and inspect the `CLAUDE.md` file and project folder structure, which is harder to do from the desktop-only app.
- A student also demoed a related project: an AI-driven "problem solver" chat app (following on from Demo 45's concept) that interviews a user about a job-search struggle and produces a long-term (e.g., 365-day, 5-minutes/day) action plan — reinforcing that these AI-interview-style apps are a practical, recurring pattern.

Key takeaways

- Claude Code can generate a complete n8n workflow JSON from a plain-English description — this alone saves the hour or more it would take to hand-build nodes.
- MCP is the protocol that lets Claude directly control other applications (like n8n) instead of just producing files for you to move manually — this requires an API key for the target application (n8n API key, requiring a paid n8n plan).
- A `CLAUDE.md` file in your project acts as a reusable system prompt: define it once (with reference resources/URLs and standing instructions), and Claude automatically applies that knowledge to

every future prompt in that project — this is the practical meaning of a Claude Code "skill" for a project.

- With MCP + an API key + the right CLAUDE.md instructions, the entire n8n workflow lifecycle — design, build, upload, activate, and test — can be triggered and completed from a single natural-language prompt, with no manual steps in n8n's UI at all.